

□□□□□□AWS□□

□□ □□ (Tomoyuki Mano) <tomoyukimano@gmail.com>

Version 1.0, 2021-06-14

Table of Contents

1. 序	2
1.1. 本書の目的	2
1.2. 本書の構成	2
1.3. AWSの紹介	3
1.4. 環境	3
1.5. 前提知識	4
1.6. 開発環境	4
1.7. 開発環境の構築	4
2. 基礎	6
2.1. 基礎	6
2.2. 基礎	8
3. AWS	11
3.1. AWS	11
3.2. AWS	11
3.3. Region と Availability Zone	13
3.4. AWS	14
3.5. CloudFormation と AWS CDK	18
4. Hands-on #1: EC2	22
4.1. 序	22
4.2. SSH	22
4.3. 基礎	23
4.4. 基礎	27
4.5. 序	33
5. 基礎	34
5.1. 基礎	34
5.2. GPU	34
6. Hands-on #2: AWS	38
6.1. 序	38
6.2. 基礎	38
6.3. 基礎	41
6.4. 基礎	42
6.5. Jupyter Notebook	44
6.6. PyTorch	46
6.7. 基礎! MNIST	49
6.8. 基礎	54
7. Docker	56
7.1. 基礎	56
7.2. Docker	57

7.3. Docker	59
7.4. Elastic Container Service (ECS)	62
8. Hands-on #3: AWS	64
8.1. Fargate	64
8.2.	65
8.3. Transformer question-answering	65
8.4.	67
8.5.	71
8.6.	73
8.7.	74
8.8.	76
9. Hands-on #4: AWS Batch	77
9.1. Auto scaling groups (ASG)	77
9.2. AWS Batch	77
9.3.	79
9.4. MNIST (MNIST)	79
9.5.	82
9.6.	86
9.7. Docker image ECR	88
9.8.	91
9.9. Job	95
9.10.	99
9.11.	100
10. Web	102
10.1. — Twitter	102
10.2. REST API	103
10.3. Twitter API	104
11. Serverless architecture	106
11.1. Serverful (Serverful)	106
11.2. Serverless	107
11.3.	108
12. Hands-on #5:	113
12.1. Lambda	113
12.2. DynamoDB	118
12.3. S3	122
13. Hands-on #6: Bashoutter	126
13.1.	126
13.2.	127
13.3.	134
13.4. API	136
13.5. API	138

13.6. Bashoutter GUI	139
13.7.	140
13.8.	141
14.	142
15. Appendix:	143
15.1. AWS	143
15.2. AWS	147
15.3. AWS CLI	149
15.4. AWS CDK	150
15.5. WSL	151
15.6. Docker	154
15.7. Python <code>venv</code>	155
15.8. Docker image	156
16.	158
17.	159
18.	160

~~~~~

<https://github.com/tomomano/learn-aws-by-coding>

~~~~~

~~~~~  
(1000000)~~~~~20210927~~~~~  
~~~~~

~~~~~: AWS~~~~~ (~~~~~360~~~~~)

- Amazon (~~~~~ or Kindle) ⇒ <https://www.amazon.co.jp/dp/4839977607/>
- ~~~~~ (~~~~~ or PDF) ⇒ <https://book.mynavi.jp/ec/products/detail/id=124113>

CDK v2 ~~~~~

~~~~~~~CDK v1~~~~~~~ ~~AWS 20230601~~ ~~v1~~~~~~~~~CDK v2~~~~~~~ (~~~~~)

~~~~~~~@takashi-uchida~~~~~~~~~CDK v2~~~~~~~~~(PR#49)~~~~~~~  
~~[https://github.com/takashi-uchida/learn-aws-by-coding/tree/feature/to\\_cdkv2](https://github.com/takashi-uchida/learn-aws-by-coding/tree/feature/to_cdkv2)~~

~~~~~

(2026/04/18 ~~~) @takashi-uchida ~~~ CDK v2 ~~~ PR#49 ~~~~~ CDK v2
~~~~~ Issue ~~~~~

~~~~~ ~~~ ~~~~~

Chapter 1. 序論

1.1. 環境構築

本書は2021年S1/S2版の「環境構築」をベースに、

本書はAmazon Web Services (AWS) を利用して、

本書は、AWS を利用して、

本書は、

Table 1. 環境構築

| | 環境 | 環境構築 |
|------------|------|--|
| 環境 (1-4) | 環境構築 | • AWS環境構築 |
| 環境 (5-9) | 環境構築 | • AWS 上 Jupyter Notebook 環境構築
• 環境構築
• 環境構築 |
| 環境 (10-13) | 環境構築 | • Lambda, DynamoDB, S3 環境
• SNS "Bashoutter" 環境 |

本書は、AWS を利用して、

本書は、AWS を利用して、

本書は、AWS を利用して、

本書は、AWS を利用して、

1.2. 環境構築

本書は「環境構築」をベースに、

本書は、

本書は「環境構築」をベースに、

- **UNIX** ユーザー: ユーザー名を指定して SSH 接続して、UNIX ユーザー名を指定して Mac 側 Linux 環境に OS をインストールする(インストール済み)環境に Windows 環境を [Windows Subsystem for Linux \(WSL\)](#) として Linux 環境をインストールする ([Section 15.5](#) 参照)
- **Docker**: Docker をインストールして、[Section 15.6](#) を参照
- **Python**: Version 3.6 をインストールして `venv` をインストールして `venv` を[Section 15.7](#) を参照
- **Node.js**: version 12.0 以上をインストール
- **AWS CLI**: [Version 2](#) をインストールして [Section 15.3](#) を参照
- **AWS CDK**: Version 1.100 をインストールして Version 2 をインストールして [Section 15.4](#) を参照
- **AWS** 環境: AWS API をインストールして (secret key) をインストールして [Section 15.3](#) を参照

1.4.1. Docker Image

Python, Node.js, AWS CDK などのアプリケーション/インフラコードを Docker image にビルドして Docker 上で実行する。

1111111111111111

```
$ docker run -it tomomano/labc
```

□□ Docker image □□□□□□□□ [Section 15.8](#) □□□□□□□□

1.5. □□□□

(OS)

- **Python** 書籍タイトル: Pythonの教科書
Pythonの教科書の目次を参照してください。 Python の教科書の目次
- **Linux** 書籍タイトル: Linux のコマンドライン Linux Linux
Linux のコマンドラインの目次を参照してください。 The Linux Command Line by William Shotts
Linux のコマンドラインの目次を参照してください。

1.6. □□□□□□□□

□ □

<https://github.com/tomomano/learn-aws-by-coding>

1.7. □□□□□□□□□□□□□□□□

- `monospace letter` `space`
- `monospace letter` `space` `$` `monospace letter` `$` `monospace letter` `&` `monospace letter` `space` `monospace letter` `$` `monospace letter` `space`

□□□□□□□□□□□□□□□□□□□□



□□□□□□□□□□□□



□□□□□□□□□□□□□□□□□□



□□□□□□□□□□□□□□□□



□□□□□□□□□□□□□□□□

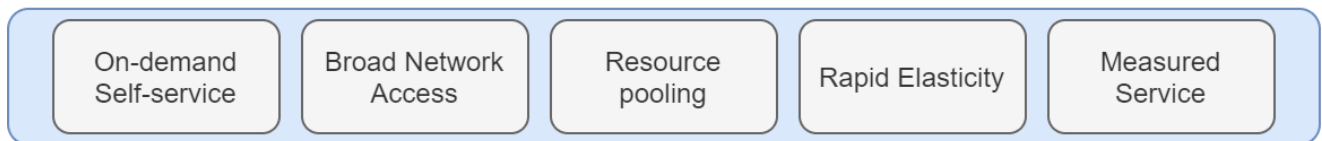
Chapter 2. □□□□□□

2.1. □□□□□□□

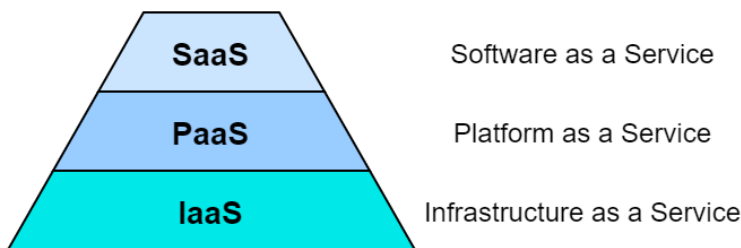
[illegible]

Figure 2: The NIST Definition of Cloud Computing

Essential characteristics



Service models



Deployment models

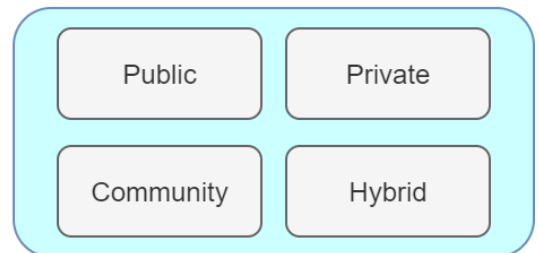


Figure 2. The NIST Definition of Cloud Computing

[illegible]

- **On-demand self-service** 用戶可以隨時隨地自助獲取雲端資源
- **Broad network access** 用戶可以通過多種網絡訪問雲端資源
- **Resource pooling** 用戶可以共享雲端資源池中的資源
- **Rapid elasticity** 用戶可以根據需要快速擴展或縮減雲端資源
- **Measured service** 用戶可以根據實際使用的資源量支付費用

...

CPU CPU CPU


```
Macintosh HD — top — 80x24
Processes: 210 total, 2 running, 9 stuck, 199 sleeping, 901 threads 23:30:03
Load Avg: 1.40, 1.75, 1.00 CPU usage: 4.15% user, 4.40% sys, 91.44% idle
SharedLibs: 1648K resident, 0B data, 0B linkedit,
MemRegions: 31278 total, 1892M resident, 117M private, 564M shared.
PhysMem: 5893M used (1191M wired), 10G unused.
VM: 523G vsize, 1026M framework vsize, 0(0) swapins, 0(0) swapouts.
Networks: packets: 12105/8925K in, 11907/1964K out.
Disks: 00156/2205M read, 21235/425M written.

PID COMMAND %CPU TIME #TH #WQ #PORT MEM PURG CMPR PGRP PPID
592 screencaptur 0.0 00:00.02 7 5 55+ 1952K+ 20K+ 0B 262 262
590 mdworker 0.0 00:00.01 3 0 44 2032K 0B 0B 590 1
589 mdworker 0.0 00:00.01 3 0 44 1572K 0B 0B 589 1
588 top 1.7 00:00.51 1/1 0 22+ 2860K 0B 0B 588 584
584 bash 0.0 00:00.00 1 0 15 588K 0B 0B 584 583
583 login 0.0 00:00.01 3 1 28 1228K 0B 0B 583 402
574 auditd 0.0 00:00.00 2 0 25 560K 0B 0B 574 1
567 System Prefe 0.0 00:03.23 3 0 270 39M 8364K 0B 567 1
561 systemstatsd 0.0 00:00.01 2 1 19 1040K 0B 0B 561 1
560 com.apple.We 0.0 00:01.42 9 0 229 25M 0B 0B 560 1
558 com.apple.We 0.0 00:05.07 15 3 224 151M 1716K 0B 558 1
555 bash 0.0 00:00.00 1 0 15 604K 0B 0B 555 554
554 login 0.0 00:00.01 3 1 28 1176K 0B 0B 554 482
550 bash 0.0 00:00.00 1 0 15 608K 0B 0B 550 549
```

터미널을 실행하는 방법

터미널을 실행하는 방법

터미널을 실행하는 방법

터미널을 실행하는 방법...

터미널

Terminal

터미널을 실행하는 방법Terminal

Windows와 Mac에서 터미널을 실행하는 방법

터미널을 실행하는 방법

터미널을 실행하는 방법"터미널"을 실행하는 방법

Chapter 3. AWS

3.1. AWS

Amazon은 클라우드 컴퓨팅 서비스인 AWS를 통해 기업들에게 다양한 클라우드 서비스를 제공합니다.

AWS (Amazon Web Services)는 Amazon이 제공하는 클라우드 서비스입니다. AWS는 Amazon이 2006년에 시작했으며, 2021년에는 미국에서 총 매출의 32%를 차지했습니다. AWS는 Netflix, Slack, Amazon 등 다양한 기업들이 사용하고 있습니다. AWS는 기업들에게 다양한 클라우드 서비스를 제공합니다.

AWS는 기업들에게 다양한 클라우드 서비스를 제공합니다. AWS는 기업들에게 다양한 클라우드 서비스를 제공합니다. AWS는 기업들에게 다양한 클라우드 서비스를 제공합니다.

3.2. AWS 서비스 카테고리

Figure 4 AWS 서비스 카테고리

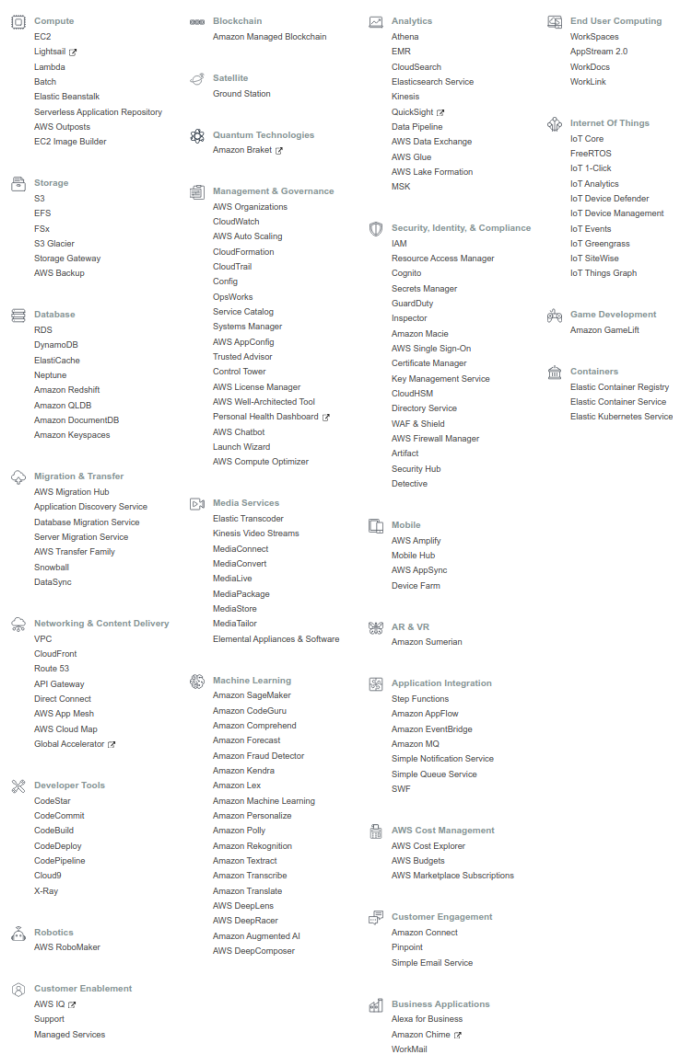


Figure 4. AWS 서비스 카테고리

이 다이어그램은 AWS 서비스의 다양한 카테고리를 보여줍니다. 각 카테고리는 아이콘과 함께 나열된 서비스 목록을 포함하고 있습니다.

アプリケーションを構築する際に必要なサービス

アプリケーションを構築する際に必要なサービス AR/VR アプリケーションを構築する際に必要なサービス 170アプリケーションを構築する際に必要なサービス (図)

AWS アプリケーションを構築する際に必要なサービス アプリケーションを構築する際に必要なサービス アプリケーションを構築する際に必要なサービス AWS アプリケーションを構築する際に必要なサービス AWS アプリケーションを構築する際に必要なサービス AWS アプリケーションを構築する際に必要なサービス AWS アプリケーションを構築する際に必要なサービス

アプリケーションを構築する際に必要なサービス AWS アプリケーションを構築する際に必要なサービス アプリケーションを構築する際に必要なサービス

3.2.1. EC2



EC2 (Elastic Compute Cloud) アプリケーションを構築する際に必要なサービス アプリケーションを構築する際に必要なサービス Chapter 4, Chapter 6, Chapter 9



Lambda Function as a Service (FaaS) アプリケーションを構築する際に必要なサービス アプリケーションを構築する際に必要なサービス (Chapter 11)

3.2.2. EBS



EBS (Elastic Block Store) EC2アプリケーションを構築する際に必要なサービス アプリケーションを構築する際に必要なサービス (OS) アプリケーションを構築する際に必要なサービス



S3 (Simple Storage Service) Object Storage アプリケーションを構築する際に必要なサービス ”アプリケーションを構築する際に必要なサービス” アプリケーションを構築する際に必要なサービス (Chapter 11)

3.2.3. DynamoDB



DynamoDB NoSQL アプリケーションを構築する際に必要なサービス (MongoDB) アプリケーションを構築する際に必要なサービス (Chapter 11)

3.2.4. VPC



VPC (Virtual Private Cloud) AWS アプリケーションを構築する際に必要なサービス EC2 アプリケーションを構築する際に必要なサービス VPC アプリケーションを構築する際に必要なサービス

API Gateway API アプリケーションを構築する際に必要なサービス (Lambda) アプリケーションを構築する際に必要なサービス Chapter 13

3.3. Region & Availability Zone

AWS 提供多個地理區域 (Region) 及 Availability Zone (AZ) (Figure 5) 以提供高可用性與容錯能力。

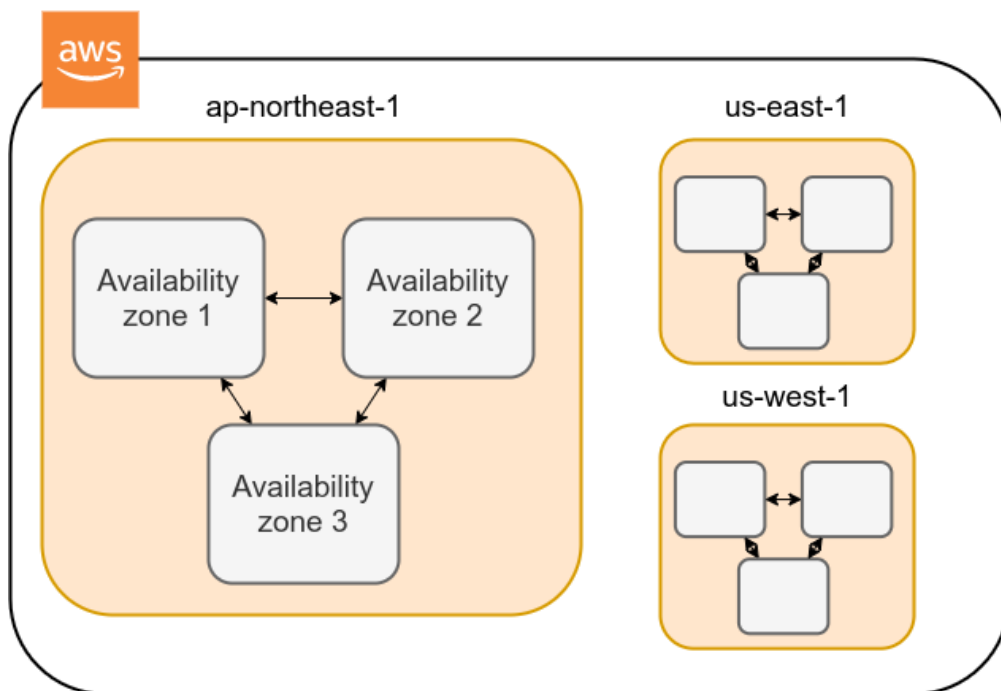


Figure 5. AWS Region & Availability Zones

每個 Region 包含多個 Availability Zones。AWS 目前提供 25 個 Region，如 Figure 6 所示。每個 Region 都有唯一的 ID，例如 `ap-northeast-1`，而 `us-east-2` 則是即將推出的 Region。



Figure 6. Regions in AWS (URL: <https://aws.amazon.com/about-aws/global-infrastructure/>)

AWS 提供多種服務，包括 EC2、S3 (Figure 7, 雲端服務) 等。AWS 服務在全球多個 Region 中可用。

為了確保服務的高可用性，建議在多個 Availability Zones 中部署服務 (i.e. 多可用區部署)。

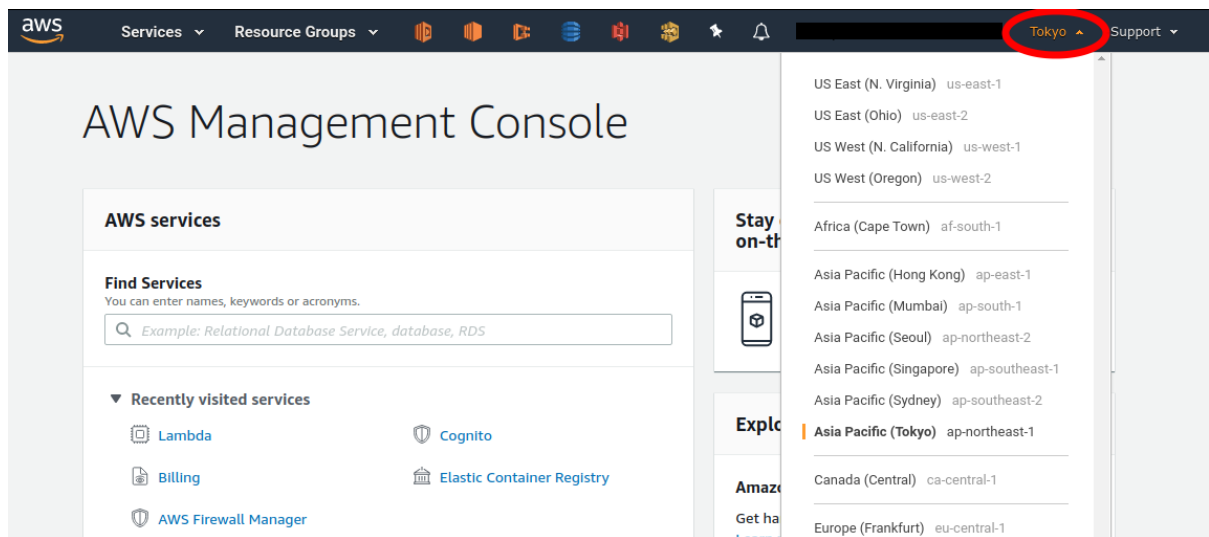


Figure 7. AWS Management Console

Availability Zone (AZ) is a physically separate location within the same Region, designed to be isolated from failures in other Availability Zones. Each Region has at least two Availability Zones, and some Regions have more than two. Availability Zones are independent of each other, so that if one Availability Zone experiences an outage, your application can continue to run in the other Availability Zones. You can design your architecture to be resilient to Availability Zone failures by distributing your resources across multiple Availability Zones in the same Region.



AWS

AWS provides a variety of services that are available in multiple Availability Zones. For example, Amazon EC2 instances can be launched in multiple Availability Zones to ensure high availability. AWS also provides services that are available in a single Availability Zone, such as Amazon ElastiCache. When you design your architecture, you should consider the availability of the services you are using and how you can ensure high availability for your application.



AWS Educate Starter Account

When you create an AWS Educate Starter Account, you are automatically assigned to the **us-east-1** region. This region is the default region for all AWS services that you use in your account. You can change the default region for your account, but you must be careful to update all of your resources to the new region.

Further reading

- [AWS documentation "Regions, Availability Zones, and Local Zones"](#)

3.4. AWS Regions and Availability Zones

AWS provides a variety of services that are available in multiple Availability Zones. For example, Amazon EC2 instances can be launched in multiple Availability Zones to ensure high availability.

AWS also provides services that are available in a single Availability Zone, such as Amazon ElastiCache. When you design your architecture, you should consider the availability of the services you are using and how you can ensure high availability for your application.

3.4.1. Regions and Availability Zones

AWS provides a variety of services that are available in multiple Availability Zones. For example, Amazon EC2 instances can be launched in multiple Availability Zones to ensure high availability.

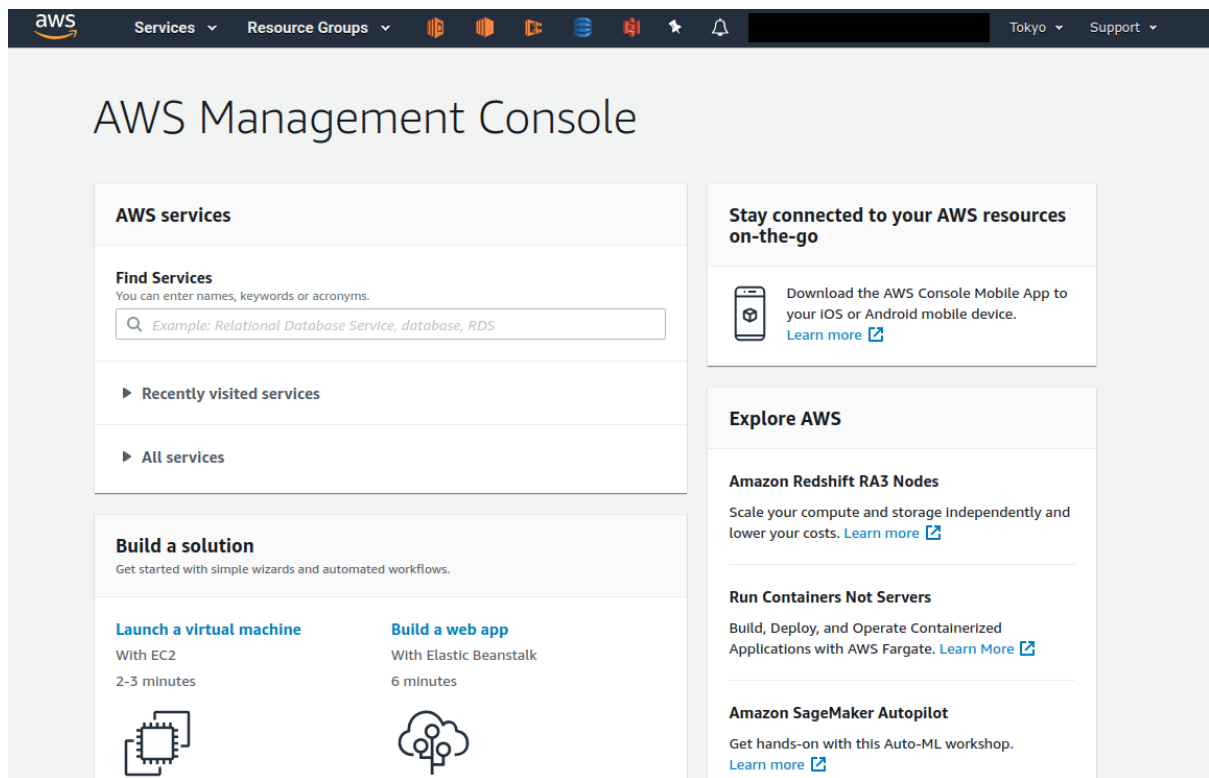


Figure 8. AWS

EC2 S3 AWS GUI (Graphical User Interface)

API AWS AWS

3.4.2. API

API (Application Programming Interface) AWS API GET, POST, DELETE REST API (REST API Section 10.2) REST API

AWS CLI UNIX AWS API CLI (Command Line Interface) CLI SDK (Software Development Kit)

- Python ⇒ boto3
- Ruby ⇒ AWS SDK for Ruby
- Node.js ⇒ AWS SDK for Node.js

API

S3 (Bucket) AWS CLI

```
$ aws s3 mb s3://my-bucket --region ap-northeast-1
```

my-bucket ap-northeast-1

Python boto3

```
1 import boto3
2
3 s3_client = boto3.client("s3", region_name="ap-northeast-1")
4 s3_client.create_bucket(Bucket="my-bucket")
```

EC2

```
$ aws ec2 run-instances --image-id ami-xxxxxxx --count 1 --instance-type t2.micro
--key-name MyKeyPair --security-group-ids sg-903004f8 --subnet-id subnet-6e7f829e
```

t2.micro (1 vCPU, 1.0 GB RAM) (Chapter 4)

Python

```
1 import boto3
2
3 ec2_client = boto3.client("ec2")
4 ec2_client.run_instances(
5     ImageId="ami-xxxxxxxx",
6     MinCount=1,
7     MaxCount=1,
8     KeyName="MyKeyPair",
9     InstanceType="t2.micro",
10    SecurityGroupIds=["sg-903004f8"],
11    SubnetId="subnet-6e7f829e",
12 )
```

API

CPU RAM

API

3.4.3. AWS CLI

AWS CLI AWS CLI S3 (EC2) aws s3



AWS CLI Section 15.3



S3



~~~~~AWS~~~~~  
 ~~~~~  
 (AWS_ACCESS_KEY_ID,
 AWS_DEFAULT_REGION) ~~~~~ Section 15.3 ~~~~~

~/.aws/credentials
 AWS_SECRET_ACCESS_KEY,

~~~~~S3~~~~~ (Bucket ~~~~~ OS ~~~~~) ~~~~~

```
$ bucketName="mybucket-$(openssl rand -hex 12)"
$ echo $bucketName
$ aws s3 mb "s3://${bucketName}"
```

S3~~~~~ AWS ~~~~~bucketName ~~~~~  
 ~~~~~ aws s3 mb (mb ~ make bucket ~) ~~~~~

~~~~~

```
$ aws s3 ls

2020-06-07 23:45:44 mybucket-c6f9385550a72b5b66f5efe
```

~~~~~



~~~~~  
 ~~~~~&~~~~~ ~~~~~ \$ ~~~~~

\$

~~~~~

\$

~~~~~

```
$ echo "Hello world" > hello_world.txt
$ aws s3 cp hello_world.txt "s3://${bucketName}/hello_world.txt"
```

~~~~~ hello\_world.txt ~~~~~

~~~~~

```
$ aws s3 ls "s3://${bucketName}" --human-readable

2020-06-07 23:54:19    13 Bytes hello_world.txt
```

~~~~~

~~~~~

```
$ aws s3 rb "s3://${bucketName}" --force
```

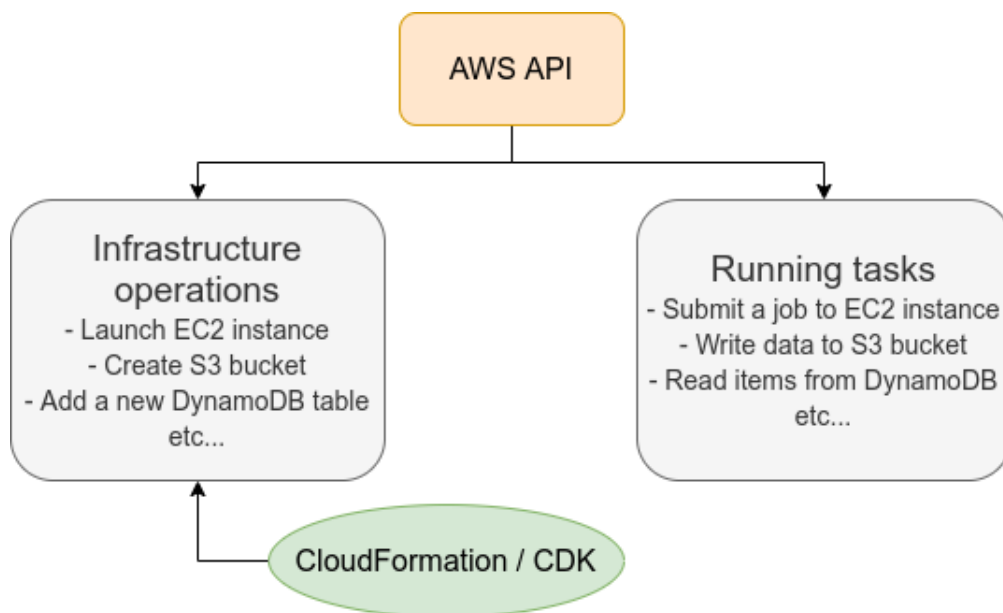
rb ~ remove bucket ~~~~~ ~~~~~ --force

1. **インストール**
 AWS CLI をインストールして、S3、EC2、Lambda、DynamoDB などの AWS サービスと連携するためのコマンドを実行する。

S3 EC2 ARN ARN

3.5.1. CloudFormation ☐☐☐ Infrastructure as Code (IaC)

Figure 9: AWS API request rate (Figure 9)



1. **EC2 实例与 S3 存储桶的 IAM 角色**
 2. **配置 IAM 策略**

18

AWS CloudFormation CloudFormation CloudFormation
AWS CloudFormation
EC2 CloudFormation
CloudFormation AWS
CloudFormation
Infrastructure as Code (IaC)

CloudFormation 使用 **JSON** (JavaScript Object Notation) 来描述云资源。JSON 是一种轻量级的数据交换格式，易于人阅读和机器生成/解析。

```

1  "Resources" : {
2      ...
3      "WebServer": {
4          "Type" : "AWS::EC2::Instance",
5          "Properties": {
6              "ImageId" : { "Fn::FindInMap" : [ "AWSRegionArch2AMI", { "Ref" :
7                  "AWS::Region" },
8                      { "Fn::FindInMap" : [ "AWSInstanceType2Arch", { "Ref" :
9                          "InstanceType" }, "Arch" ] } ] } },
10             "InstanceType" : { "Ref" : "InstanceType" },
11             "SecurityGroups" : [ { "Ref" : "WebServerSecurityGroup" } ],
12             "KeyName" : { "Ref" : "KeyName" },
13             "UserData" : { "Fn::Base64" : { "Fn::Join" : [ "", [
14                 "#!/bin/bash -xe\n",
15                 "yum update -y aws-cfn-bootstrap\n",
16                 "\n/opt/aws/bin/cfn-init -v ",
17                 "    --stack ", { "Ref" : "AWS::StackName" },
18                 "    --resource WebServer ",
19                 "    --configsets wordpress_install ",
20                 "    --region ", { "Ref" : "AWS::Region" }, "\n",
21                 "\n/opt/aws/bin/cfn-signal -e $? ",
22                 "    --stack ", { "Ref" : "AWS::StackName" },
23                 "    --resource WebServer ",
24                 "    --region ", { "Ref" : "AWS::Region" }, "\n"
25             ] } } }
26         },
27         ...
28     },
29     ...
30 },

```

```
"WebServer"                                EC2
```

```
OS EC2
```

3.5.2. AWS CDK

[illegible]

AWS Cloud Development Kit (CDK) を用いて CDK を Python を利用して CloudFormation を利用して CDK を 2019 年にリリースされた (GitHub) の CDK を TypeScript (JavaScript), Python, Java などを利用して

CDKとCloudFormation

Python による CDK のインストール (2)

```

1 import aws_cdk as cdk
2 from aws_cdk import (
3     Stack,
4     aws_ec2 as ec2,
5 )
6
7 class MyFirstEc2(Stack):
8
9     def __init__(self, scope, construct_id, **kwargs):
10         super().__init__(scope, construct_id, **kwargs)
11
12         vpc = ec2.Vpc(
13             ... # some parameters
14         )
15
16         sg = ec2.SecurityGroup(
17             ... # some parameters
18         )
19
20         host = ec2.Instance(
21             self, "MyGreatEc2",
22             instance_type=ec2.InstanceType("t2.micro"),
23             machine_image=ec2.MachineImage.latest_amazon_linux(),
24             vpc=vpc,
25             ...
26         )

```

JSON CloudFormation CloudFormation
CDK Python

CDK **AWS** **CDK**

Further reading

- [AWS CDK Examples](#): CDKのサンプルコードをまとめたリポジトリ。AWS CDKの基本的な使用法や、様々なAWSサービスを利用するためのサンプルコードが豊富にある。

Chapter 4. Hands-on #1:

EC2

この章では、AWS Cloud Development Kit (CDK) を使用して、Amazon EC2 インスタンスを構築し、SSH で接続できるようにします。

4.1. 準備

この章のコードは GitHub の [handson/ec2-get-started](https://github.com/tomomano/ec2-get-started) リポジトリにあります。



この章のコードは AWS のリポジトリにあります。

この章のコードは AWS のリポジトリにあります。

- **AWS Account:** AWS アカウントをお持ちの場合は、AWS の [Section 15.1](#) を参照してください。
- **Python** と **Node.js:** Python (3.6 以上) と Node.js (12.0 以上) をインストールしてください。
- **AWS CLI:** AWS CLI をインストールしてください。 [Section 15.3](#) を参照してください。
- **AWS CDK:** AWS CDK をインストールしてください。 [Section 15.4](#) を参照してください。
- **Git:** Git をインストールしてください。 GitHub の [Git](#) を参照してください。

```
$ git clone https://github.com/tomomano/learn-aws-by-coding.git
```

この章のコードは <https://github.com/tomomano/learn-aws-by-coding> リポジトリにあります。

Docker

Python, Node.js, AWS CDK をインストールする必要がある場合、/usr/bin/ を参照してください。 Docker image をインストールする必要がある場合は、Docker をインストールしてください。

この章のコードは [Section 15.8](#) を参照してください。

4.2. SSH

SSH (secure shell) は Unix 系オペレーティングシステムで広く使われている、リモートホストに接続するためのプロトコルです。

SSH は、Linux/Mac などのオペレーティングシステムで広く使われている、リモートホストに接続するためのプロトコルです。 Windows などのオペレーティングシステムでは、WSL (Windows Subsystem for Linux) を使用して SSH を実行できます。

SSH を実行するには、`<host name>` (ホスト名) と `<user name>` (ユーザー名) を指定する必要があります。

```
$ ssh <user name>@<host name>
```

SSH (Public Key Cryptography) EC2 (Public key)(Private key) EC2 (Public key) (Private key) EC2

SSH `-i` `--identity_file`

```
$ ssh -i Ec2SecretKey.pem <user name>@<host name>
```

4.3.

Figure 10

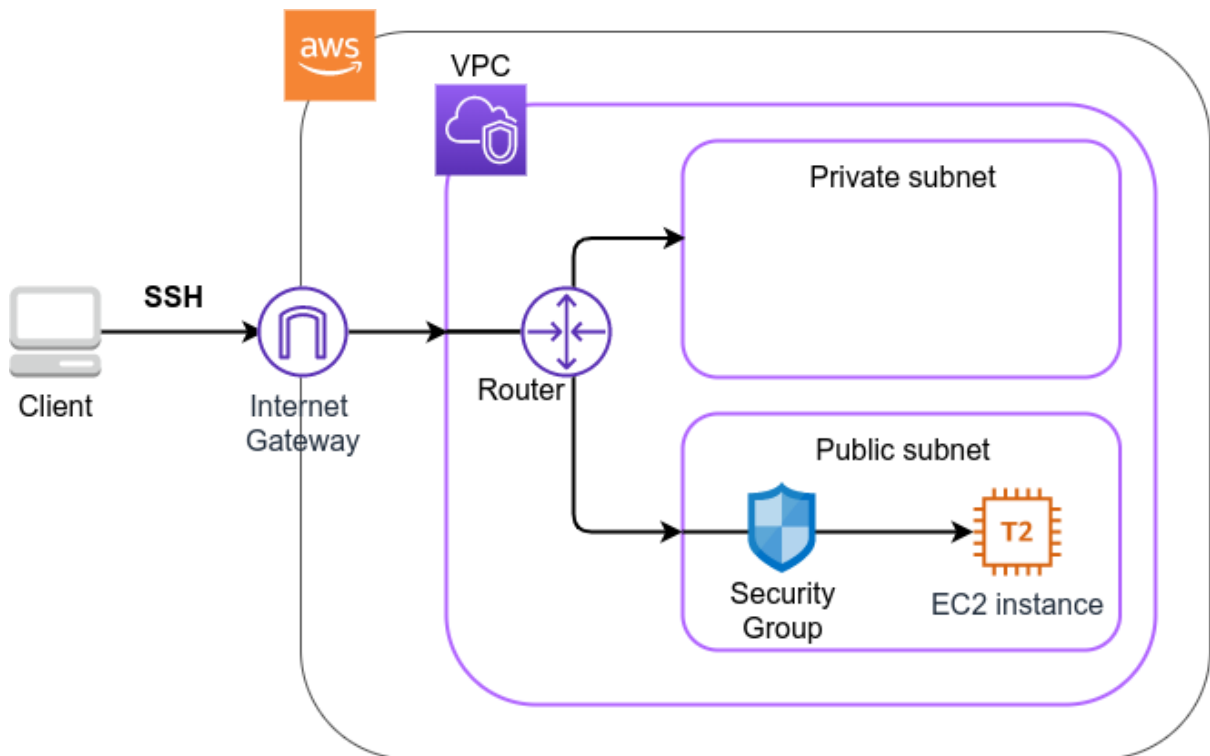


Figure 10. #1

VPC (Virtual Private Cloud) public subnet EC2 (Elastic Compute Cloud) Security Group EC2 SSH

Figure 10 CDK

([handson/ec2-get-started/app.py](https://github.com/hands-on-aws/ec2-get-started/app.py))

```
1 class MyFirstEc2(Stack):
```

```

2
3     def __init__(self, scope: Construct, construct_id: str, key_name: str, **
kwargs) -> None:
4         super().__init__(scope, construct_id, **kwargs)
5
6         ①
7         vpc = ec2.Vpc(
8             self, "MyFirstEc2-Vpc",
9             max_azs=1,
10            ip_addresses=ec2.IpAddresses.cidr("10.10.0.0/23"),
11            subnet_configuration=[
12                ec2.SubnetConfiguration(
13                    name="public",
14                    subnet_type=ec2.SubnetType.PUBLIC,
15                )
16            ],
17            nat_gateways=0,
18        )
19
20        ②
21        sg = ec2.SecurityGroup(
22            self, "MyFirstEc2Vpc-Sg",
23            vpc=vpc,
24            allow_all_outbound=True,
25        )
26        sg.add_ingress_rule(
27            peer=ec2.Peer.any_ipv4(),
28            connection=ec2.Port.tcp(22),
29        )
30
31        ③
32        host = ec2.Instance(
33            self, "MyFirstEc2Instance",
34            instance_type=ec2.InstanceType("t2.micro"),
35            machine_image=ec2.MachineImage.latest_amazon_linux(),
36            vpc=vpc,
37            vpc_subnets=ec2.SubnetSelection(subnet_type=ec2.SubnetType.PUBLIC),
38            security_group=sg,
39            key_name=key_name
40        )

```

① VPC

② security group (SG) 22 (SSH) 4

③ VPC SG EC2 t2.micro Amazon Linux OS

4.3.1. VPC (Virtual Private Cloud)

VPC □□□□□



VPC

1

AWS

VPC

□□□□□□□□□□□□□□□□□□□□VPC□□□□□□□□□□□□□□□□□□□□EC2□□□□□□□□□□VPC□□□□□□□□□□□□□□□□□□□□
□□□□□□□□□□□□□□□□□□□□VPC□□□□□□□□

VPC

```
1 vpc = ec2.Vpc(  
2     self, "MyFirstEc2-Vpc",  
3     max_azs=1,  
4     ip_addresses=ec2.IpAddresses.cidr("10.10.0.0/23"),  
5     subnet_configuration=[  
6         ec2.SubnetConfiguration(  
7             name="public",  
8             subnet_type=ec2.SubnetType.PUBLIC,  
9         )  
10    ],  
11    nat_gateways=0,  
12 )
```



- **max_azs=1** : 可用性ゾーン (AZ) 1 個を指定
可用性ゾーンを指定するオプションは、1 個を指定
- **cidr="10.10.0.0/23"** : VPC の IPv4 CIDR ブロックを指定
[Wikipedia](#) 10.10.0.0/23 の IP アドレス範囲は 10.10.0.0 から 10.10.1.255 までです。512 個のサブネットを VPC に作成し、512 個の IPv4 アドレス範囲を VPC に作成し、512 個の VPC に作成
- **subnet_configuration=...** : VPC のサブネットを指定
private subnet と **public subnet** を指定する。private subnet は VPC の EC2 インスタンスに接続するための SSH を許可する Public subnet を指定する。サブネットの IP アドレス範囲は 10.10.0.0/23 です。
- **natgateways=0** : NAT Gateway (インターネットゲートウェイ) を 0 個指定する。NAT Gateway を指定する。

4.3.2. Security Group

Security group (SG) は EC2 インスタンスに適用されるネットワークセキュリティグループです。IP アドレス範囲 (CIDR ブロック) を指定して、特定の IP アドレス範囲からのトラフィックを許可または拒否します。

以下は、SG を作成する Python コードです。

```
1 sg = ec2.SecurityGroup(
2     self, "MyFirstEc2Vpc-Sg",
3     vpc=vpc,
4     allow_all_outbound=True,
5 )
6 sg.add_ingress_rule(
7     peer=ec2.Peer.any_ipv4(),
8     connection=ec2.Port.tcp(22),
9 )
```

このコードは、SSH (ポート 22) を許可する SG を作成します。sg.add_ingress_rule(peer=ec2.Peer.any_ipv4(), connection=ec2.Port.tcp(22)) は、任意の IPv4 アドレスからのポート 22 の TCP トラフィックを許可します。allow_all_outbound=True は、EC2 インスタンスから任意の IP アドレスへの任意のポートへのトラフィックを許可します。



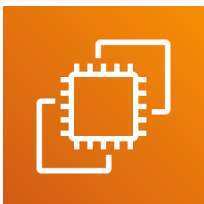
SSH はポート 22 で実行されます。



このコードは、SSH を許可する SG を作成します。

4.3.3. EC2 (Elastic Compute Cloud)

EC2 インスタンス



EC2 は AWS のクラウドコンピューティングサービスです。仮想マシン (instance) を作成し、実行します。

EC2 のインスタンスタイプは [Table 2](#) に示されています。 (Table 2) EC2 インスタンスタイプは ["Amazon EC2 Instance Types"](#) を参照してください。

Table 2. EC2 instance types

Instance	vCPU	Memory (GiB)	Network bandwidth (Gbps)	Price per hour (\$)
t2.micro	1	1	-	0.0116
t2.small	1	2	-	0.023

Instance	vCPU	Memory (GiB)	Network bandwidth (Gbps)	Price per hour (\$)
t2.medium	2	4	-	0.0464
c5.24xlarge	96	192	25	4.08
c5n.18xlarge	72	192	100	3.888
x1e.16xlarge	64	1952	10	13.344

Table 2 EC2 インスタンスの CPU、メモリ、ネットワーク帯域幅の比較。Table 2 は、AWS の EC2 インスタンスの CPU、メモリ、ネットワーク帯域幅の比較を示しています。Table 2 は、AWS の EC2 インスタンスの CPU、メモリ、ネットワーク帯域幅の比較を示しています。Table 2 は、AWS の EC2 インスタンスの CPU、メモリ、ネットワーク帯域幅の比較を示しています。



AWS の EC2 インスタンスの t2.micro は、750 時間無料で利用できます。



Table 2 は、us-east-1 リージョンの EC2 インスタンスの価格を示しています。



t2.micro は、\$0.0116 / hour の On-demand インスタンスです。EC2 インスタンスには、Spot Instance というオプションがあります。Spot Instance は、AWS の EC2 インスタンスの Spot Instance を利用するためのオプションです。Spot Instance は、AWS の EC2 インスタンスの Spot Instance を利用するためのオプションです。Spot Instance は、AWS の EC2 インスタンスの Spot Instance を利用するためのオプションです。

EC2 インスタンスの作成方法

```
1 host = ec2.Instance(
2     self, "MyFirstEc2Instance",
3     instance_type=ec2.InstanceType("t2.micro"),
4     machine_image=ec2.MachineImage.latest_amazon_linux(),
5     vpc=vpc,
6     vpc_subnets=ec2.SubnetSelection(subnet_type=ec2.SubnetType.PUBLIC),
7     security_group=sg,
8     key_name=key_name
9 )
```

t2.micro は、Amazon Linux (Machine image) を OS として利用します。Machine image は、Chapter 6 で説明されています。VPC、SG は、Chapter 6 で説明されています。

EC2 インスタンスの作成方法

4.4. EC2 インスタンスの作成

EC2 インスタンスの作成方法については、Section 4.1 で説明されています。

4.4.1. Python 環境構築

Python 環境構築は Python 環境構築用の `venv` を利用する

手元 `handson/ec2-get-started` 環境に移動する

```
$ cd handson/ec2-get-started
```

Python 環境構築用の `venv` を利用する

```
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt
```

Python 環境構築



`venv` 環境構築は [Section 15.7](#) を参照する



環境構築用の `pip` は `pip3` を用いて `python3 -m pip` でインストールする

4.4.2. AWS 環境構築

AWS CLI と AWS CDK を利用して AWS 環境構築を行う。環境構築は [Section 15.2](#) を参照する。
環境構築は [Section 15.3](#) を参照する。

環境構築用の `AWS_ACCESS_KEY_ID` は `~/.aws/credentials` に記載する。
環境構築用の `AWS CLI/CDK` は `~/.aws/credentials` に記載する。

4.4.3. SSH 環境構築

EC2 環境構築用の SSH 環境構築は EC2 環境構築用の SSH 環境構築を行う。

AWS CLI 環境構築 `HirakeGoma` 環境構築

```
$ export KEY_NAME="HirakeGoma"
$ aws ec2 create-key-pair --key-name ${KEY_NAME} --query 'KeyMaterial' --output text >
${KEY_NAME}.pem
```

環境構築用の `HirakeGoma.pem` は `~/.ssh/` に移動する。SSH 環境構築は `400` を参照する。

```
$ mv HirakeGoma.pem ~/.ssh/
$ chmod 400 ~/.ssh/HirakeGoma.pem
```

4.4.4. CDK deploy

CDK deploy 実行には、EC2 インスタンスの作成に必要な AWS クレジットカードの登録と、AWS CLI のインストールが必要です。-c key_name="HirakeGoma" オプションは、キーペアの名前を指定します。

```
$ cdk deploy -c key_name="HirakeGoma"
```

CDK deploy 実行後は、VPC に EC2 インスタンスが作成され、Figure 11 のように、InstancePublicIp が表示されます。IP アドレスは、InstancePublicIp の値です。

```
✓ MyFirstEc2
Outputs:
MyFirstEc2.InstancePublicIp = 54.238.112.5
MyFirstEc2.InstancePublicDnsName = ec2-54-238-112-5.ap-northeast-1.compute.amazonaws.com
Stack ARN:
arn:aws:cloudformation:ap-northeast-1:606887060834:stack/MyFirstEc2/46ed0490-aa2d-
```

Figure 11. CDK deploy の出力

4.4.5. SSH 接続

SSH 接続を行うには、

```
$ ssh -i ~/.ssh/HirakeGoma.pem ec2-user@<IP address>
```

-i オプションは、キーペアのファイルパスを指定します。ec2-user は、EC2 インスタンスのデフォルトユーザーです。<IP address> は、Figure 11 の InstancePublicIp の値です。

Figure 12 のように、SSH 接続が成功すると、[ec2-user@ip-10-10-1-217 ~]\$ のプロンプトが表示されます。

```
(.env) tomoyuki@eiffel:01-ec2$ ssh -i ~/.ssh/HirakeGoma.pem ec2-user@54.238.112.5
Last login: Tue Jun  9 09:18:09 2020 from 157.82.122.171

 _ _ | _ _ | _ _ )
 _ | ( _ /   Amazon Linux AMI
 _ | \ _ | _ |

https://aws.amazon.com/amazon-linux-ami/2018.03-release-notes/
5 package(s) needed for security, out of 7 available
Run "sudo yum update" to apply all updates.
[ec2-user@ip-10-10-1-217 ~]$
```

Figure 12. SSH で EC2 インスタンスに接続

SSH 接続が成功すると、AWS の EC2 コンソールにログインできます。

4.4.6. EC2 インスタンスの起動

EC2 インスタンスの起動には、

EC2 インスタンスの起動には、CPU のタイプを選択する必要があります。

```
$ cat /proc/cpuinfo
```

processor	: 0
vendor_id	: GenuineIntel
cpu family	: 6
model	: 63
model name	: Intel(R) Xeon(R) CPU E5-2676 v3 @ 2.40GHz
stepping	: 2
microcode	: 0x43
cpu MHz	: 2400.096
cache size	: 30720 KB

[illegible]

```
$ top -n 1

top - 09:29:19 up 43 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  76 total,    1 running,  51 sleeping,    0 stopped,    0 zombie
Cpu(s):  0.3%us,  0.3%sy,  0.1%ni, 98.9%id,  0.2%wa,  0.0%hi,  0.0%si,  0.2%st
Mem:   1009140k total,  270760k used,  738380k free,   14340k buffers
Swap:        0k total,    0k used,    0k free,  185856k cached

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
    1 root        20   0 19696 2596 2268 S   0.0   0.3   0:01.21 init
    2 root        20   0     0     0     0 S   0.0   0.0   0:00.00 kthreadd
    3 root        20   0     0     0     0 I   0.0   0.0   0:00.00 kworker/0:0
```

t2.micro 1009140k = 1GB

Python 2 Python 3 Python 3.6

```
$ sudo yum update -y
$ sudo yum install -y python36
```

```
$ sudo yum update -y
$ sudo yum install -y python36
```

Python 環境構築

```
$ python3
Python 3.6.10 (default, Feb 10 2020, 19:55:14)
[GCC 4.8.5 20150623 (Red Hat 4.8.5-28)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Python の標準ライブラリには `Ctrl + D` を叩くと `exit()` を呼び出して標準出力を閉じる関数 `exit()` が定義されています。

\$ exit

4.4.7. AWS 環境構築

環境構築には、EC2 インスタンスを起動し、EC2インスタンスにAWS CLIをインストールし、環境構築を行います。

環境構築には、AWS Management ConsoleのServicesからEC2を選択し、Instancesを選択します。Figure 13に示すように、EC2インスタンスのリストが表示されます。VPCとSGも表示されます。

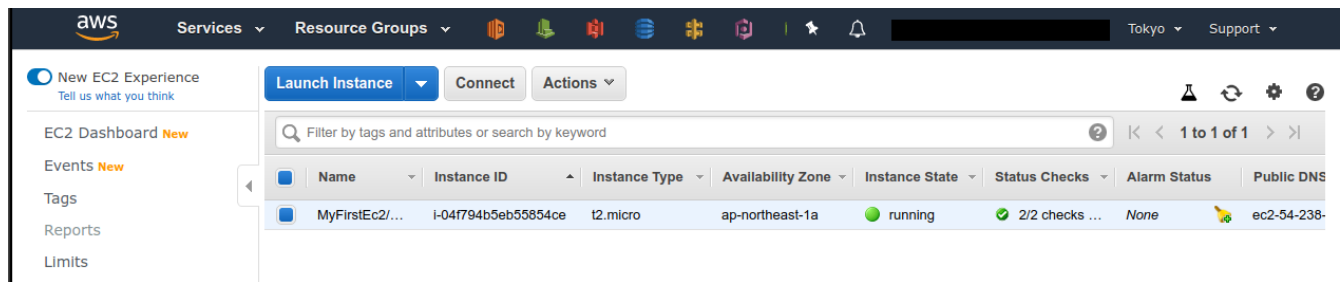


Figure 13. EC2 インスタンス



環境構築には、AWS Management ConsoleのServicesからEC2を選択し、Instancesを選択します。Figure 13に示すように、EC2インスタンスのリストが表示されます。VPCとSGも表示されます。

CloudFormation Stackを作成します。CloudFormation Stack (stack) を作成し、AWS Management ConsoleのCloudFormation Stackを作成します。VPC/EC2/SG 環境構築には、CloudFormation Stackを作成します (Figure 14)。

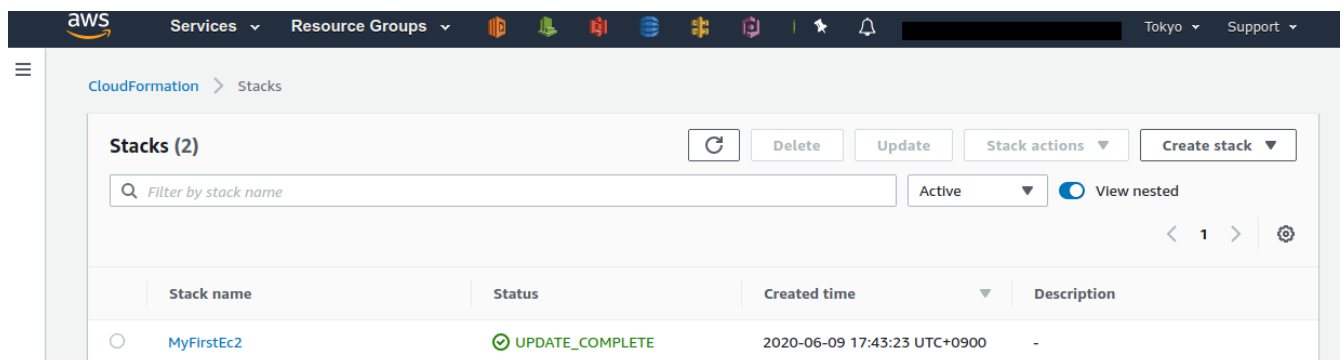


Figure 14. CloudFormation Stack

"MyFirstEc2" Stackを作成します。CloudFormation Stackを作成します。VPC/EC2/SG 環境構築には、CloudFormation Stackを作成します (Figure 14)。

4.4.8. 環境構築

環境構築には、AWS Management ConsoleのCloudFormation Stackを作成します。VPC/EC2/SG 環境構築には、CloudFormation Stackを作成します (Figure 14)。

CloudFormation Stack "Delete" を実行します (Figure 15)。CloudFormation Stack "DELETE_IN_PROGRESS" 状態になります。CloudFormation Stack "DELETE_IN_PROGRESS" 状態になります。

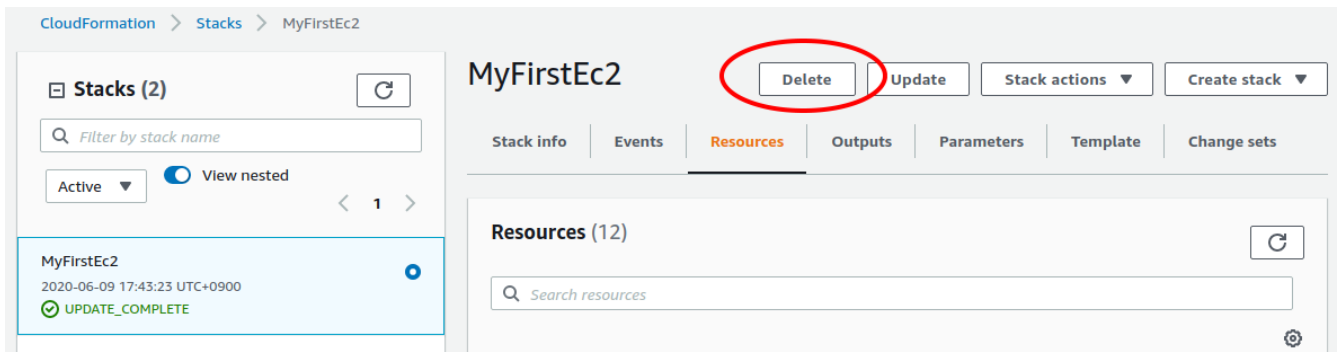


Figure 15. CloudFormation

```
$ cdk destroy
```

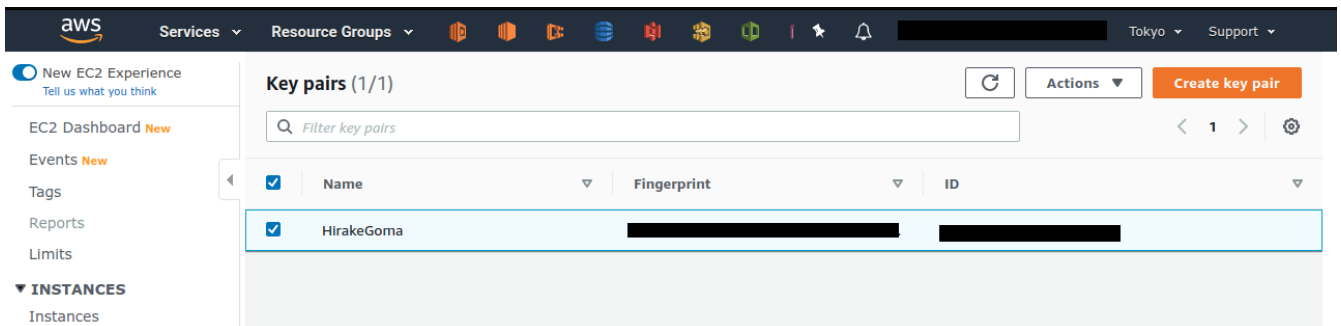


Figure 16. EC2

```
$ aws ec2 delete-key-pair --key-name "HirakeGoma"
```

```
$ rm -f ~/.ssh/HirakeGoma.pem
```



4.5. 概要

本章では、EC2 インスタンスに SSH 接続する方法を説明します。

[Chapter 2](#) で説明したように、EC2 インスタンスは、[Chapter 3](#) で説明したように、AWS IAM ユーザーを使用してアクセスします。AWS CLI または AWS CDK を使用して、AWS IAM ユーザーにアクセスします。

本章では、(オプション) [Chapter 2](#) で説明したように、EC2 インスタンスに SSH 接続する方法を説明します。本章では、EC2 インスタンスに SSH 接続する方法を説明します。

本章では、EC2 インスタンスに SSH 接続する方法を説明します。

Chapter 5. 環境構築と学習環境の準備

本章では、AI 学習のための環境構築と学習環境の準備について説明します。具体的には、Python、PyTorch、GPU のインストールと設定を行います。

本章では、Python、PyTorch、GPU のインストールと設定を行います。PyTorch は、AI 学習のためのフレームワークであり、GPU を利用して学習を行うことができます。

5.1. 環境構築と学習環境の準備

2010年代後半から AI 学習の分野で大きな進歩がみられるようになりました。特に、自然言語処理 (NLP) の分野では、Transformer などのモデルが注目を集めています。

本章では、AI 学習のための環境構築と学習環境の準備について説明します。具体的には、Python、PyTorch、GPU のインストールと設定を行います。GPT-3 は、OpenAI が開発した大規模な自然言語処理モデルです。

GPT-3 は、現在の SOTA (State-of-the-art) のモデルであり、自然言語処理の多くのタスクで高い性能を発揮しています。本章では、GPT-3 を学習するための環境構築と学習環境の準備について説明します。

本章では、AI 学習のための環境構築と学習環境の準備について説明します。具体的には、Python、PyTorch、GPU のインストールと設定を行います。Chapter 4 では、AI 学習のための環境構築と学習環境の準備について説明しました。GPU (Graphics Processing Unit) は、AI 学習のための重要なハードウェアです。GPT-3 は、Microsoft が開発した大規模な自然言語処理モデルです。

5.2. GPU のインストールと設定

本章では、GPU (Graphics Processing Unit) のインストールと設定について説明します。

GPU は、AI 学習のための重要なハードウェアです。CPU (Central Processing Unit) と比較して、GPU は並列処理能力が非常に高いです。XBox、PS5 などのゲーム機にも GPU が搭載されています。GPU の性能は、FPS (Frames Per Second) で表されます。GPU のインストールと設定は、AI 学習のための重要なステップです。(Figure 17)は、GPU のインストールと設定の手順を示しています。



Figure 17. GPU architecture (source: <https://devblogs.nvidia.com/nvidia-turing-architecture-in-depth/>)

GPU is designed for **General-purpose computing on GPU; GPGPU**. CPU is designed for **General-purpose computing on CPU; GPGPU**. GPU is designed for **General-purpose computing on GPU; GPGPU**.

GPU is designed for **General-purpose computing on GPU; GPGPU**. CPU is designed for **General-purpose computing on CPU; GPGPU**. GPU is designed for **General-purpose computing on GPU; GPGPU**.

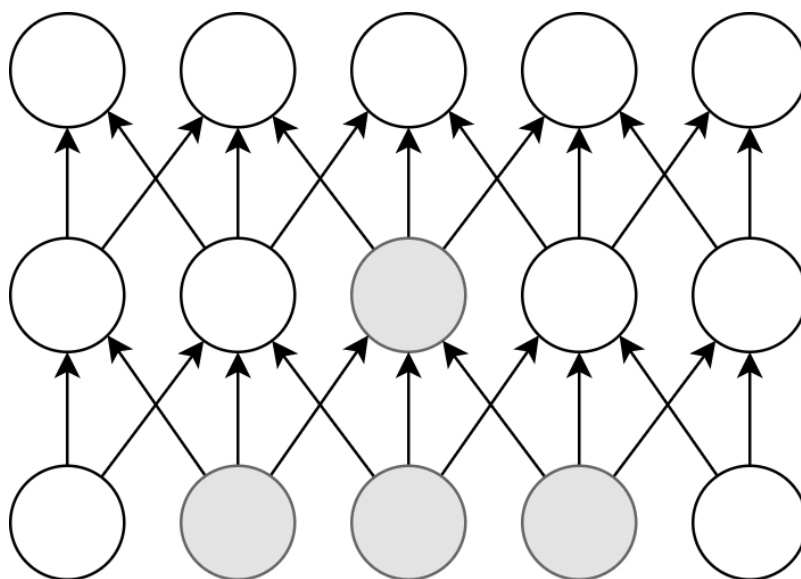


Figure 18. Neural network architecture

GPU is designed for **General-purpose computing on GPU; GPGPU**. CPU is designed for **General-purpose computing on CPU; GPGPU**. GPU is designed for **General-purpose computing on GPU; GPGPU**.



Amazon EC2 V100 GPU instances (NVIDIA GeForce V100) are available in the us-east-1 region. The V100 GPU has 16 GB of memory and 8 vCPUs. The price per hour is \$3.06. The V100 GPU is available in the p3.xlarge instance type. The V100 GPU is available in the p3n.16xlarge instance type. The V100 GPU is available in the p2.xlarge instance type. The V100 GPU is available in the g4dn.xlarge instance type.

Amazon EC2 GPU instances (P3, P2, G3, G4) are available in the us-east-1 region. Table 3 shows the GPU instances and their specifications.

Table 3. GPU instances on EC2

Instance	GPUs	GPU model	GPU Mem (GiB)	vCPU	Mem (GiB)	Price per hour (\$)
p3.2xlarge	1	NVIDIA V100	16	8	61	3.06
p3n.16xlarge	8	NVIDIA V100	128	64	488	24.48
p2.xlarge	1	NVIDIA K80	12	4	61	0.9
g4dn.xlarge	1	NVIDIA T4	16	4	16	0.526

Table 3 shows the GPU instances and their specifications. The CPU instances are p3.xlarge, p3n.16xlarge, p2.xlarge, and g4dn.xlarge. The GPU instances are V100, K80, and T4. The price per hour is shown in the last column.

The g4dn.xlarge instance type is available in the us-east-1 region. The NVIDIA T4 GPU is available in the g4dn.xlarge instance type. The price per hour is \$0.526.



Table 3 shows the GPU instances and their specifications in the us-east-1 region.



The V100 GPU is available in the p3.2xlarge instance type. The price per hour is \$3.06. The V100 GPU has 16 GB of memory and 8 vCPUs. The V100 GPU is available in the p3n.16xlarge instance type. The V100 GPU is available in the p2.xlarge instance type. The V100 GPU is available in the g4dn.xlarge instance type.

GPT-3 is available in the us-east-1 region. The Lambda function is available in the us-east-1 region. The Lambda function is available in the us-east-1 region.



The GPT-3 model is available in the us-east-1 region. The Lambda function is available in the us-east-1 region. The Lambda function is available in the us-east-1 region.

Further reading

For more information on GPU instances, see the Amazon EC2 documentation.

- [Deep Learning \(Ian Goodfellow, Yoshua Bengio and Aaron Courville\)](#) is a book that covers the fundamentals of deep learning. It is available on Amazon.

- [Intro to Deep Learning \(CS 231n\)](#) 斯坦福大学深度学习课程 包含理论和实践内容
- [Dive into Deep Learning \(Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola\)](#)
包含理论和实践内容 1000 多个练习
包含理论和实践内容

Chapter 6. Hands-on #2: AWS 環境構築

6.1. 準備

このハンズオンでは GPU を利用した EC2 インスタンスを起動し、MNIST の推論を行います。

このハンズオンに必要な GitHub のリポジトリは [handson/mnist](https://github.com/handson/mnist) です。

このハンズオンでは AWS の基本操作 (Section 4.1) を確認しながら進めます。



このハンズオンでは AWS の GPU インスタンス G4dn を利用します。G4dn インスタンスは AWS の EC2 インスタンスの Limits を確認する必要があります。Running On-Demand All G instances の制限を確認してください。



このハンズオンでは 0 台の GPU を利用します。AWS の制限を確認してください。Amazon EC2 service quotas



このハンズオンでは g4dn.xlarge を利用します。EC2 インスタンス (ap-northeast-1) の制限を確認してください。0.71 \$/hour

AWS Educate Starter Account の制限を確認してください: Starter Account の GPU インスタンスの制限を確認してください。Starter Account の制限を確認してください。

6.2. 環境構築

このハンズオンでは Figure 19 の環境構築を行います。

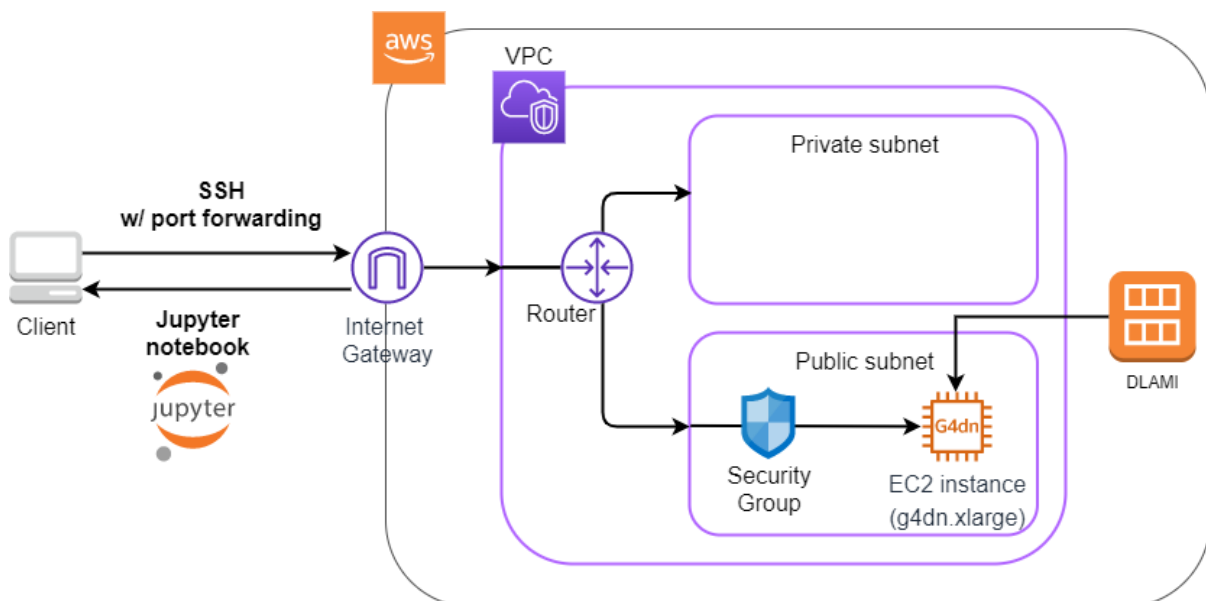


Figure 19. ハンズオン#2の環境構築

このハンズオンでは AWS の基本操作を確認しながら進めます。

- GPU `g4dn.xlarge` を使いたい
- 必要に応じて `DLAMI` (AMI) を指定
- SSH 接続できるように `Jupyter Notebook` (AMI) を指定

必要に応じて `handson/mnist/app.py` を指定

```

1 class Ec2ForDL(Stack):
2
3     def __init__(self, scope: Construct, construct_id: str, key_name: str, **
4         kwargs) -> None:
5         super().__init__(scope, construct_id, **kwargs)
6
7         vpc = ec2.Vpc(
8             self, "Ec2ForDL-Vpc",
9             max_azs=1,
10            ip_addresses=ec2.IpAddresses.cidr("10.10.0.0/23"),
11            subnet_configuration=[
12                ec2.SubnetConfiguration(
13                    name="public",
14                    subnet_type=ec2.SubnetType.PUBLIC,
15                )
16            ],
17            nat_gateways=0,
18        )
19
20        sg = ec2.SecurityGroup(
21            self, "Ec2ForDL-Sg",
22            vpc=vpc,
23            allow_all_outbound=True,
24        )
25        sg.add_ingress_rule(
26            peer=ec2.Peer.any_ipv4(),
27            connection=ec2.Port.tcp(22),
28        )
29
30        host = ec2.Instance(
31            self, "Ec2ForDL-Instance",
32            instance_type=ec2.InstanceType("g4dn.xlarge"), ①
33            machine_image=ec2.MachineImage.generic_linux({
34                "us-east-1": "ami-060f07284bb6f9faf",
35                "ap-northeast-1": "ami-09c0c16fc46a29ed9"
36            }), ②
37            vpc=vpc,
38            vpc_subnets=ec2.SubnetSelection(subnet_type=ec2.SubnetType.PUBLIC),
39            security_group=sg,
40            key_name=key_name
41        )

```

① `g4dn.xlarge` は GPU を搭載したインスタンス (標準 CPU は `t2.micro`) `g4dn.xlarge` は GPU を搭載した

Chapter 5 **□□□□□□□□ NVIDIA T4 □□□□□□□□ GPU □□□□□□□□□□ CPU □ 4 core, □□□□□□ 16GB**
□□□□□□□□□□

② Deep Learning 環境構築(AMI (Deep Learning Amazon Machine Image; DLAMI) を利用 (Amazon Linux AMIAMIを利用)) として AMI の ID を 指定する(us-east-1 ap-northeast-1)

[illegible]

```
AMI                                us-east-1                                ap-northeast-1
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX  XXXXXXXXXXXXXXXXXXXXXXXXXXXX AMI ID
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

6.2.1. DLAMI (Deep Learning Amazon Machine Image)

AMI (Amazon Machine Image) 是預先建置好的 OS (Operating System) 映像檔。您可以將 AMI 映像檔上傳到 EC2 实例，然後在 EC2 实例中安裝 OS。AMI 映像檔可以是 Ubuntu、Linux 或 Windows Server 等 OS。您可以在 Amazon Linux 中找到 AMI 映像檔。

	AMI		OS		AMI		(AMI)		OS
				AMI					AMI
						EC2		Python	3.6

AMI は AWS マarketplaceから選択可能なAMI (AMI) AMI は EC2 インスタンスを起動するためのイメージです。AWS CLI を使用してAMIを起動する方法は、(AMI) AMI

```
$ aws ec2 describe-images --owners amazon
```

AMI DLAMI (Deep Learning AMI) DLAMI
TensorFlow, PyTorch EC2

`aws ec2 run-instances --image-id ami-09c0c16fc46a29ed9 --instance-type t2.micro --key-name my-key-pair --security-groups sg-78d3e0f1 --cli-input-json '{\"DryRun\": true}'`

```
$ aws ec2 describe-images --owners amazon --image-ids "ami-09c0c16fc46a29ed9" --region ap-northeast-1
```

```

tomoyuki@balthasar:02-ec2-dnn$ aws ec2 describe-images --owners amazon --image-ids "ami-09c0c16fc46a29ed9"
{
  "Images": [
    {
      "Architecture": "x86_64",
      "CreationDate": "2020-05-20T14:47:04.000Z",
      "ImageId": "ami-09c0c16fc46a29ed9",
      "ImageLocation": "amazon/Deep Learning AMI (Amazon Linux 2) Version 29.0",
      "ImageType": "machine",
      "Public": true,
      "OwnerId": "898082745236",
      "PlatformDetails": "Linux/UNIX",
      "UsageOperation": "RunInstances",
      "State": "available",
      "BlockDeviceMappings": [
        {
          "DeviceName": "/dev/xvda",
          "Ebs": {
            "DeleteOnTermination": true,
            "SnapshotId": "snap-0bd381ab76e5a6146",
            "VolumeSize": 90,
            "VolumeType": "gp2",
            "Encrypted": false
          }
        }
      ],
      "Description": "MXNet-1.6.0, Tensorflow-2.1.0 & 1.15.2, PyTorch-1.4.0 & 1.5.0, Neuron, & other frameworks, NVIDIA CUDA, cuDNN, NCCL, Intel MKL-DNN, Docker, NVIDIA-Docker & EFA support. For fully managed experience, check: https://aws.amazon.com/sagemaker",
      "EnaSupport": true,
      "Hypervisor": "xen",
      "ImageOwnerAlias": "amazon",
      "Name": "Deep Learning AMI (Amazon Linux 2) Version 29.0",
      "RootDeviceName": "/dev/xvda",
      "RootDeviceType": "ebs",
      "SriovNetSupport": "simple",
      "VirtualizationType": "hvm"
    }
  ]
}

```

Figure 20. AMI ID = ami-09c0c16fc46a29ed9

Figure 20 DLAMI PyTorch 1.4.0 1.5.0 DLAMI



DLAMI (何: ["What Is the AWS Deep Learning AMI?"](#))

low-level GPU OS GPU CUDA cuDNN NVIDIA GPU C++ cuDNN CUDA n "Base" DLAMI

"Conda" "Base" TensorFlow PyTorch Anaconda TensorFlow PyTorch MxNet Jupyter Notebook

6.3.

1. 環境構築 (Section 15.3)

```
# 環境構築
$ cd handson/mnist

# venv 環境構築
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# SSH
$ export KEY_NAME="HirakeGoma"
$ aws ec2 create-key-pair --key-name ${KEY_NAME} --query 'KeyMaterial' --output text >
${KEY_NAME}.pem
$ mv HirakeGoma.pem ~/.ssh/
$ chmod 400 ~/.ssh/HirakeGoma.pem

# CDK
$ cdk deploy -c key_name="HirakeGoma"
```



1. SSH 環境構築 (Section 15.3)

Figure 21 AWS インスタンスの IP アドレス (InstancePublicIp) を取得する

```
✓ Ec2ForDL
Outputs:
Ec2ForDL.InstancePublicIp = 52.192.211.12
Ec2ForDL.InstancePublicDnsName = ec2-52-192-211-12.ap-northeast-1.compute.amazonaws.com

Stack ARN:
arn:aws:cloudformation:ap-northeast-1:606887060834:stack/Ec2ForDL/dd8361d0-...
```

Figure 21. CDK の出力結果

6.4. 接続

SSH 環境構築後、Jupyter Notebook を起動するためのポート転送 (port forwarding) を行う (-L) が必要

```
$ ssh -i ~/.ssh/HirakeGoma.pem -L localhost:8931:localhost:8888 ec2-user@<IP address>
```

SSH 環境構築後、Jupyter Notebook を起動するためのポート転送 (port forwarding) を行う (-L localhost:8931:localhost:8888) が必要。localhost:8931 は Jupyter Notebook が起動しているポート (localhost:8888) に接続するためのポート (TCP/IP) である。Jupyter Notebook は localhost:8931 で起動している (Figure 22)。

```
ssh -L localhost:8931:localhost:8888 ec2-user@0.0.0.0
```

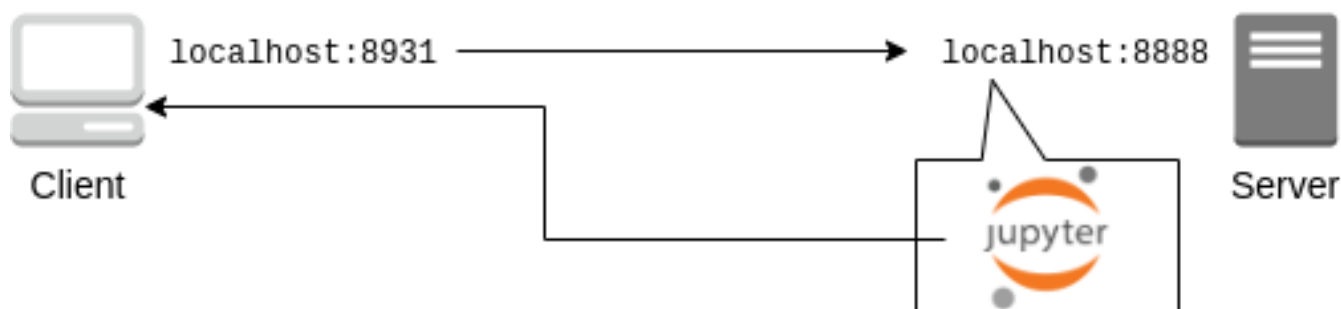


Figure 22. SSH 接続で Jupyter Notebook を実行



接続先 (:8931, :8888) は 1065535 の IP アドレスで、 22 (SSH) と 80 (HTTP) のポート番号で Jupyter Notebook を実行する。 8888 のポート番号で実行する。



SSH 接続は <IP address> の IP アドレスで実行する。



Docker を実行する。

SSH 接続で Docker を実行 (Docker を実行) する Jupyter Notebook を実行する Docker を実行する。

Docker を実行する。 cat ~/.ssh/HirakeGoma の内容を確認する。 -v Docker を実行する "Use volumes" を実行する。

SSH 接続で GPU を実行する。

```
$ nvidia-smi
```

Figure 23 接続先 Tesla T4 の GPU 1 の GPU Driver と CUDA の GPU を実行する。

```
[ec2-user@ip-10-10-1-172 ~]$ nvidia-smi
Sat Jun 13 04:55:22 2020

+-----+
| NVIDIA-SMI 440.33.01      Driver Version: 440.33.01      CUDA Version: 10.2      |
+-----+-----+-----+-----+-----+-----+
| GPU   Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
|    0   Tesla T4              On      | 00000000:00:1E.0 Off  |          0          |
| N/A   31C    P8      11W /  70W |  0MiB / 15109MiB |    0%      Default  |
+-----+-----+-----+-----+-----+-----+

Processes:
GPU      PID    Type   Process name                      GPU Memory
                               Usage
+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+-----+-----+-----+-----+-----+

```

Figure 23. nvidia-smi

6.5. Jupyter Notebook

Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and explanatory text. It consists of a GUI and a Python kernel. (Figure 24) Python kernel is a program that runs Python code and returns the output.

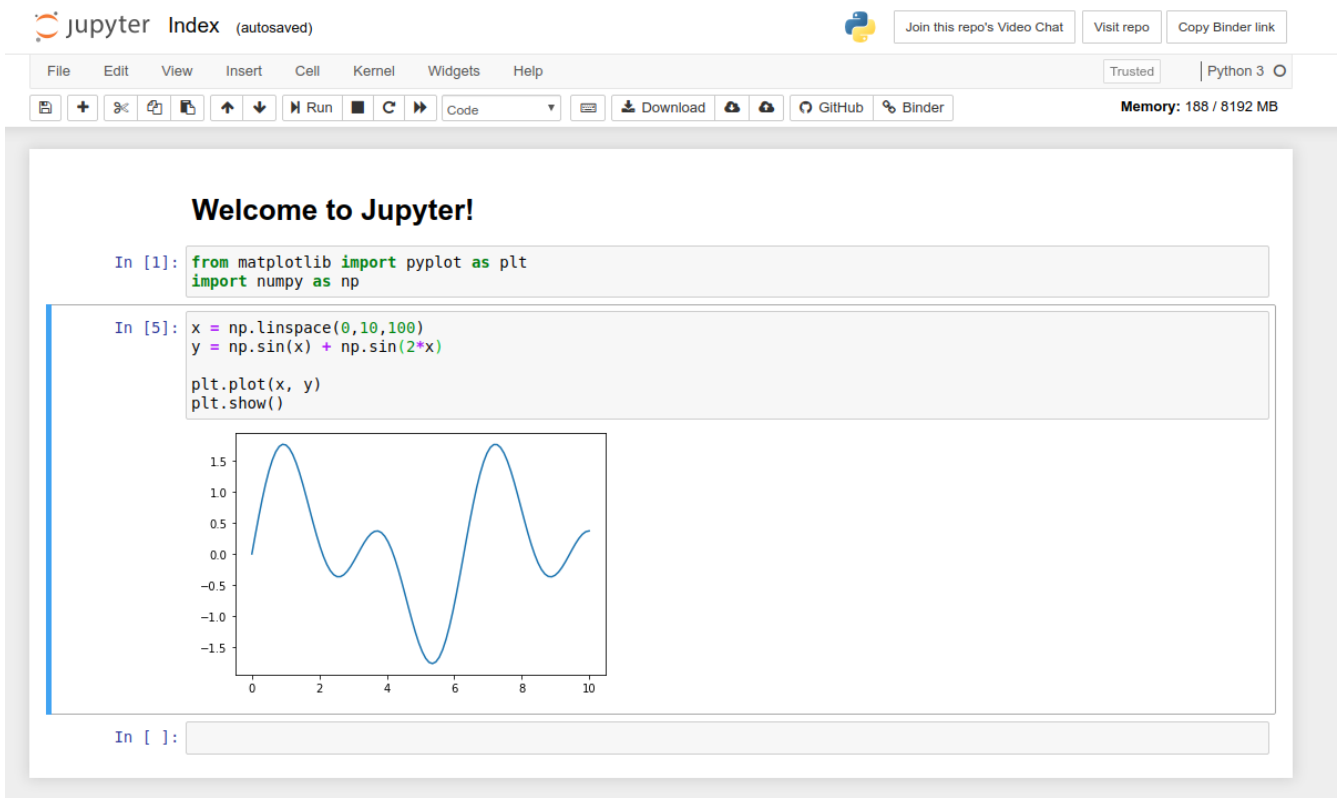


Figure 24. Jupyter Notebook

Jupyter Notebook is a web application that allows you to create and share documents that contain live code, equations, visualizations and explanatory text. It consists of a GUI and a Python kernel. DLAMI is a Jupyter Notebook kernel that runs Python code and returns the output.

Jupyter Notebook can be installed on an EC2 instance using SSH.

```
$ cd ~ # go to home directory
$ jupyter notebook
```

Figure 25 Jupyter EC2 localhost:8888
localhost:8888 ?token=XXXX

```
[I 06:01:10.466 NotebookApp] The Jupyter Notebook is running at:
[I 06:01:10.466 NotebookApp] http://localhost:8888/?token=537fee42934a7db9d0540024260d305b08bb0bf9fc41c5a7
[I 06:01:10.466 NotebookApp] or http://127.0.0.1:8888/?token=537fee42934a7db9d0540024260d305b08bb0bf9fc41c5a7
[I 06:01:10.466 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[W 06:01:10.469 NotebookApp] No web browser found: could not locate runnable browser.
[C 06:01:10.469 NotebookApp]

To access the notebook, open this file in a browser:
    file:///home/ec2-user/.local/share/jupyter/runtime/nbserver-11720-open.html
Or copy and paste one of these URLs:
    http://localhost:8888/?token=537fee42934a7db9d0540024260d305b08bb0bf9fc41c5a7
    or http://127.0.0.1:8888/?token=537fee42934a7db9d0540024260d305b08bb0bf9fc41c5a7
```

Figure 25. Jupyter Notebook



Jupyter Notebook
AWS GPU

SSH Jupyter localhost:8888
localhost:8931 Jupyter (Chrome, FireFox)

http://localhost:8931/?token=XXXX

?token=XXXX Jupyter

Jupyter (Figure 26) Jupyter

Quit Logout

Files Running IPython Clusters Conda

Select items to perform actions on them. Upload New

	Name	Last Modified	File size
0 /			
☐	anaconda3	25 days ago	
☐	data	16 hours ago	
☐	examples	20 hours ago	
☐	examples_	25 days ago	
☐	src	25 days ago	
☐	tools	a month ago	
☐	tutorials	25 days ago	

Figure 26. Jupyter



Jupyter Notebook

- **Shift + Enter**: 실행
- **Esc: Command mode** 모드
- 커서로 "+" 키를 누르면 Command mode → **A** ⇒ 실행
- 커서로 "X" 키를 누르면 Command mode → **X** ⇒ 실행

이제 Ventsislav Yordanov 님의 블로그를 살펴보겠습니다.

6.6. PyTorch를 설치합니다

PyTorch는 Facebook AI Research LAB (FAIR)에서 개발한 딥러닝 프레임워크입니다. PyTorch는 Tesla에서 개발한 딥러닝 프레임워크인 TensorFlow와 유사합니다. PyTorch는 딥러닝 프레임워크입니다.



PyTorch를 설치합니다.

Facebook은 PyTorch를 Caffe2와 비교합니다. (Caffe는 UC Berkley에서 개발한 Yangqing Jia 님의 프로젝트) Caffe2는 2018년 PyTorch로 대체되었습니다.

2019년 12월 Preferred Networks에서 Chainer를 PyTorch로 대체했습니다. (Chainer는 Preferred Networks에서 개발한 딥러닝 프레임워크) PyTorch는 Chainer의 API를 DNA로 PyTorch로 대체했습니다...!

PyTorch는 GPU를 지원합니다. GPU를 지원하는 환경을 설치합니다.

Jupyter를 실행합니다. "New" 버튼을 클릭하여 "conda_pytorch_p36" 환경을 선택합니다. (Figure 27) "conda_pytorch_p36" 환경을 PyTorch로 설치합니다.

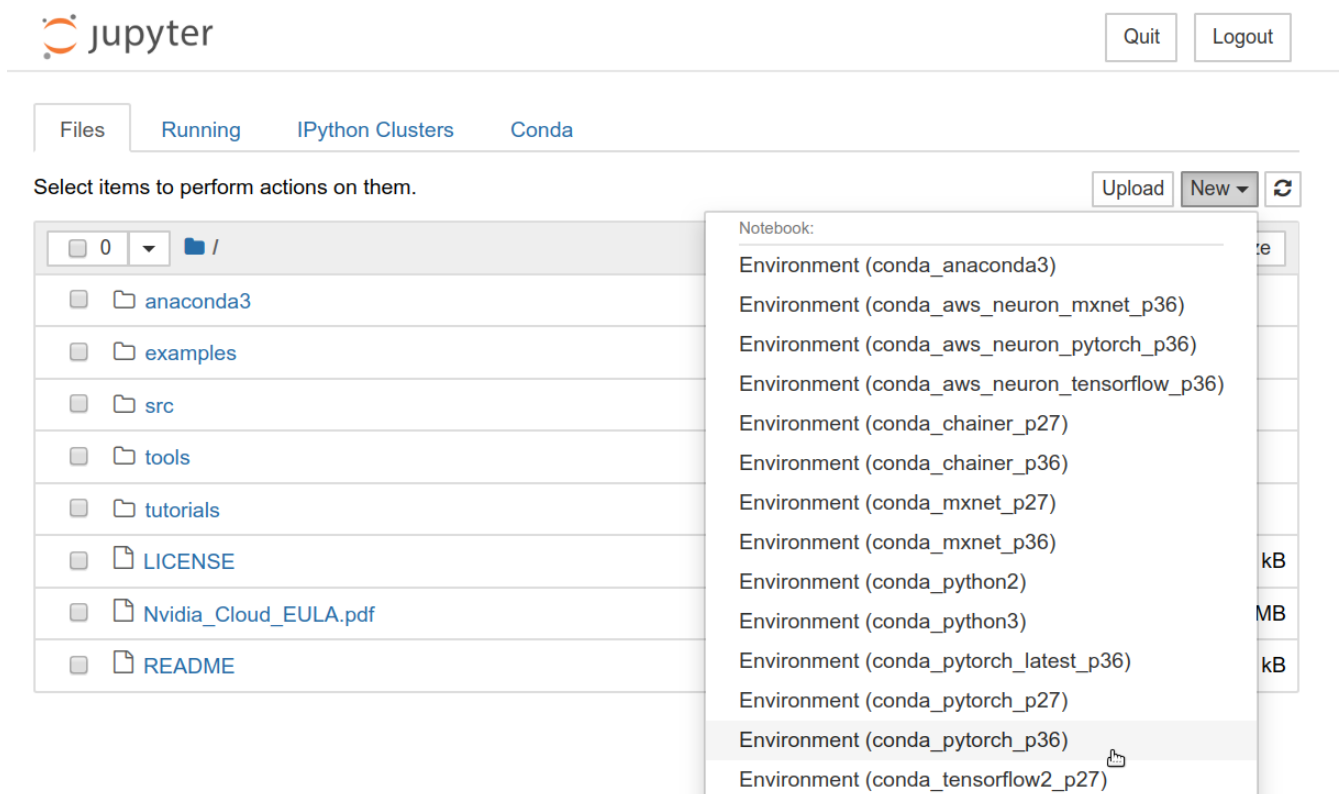


Figure 27. "conda_pytorch_p36" 환경을 선택합니다

Figure 28

```
In [1]: import torch
        print("Is CUDA ready?", torch.cuda.is_available())
        Is CUDA ready? True

In [3]: # create a random array in CPU
        x = torch.rand(3,3)
        print(x)
        tensor([[0.6896, 0.2428, 0.3269],
                  [0.0533, 0.3594, 0.9499],
                  [0.9764, 0.5881, 0.0203]])

In [4]: # create another array in GPU device
        y = torch.ones_like(x, device="cuda")
        # move 'x' from CPU to GPU
        x = x.to("cuda")

In [5]: # run addition operation in GPU
        z = x + y
        print(z)
        tensor([[1.6896, 1.2428, 1.3269],
                  [1.0533, 1.3594, 1.9499],
                  [1.9764, 1.5881, 1.0203]], device='cuda:0')

In [6]: # move z from GPU to CPU
        z = z.to("cpu")
        print(z)
        tensor([[1.6896, 1.2428, 1.3269],
                  [1.0533, 1.3594, 1.9499],
                  [1.9764, 1.5881, 1.0203]])
```

Figure 28. PyTorch

PyTorch GPU

```
1 import torch
2 print("Is CUDA ready?", torch.cuda.is_available())
```

:

```
Is CUDA ready? True
```

3x3 CPU

```
1 x = torch.rand(3,3)
2 print(x)
```

:

```
tensor([[0.6896, 0.2428, 0.3269],
        [0.0533, 0.3594, 0.9499],
        [0.9764, 0.5881, 0.0203]])
```

GPU

```
1 y = torch.ones_like(x, device="cuda")
2 x = x.to("cuda")
```

변수 `x` 와 `y` 는 모두 GPU로 이동

```
1 z = x + y
2 print(z)
```

결과:

```
tensor([[1.6896, 1.2428, 1.3269],
        [1.0533, 1.3594, 1.9499],
        [1.9764, 1.5881, 1.0203]], device='cuda:0')
```

변수 GPU로 이동하면 CPU로 이동

```
1 z = z.to("cpu")
2 print(z)
```

결과:

```
tensor([[1.6896, 1.2428, 1.3269],
        [1.0533, 1.3594, 1.9499],
        [1.9764, 1.5881, 1.0203]])
```

변수 GPU로 이동하면 CPU로 이동 GPU로 이동하면 GPU로 이동 3x3
변수 GPU로 이동하면 GPU로 이동 GPU로 이동하면 GPU로 이동



Jupyter Notebook 에서 /handson/mnist/pytorch/pytorch_get_started.ipynb 에서
Jupyter 인터프리터 "Upload" 버튼을 클릭하여 업로드합니다

업로드된 파일을 Jupyter 인터프리터에서 실행합니다

변수 GPU로 이동하면 CPU로 이동 GPU로 이동하면 GPU로 이동 Jupyter 인터프리터에서 `%time` 명령어를 사용하여

변수 CPU로 이동하면 10000x10000 행렬 곱셈을 실행합니다

```
1 s = 10000
2 device = "cpu"
3 x = torch.rand(s, s, device=device, dtype=torch.float32)
4 y = torch.rand(s, s, device=device, dtype=torch.float32)
5
6 %time z = torch.matmul(x,y)
```

このコードを実行すると、CPUの処理時間が約11.6秒、GPUの処理時間が約5.8秒で実行されます。

```
CPU times: user 11.5 s, sys: 140 ms, total: 11.6 s
Wall time: 5.8 s
```

GPUの処理時間を短縮するために、CUDAの同期を明示的に呼び出す必要があります。

```
1 s = 10000
2 device = "cuda"
3 x = torch.rand(s, s, device=device, dtype=torch.float32)
4 y = torch.rand(s, s, device=device, dtype=torch.float32)
5 torch.cuda.synchronize()
6
7 %time z = torch.matmul(x,y); torch.cuda.synchronize()
```

このコードを実行すると、CPUの処理時間が約554ms、GPUの処理時間が約553msで実行されます。

```
CPU times: user 334 ms, sys: 220 ms, total: 554 ms
Wall time: 553 ms
```



PyTorch の GPU 処理は asynchronous (非同期) であるため、`torch.cuda.synchronize()` を明示的に呼び出す必要があります。



このコードでは、`dtype=torch.float32` を指定しているため、GPUの精度は32bitで実行されます。32bit/16bitの精度を指定することで、GPUの精度を16bitに設定することもできます。

このコードを実行すると、GPUの処理時間が約10秒で実行されます。これは、GPUの精度が32bitであるためです。

6.7. MNISTの学習と推論

この章では、AWSのEC2インスタンスを使用して、MNISTの学習と推論を実行します。

この章の図表は、MNISTの学習と推論の結果を示しています。(Figure 29)



Figure 29. MNIST 데이터셋

이번 MNIST 데이터셋을 사용하여 (Convolutional Neural Network; CNN) 모델을 구현하고 </handson/minist/pytorch/>에서 `mnist.ipynb`와 `simple_mnist.py` 파일을 사용하여 PyTorch의 [Example Project](#)를 실행해 보겠습니다.

이제 `simple_mnist.py` 파일을 업로드합니다 (Figure 30). "Upload" 버튼을 클릭하여 Python 코드를 업로드하고 CNN 모델을 실행합니다.

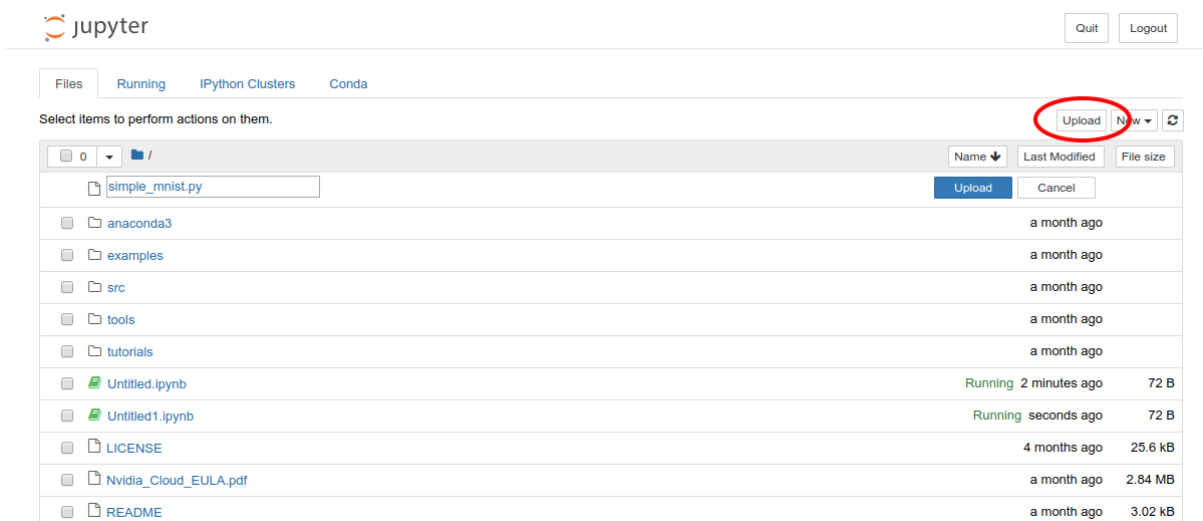


Figure 30. `simple_mnist.py` 업로드

`simple_mnist.py` 파일을 업로드한 후 notebook에서 "conda_pytorch_p36" 환경을 선택합니다.

이제 모델을 실행하고 결과를 확인합니다.

```

1 import torch
2 import torch.optim as optim
3 import torchvision
4 from torchvision import datasets, transforms
5 from matplotlib import pyplot as plt
6
7 # custom functions and classes
8 from simple_mnist import Model, train, evaluate

```

`torchvision` `datasets.MNIST` `torch.utils.data.DataLoader` `torchvision.transforms.Compose` `torchvision.transforms.ToTensor` `torchvision.transforms.Normalize` (`Model`, `train`, `evaluate`)

`datasets.MNIST` `torch.utils.data.DataLoader`

```

1 transf = transforms.Compose([transforms.ToTensor(),
2                               transforms.Normalize((0.1307,), (0.3081,))])
3
4 trainset = datasets.MNIST(root='./data', train=True, download=True, transform=
    transf)
5 trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)
6
7 testset = datasets.MNIST(root='./data', train=False, download=True, transform=
    transf)
8 testloader = torch.utils.data.DataLoader(testset, batch_size=1000, shuffle=True)

```

`datasets.MNIST` `torch.utils.data.DataLoader` `torchvision.transforms.Compose` `torchvision.transforms.ToTensor` `torchvision.transforms.Normalize` (`Model`, `train`, `evaluate`)

Figure 31

```

1 examples = iter(testloader)
2 example_data, example_targets = examples.next()
3
4 print("Example data size:", example_data.shape)
5
6 fig = plt.figure(figsize=(10,4))
7 for i in range(10):
8     plt.subplot(2,5,i+1)
9     plt.tight_layout()
10    plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
11    plt.title("Ground Truth: {}".format(example_targets[i]))
12    plt.xticks([])
13    plt.yticks([])
14 plt.show()

```

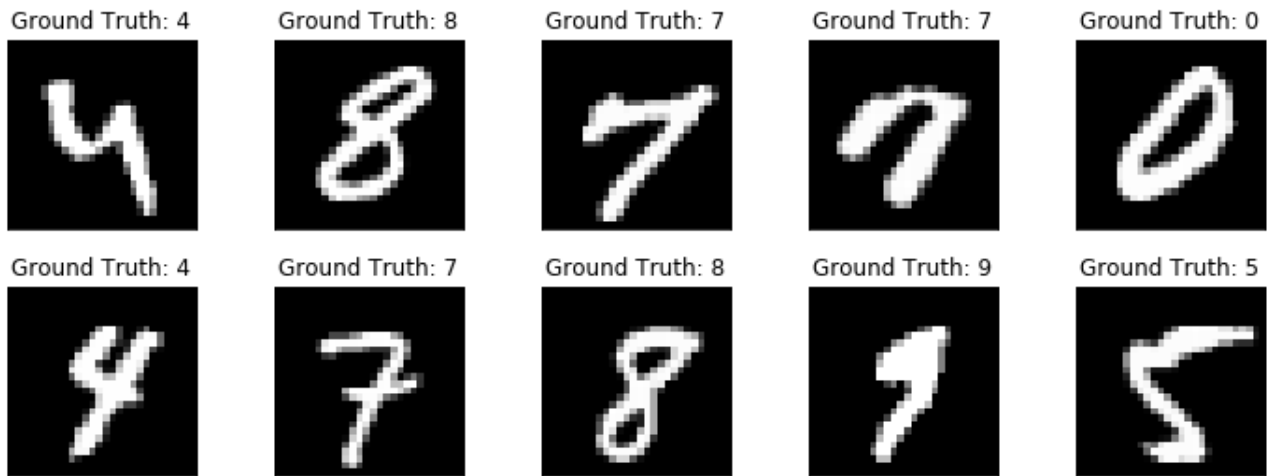


Figure 31. MNIST 데이터셋의 예시 이미지

이제 CNN 모델을 구현해 보겠습니다.

```
1 model = Model()
2 model.to("cuda") # load to GPU
```

이제 `Model` 클래스를 `simple_mnist.py` 파일에 구현해 보겠습니다. Figure 32에 표시된 구조를 따라 구현합니다. (output layer)에 Softmax 함수를 추가합니다. (Loss function)으로 Negative log likelihood (NLL)을 사용합니다.

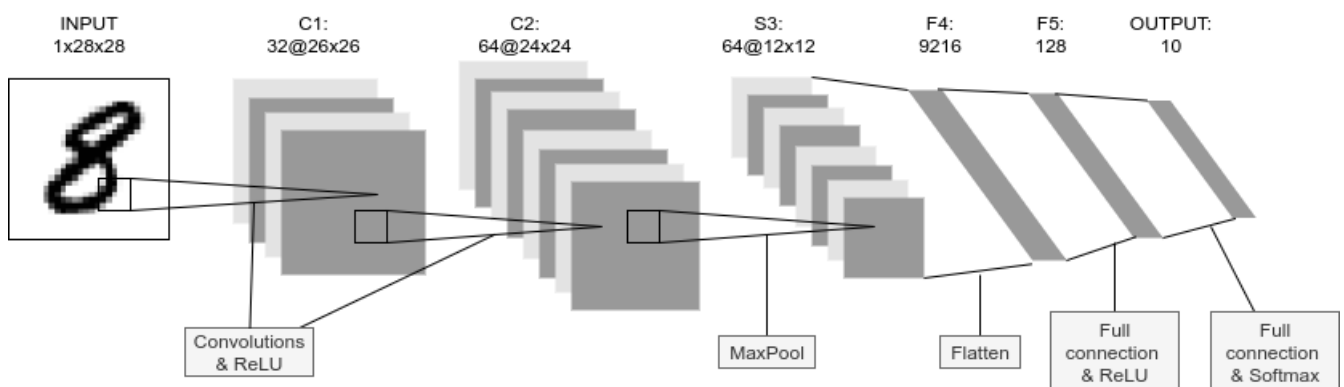


Figure 32. CNN 구조도

CNN 모델을 학습시키기 위해 Stochastic Gradient Descent (SGD) 옵티마이저를 사용합니다.

```
1 optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.5)
```

이제 CNN 모델을 학습시켜 보겠습니다!

```
1 train_losses = []
2 for epoch in range(5):
3     losses = train(model, trainloader, optimizer, epoch)
4     train_losses = train_losses + losses
5     test_loss, test_accuracy = evaluate(model, testloader)
```

```

6     print(f"\nTest set: Average loss: {test_loss:.4f}, Accuracy: {test_accuracy:.1f}%\n")
7
8     plt.figure(figsize=(7,5))
9     plt.plot(train_losses)
10    plt.xlabel("Iterations")
11    plt.ylabel("Train loss")
12    plt.show()

```

5000 iterations GPU 100%

Figure 33 shows the training loss (Loss function) over iterations. The loss starts high and decreases rapidly, stabilizing around 0.2 after approximately 1000 iterations.

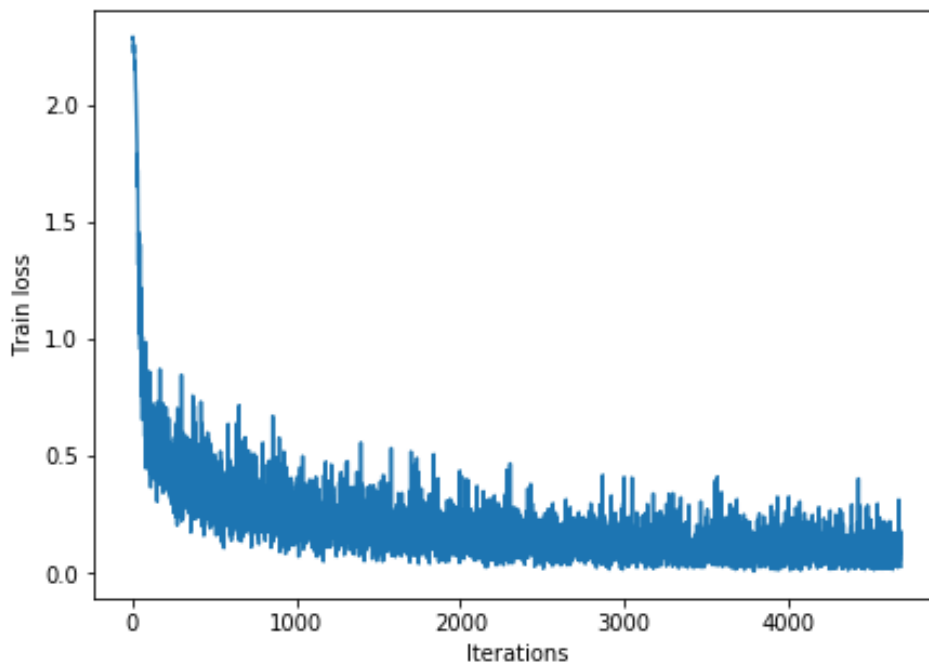


Figure 33. Training loss plot

The final accuracy of the model is 98% (Figure 34).

[mnist_final_score] | imgs/handson-jupyter/mnist_final_scdk.png

Figure 34. Final accuracy of the CNN model (5000 iterations)

The CNN model is trained on the MNIST dataset. The training process is shown in Figure 35, which displays the training loss over 10 iterations. The loss starts high and decreases rapidly, stabilizing around 0.2 after approximately 1000 iterations.

```

1 model.eval()
2
3 with torch.no_grad():
4     output = model(example_data.to("cuda"))
5
6 fig = plt.figure(figsize=(10,4))
7 for i in range(10):
8     plt.subplot(2,5,i+1)

```

```

9 plt.tight_layout()
10 plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
11 plt.title("Prediction: {}".format(output.data.max(1, keepdim=True)[1][i].
    item()))
12 plt.xticks([])
13 plt.yticks([])
14 plt.show()

```



Figure 35. `mnist_cnn` predicts MNIST digits

Save the model to a file named `mnist_cnn.pt` using the following code:

```

1 torch.save(model.state_dict(), "mnist_cnn.pt")

```

Now you can load the model from the file using the following code:

6.8. Deploying to AWS

In this section, you will learn how to deploy the `mnist_cnn` model to an Amazon EC2 instance.

You will use AWS CloudFormation to create the EC2 instance and the `mnist_cnn` model.

```
$ cdk destroy
```



The `mnist_cnn` model is deployed to an Amazon EC2 instance with the `g4dn.xlarge` instance type, which costs \$0.71 / hour. The total cost of the deployment is \$17.

AWS Budgets

AWS Budgets (AWS Budgets) allows you to set limits on your AWS spending. You can create a budget to track your spending and receive alerts when you reach a certain threshold. For more information, see [AWS Budgets](#).

Chapter 7. Docker

本書は、Docker のインストールと基本的な操作について説明します。SSH を使ってリモートサーバーにアクセスする方法も紹介します。2

本書は3冊 (Chapter 7, Chapter 8, Chapter 9) に分けています。Chapter 7 は Docker のインストールと基本的な操作について説明します。Chapter 8 は Docker Compose の使い方について説明します。Chapter 9 は Docker Swarm の使い方について説明します。Figure 38 は Docker のアーキテクチャを示しています。AWS を使って Docker をデプロイする方法も説明します。

7.1. Docker のインストール

本書は、Docker のインストールと基本的な操作について説明します。SSH を使ってリモートサーバーにアクセスする方法も紹介します。

Chapter 5 は GPT-3 のインストールと基本的な操作について説明します。Figure 36 は GPT-3 のアーキテクチャを示しています。

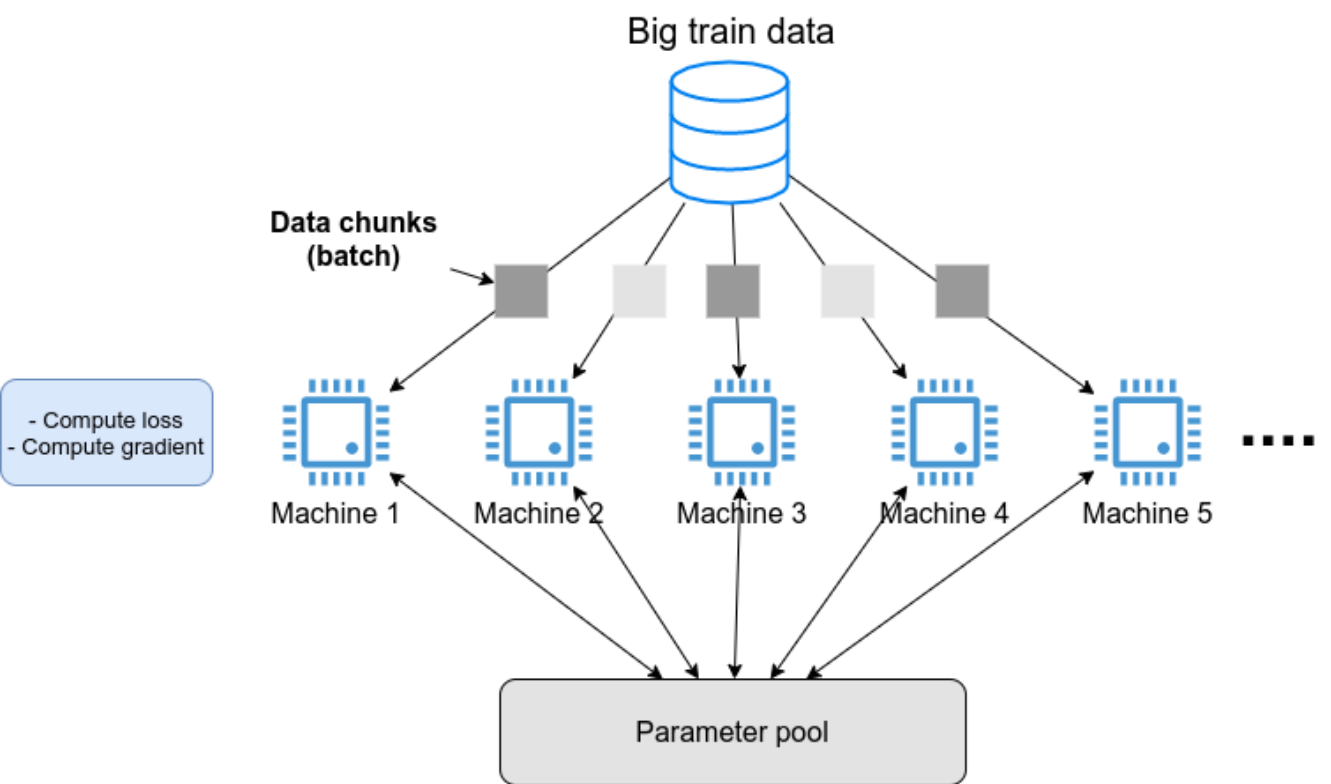


Figure 36. Distributed training architecture

本書は、Docker のインストールと基本的な操作について説明します。SSH を使ってリモートサーバーにアクセスする方法も紹介します。Figure 37 は Docker のアーキテクチャを示しています。

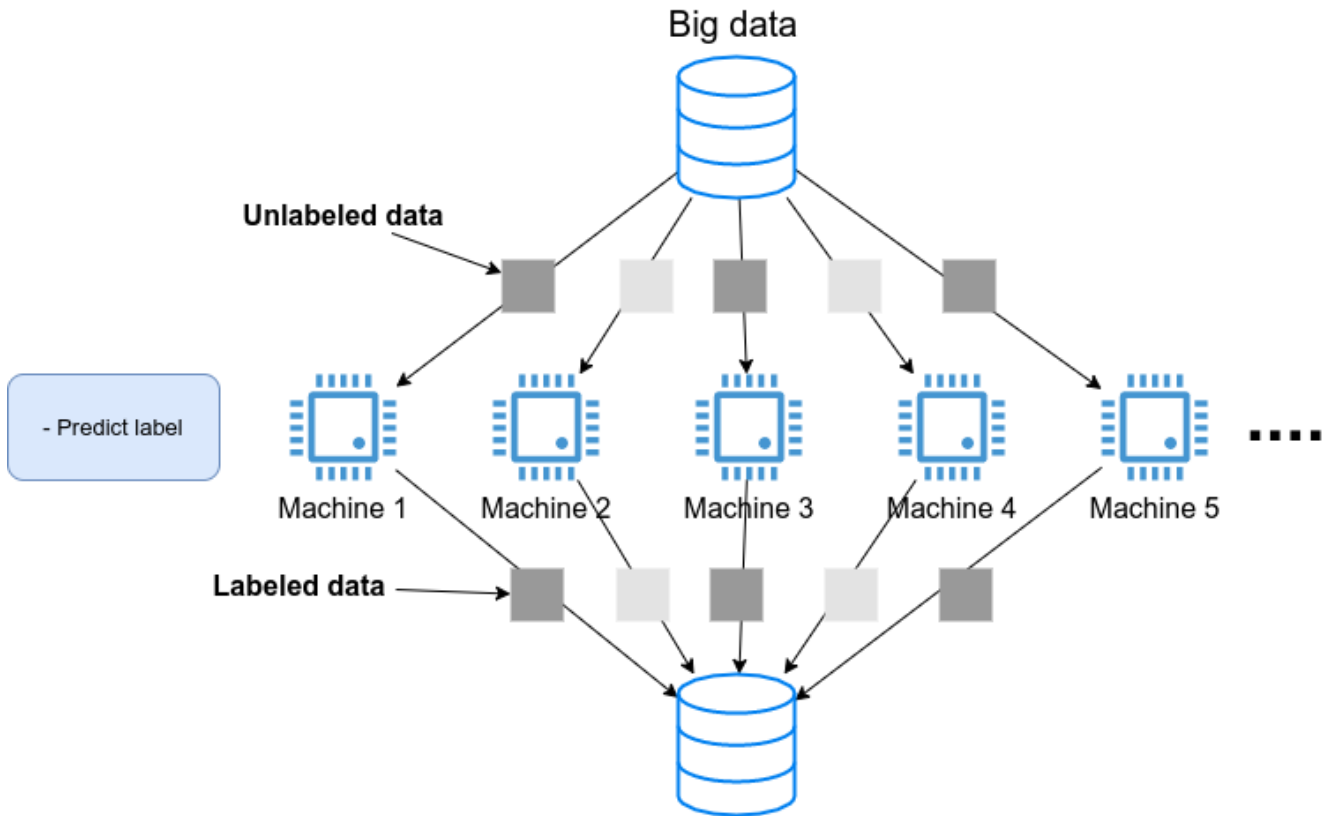


Figure 37. Distributed machine learning architecture

Figure 37 illustrates a distributed machine learning architecture. It shows a 'Big data' source at the top, which is split into unlabeled and labeled data. The unlabeled data is distributed to multiple machines (Machine 1 to Machine 5) for prediction. The labeled data is used for training or evaluation. The machines are connected to a central data source at the bottom.

Figure 36 and Figure 37 show different data distribution strategies. Figure 36 shows a single machine processing all data, while Figure 37 shows a distributed architecture where data is split across multiple machines. This is particularly useful for large datasets that cannot fit on a single machine.

Docker is a container management tool that allows you to run applications in isolated environments called containers. It is based on Linux kernel features and provides a lightweight alternative to virtual machines.

7.2. Docker



Figure 38. Docker logo

Docker is a container management tool that allows you to run applications in isolated environments called containers. It is based on Linux kernel features and provides a lightweight alternative to virtual machines. Docker containers are managed using Docker Engine, which interacts with the Docker API. The Docker API is a REST API that allows you to manage containers programmatically. Docker also provides a command-line interface (CLI) for managing containers. Docker containers are created from Docker images, which are templates for containers. Docker images are stored in a registry, such as Docker Hub. Docker containers are isolated from each other and from the host operating system. This allows you to run multiple versions of the same application on the same host without conflicts. Docker is a popular tool for deploying and managing applications in a cloud environment.

Docker was created by Solomon Hykes in 2013. It is a lightweight container management tool that allows you to run applications in isolated environments called containers. Docker containers are managed using Docker Engine, which interacts with the Docker API. The Docker API is a REST API that allows you to manage containers programmatically. Docker also provides a command-line interface (CLI) for managing containers. Docker containers are created from Docker images, which are templates for containers. Docker images are stored in a registry, such as Docker Hub. Docker containers are isolated from each other and from the host operating system. This allows you to run multiple versions of the same application on the same host without conflicts. Docker is a popular tool for deploying and managing applications in a cloud environment. Docker is a lightweight container management tool that allows you to run applications in isolated environments called containers. Docker containers are managed using Docker Engine, which interacts with the Docker API. The Docker API is a REST API that allows you to manage containers programmatically. Docker also provides a command-line interface (CLI) for managing containers. Docker containers are created from Docker images, which are templates for containers. Docker images are stored in a registry, such as Docker Hub. Docker containers are isolated from each other and from the host operating system. This allows you to run multiple versions of the same application on the same host without conflicts. Docker is a popular tool for deploying and managing applications in a cloud environment.

VM (Virtual Machine) は、OSを仮想化する (Figure 39) VM は Hypervisor (Hypervisor) によって管理される (CPU, RAM, network など) VM は OS (CPU 4コア、メモリ 2GB) VM OS OS-A OS-B CPU isolation VM VMware VirtualBox Xen EC2 VM

Docker VM OS OS Docker (Figure 39) Docker OS OS Docker OS VM (=) OS Docker VM (Docker VM) Docker

VM Docker Docker 2013

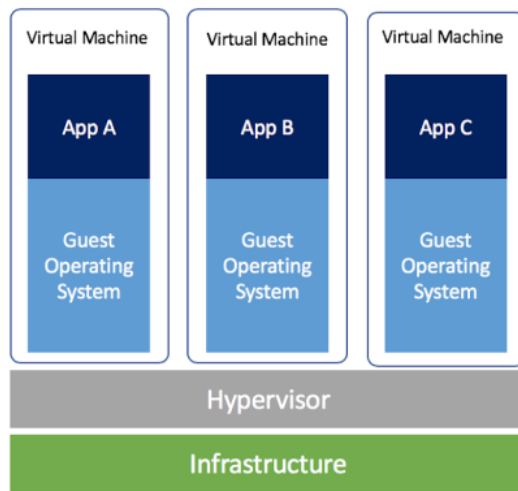
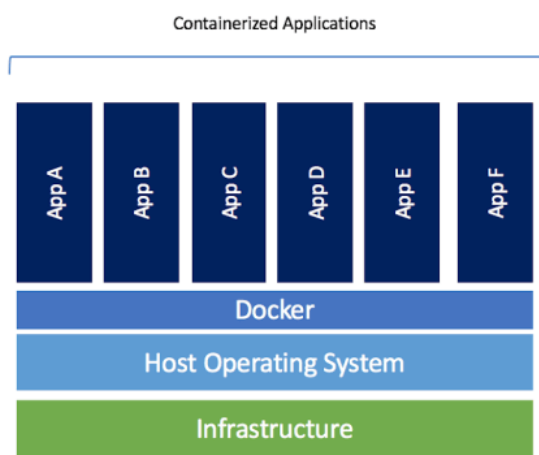


Figure 39. Docker () VM () (URL: <https://www.docker.com/blog/containers-replacing-virtual-machines/>)

Git: 開発者にとって?

開発者にとって "Git, Vim Docker" のようなツールは、開発者にとって重要なツールです。

Git は、開発者にとって重要なツールです。Linux の開発者 Linus Torvalds は、2005年に Git を開発しました。

Vim は、1991年に開発されたテキストエディタです。Stackoverflow には、2019年に Vim の開発者 Linus Torvalds が、Vim の開発について述べています。

Docker は、開発者にとって重要なツールです。Docker は、OS を仮想化するツールです。Docker は、OS を仮想化するツールです。

[illegible]

7.3. Docker □□□□□□

[illegible]

Docker Section 15.6 Docker

7.3.1. Docker

Docker

Docker 環境構築のイメージ参照 **Figure 40** Docker 環境構築のイメージ参照 **(Image)** Docker 環境構築のイメージ参照 Docker Hub 環境構築のイメージ参照 **Dockerfile** Dockerfile 環境構築のイメージ参照 **build** Docker 環境構築のイメージ参照 **(Container)** Container 環境構築のイメージ参照 **run** Docker



Figure 40. Image $\square$ Container

7.3.2. 〇〇〇〇〇〇〇〇〇〇〇〇

1. Docker イメージ (= **Image**) は [Docker Hub](#) の Docker Hub レポジトリから
 Docker イメージを [GitHub](#) レポジトリから取得する

Ubuntu Ubuntu pull

```
$ docker pull ubuntu:18.04
```

[illegible]

```
pull  Docker Hub  Docker  (registry)  Docker Hub  GitLab  GitHub
Docker Hub pull
myregistry.local:5000/testing/test-image
```

7.3.3. コンテナ

Pull コンテナイメージ `run` コマンド

```
$ docker run -it ubuntu:18.04
```

コンテナ `-it` オプションは `shell` コマンドを実行するためのオプション

コンテナイメージ `Ubuntu` をプルして実行 (Figure 41) のコンテナイメージ (コンテナ) は **Container** (コンテナ) である

```
tomoyuki@eiffel:~$ docker run -it ubuntu:18.04
root@3d7e5903b640:/#
```

Figure 41. Docker コンテナ `ubuntu:18.04` を実行

コンテナ `ubuntu:18.04` は Ubuntu OS のコンテナイメージである。Chapter 6 の DLAMI のコンテナイメージは PyTorch のコンテナイメージ `PyTorch` の Docker Hub のコンテナイメージ

コンテナイメージ

```
$ docker run -it pytorch/pytorch
```



`docker run` コマンドは Docker Hub のコンテナイメージ

`pytorch` コンテナイメージは Python のコンテナイメージ `pytorch` のコンテナイメージ

```
$ python3
Python 3.7.7 (default, May 7 2020, 21:25:33)
[GCC 7.3.0] :: Anaconda, Inc. on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> torch.cuda.is_available()
False
```

コンテナ Docker コンテナイメージ OS コンテナイメージ

7.3.4. コンテナイメージ

コンテナイメージはコンテナイメージ

コンテナ `docker` コマンドは Python, Node.js, AWS CLI, AWS CDK のコンテナイメージ

コンテナ `docker` コマンドは `Dockerfile` コマンドを実行するためのコマンド

コンテナイメージ Docker コンテナイメージ (`docker/Dockerfile`)

```
FROM node:12
LABEL maintainer="Tomoyuki Mano"

RUN apt-get update \
    && apt-get install nano

①
RUN cd /opt \
    && curl -q "https://www.python.org/ftp/python/3.7.6/Python-3.7.6.tgz" -o Python-3.7.6.tgz \
    && tar -xzf Python-3.7.6.tgz \
    && cd Python-3.7.6 \
    && ./configure --enable-optimizations \
    && make install

RUN cd /opt \
    && curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip" \
    && unzip awscliv2.zip \
    && ./aws/install

②
RUN npm install -g aws-cdk@1.100

# clean up unnecessary files
RUN rm -rf /opt/*

# copy hands-on source code in /root/
COPY handson/ /root/handson
```

Dockerfile `FROM python:3.7` `WORKDIR /app` `ADD . /app` `ENTRYPOINT ["python", "main.py"]`

"oooooooooooooooooooo..." oooooooooooooooooooooooooooooo Docker oooooooooooooooooooooooooooooo
oooooooooooooooooooooooooooo Docker oooooooooooooooooooooo

☐☐☐: Is Docker alone?

1. 在 Docker 中安装 API 接口
 2. 在 Docker 中安装 Docker
 3. 在 Docker 中安装 Docker

Singularity  **HPC (High Performance Computing)**  **Singularity** 
 **HPC**  **Docker**  **root** 
Singularity  (Singularity)  **root**  **Web**
 **HPC** 
 **Singularity**  **Singularity** 

podman Red Hat podman Docker
Red Hat podman Singularity
HPC pod

Docker

7.4. Elastic Container Service (ECS)



Figure 42. ECS

Docker AWS Docker

Elastic Container Service (ECS) Docker AWS (Figure 42) ECS Docker (=)

ECS Figure 43 ECS (Task) ECS Docker Docker Hub AWS Docker ECR (Elastic Container Registry)

ECS Docker "Docker Hub" Docker

ECS ECS (80%) scale-out (scale-in) ECS AWS Auto scaling group (ASG) Fargate ASG Chapter 9, Fargate Chapter 8

ECS (Docker Hub) Docker

Docker AWS

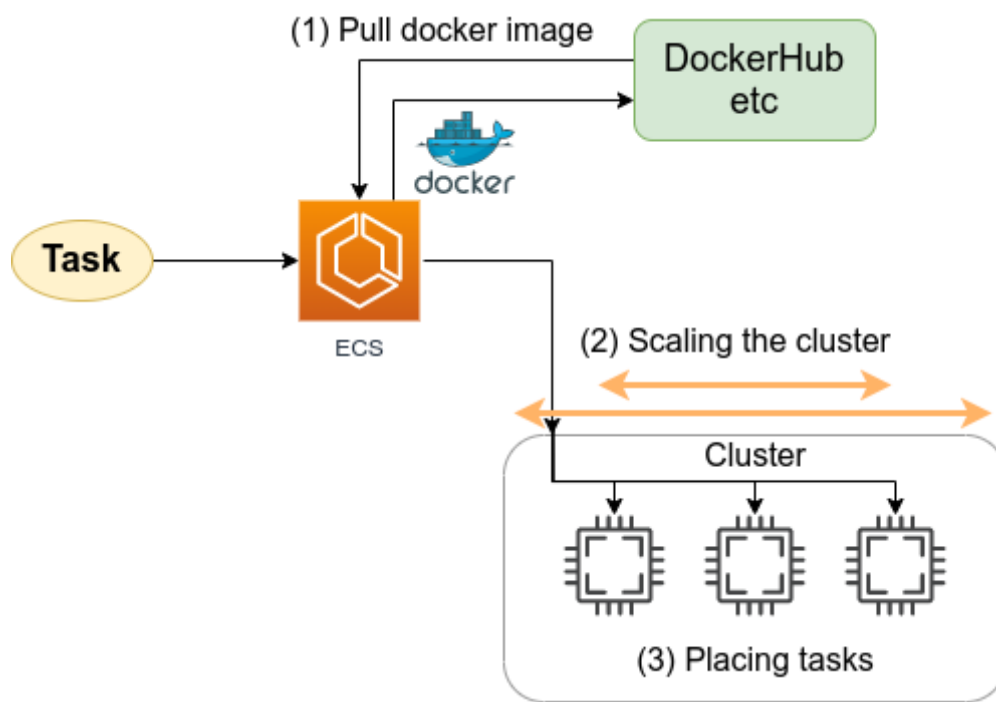


Figure 43. ECS

8.2.

이 코드를 실행하려면 GitHub 에서 [handson/qa-bot](#) 을

클론하고 (Section 4.1) Docker 를 사용하여 실행합니다.



이 코드는 1CPU/4GB RAM 을 Fargate 에 배포할 수 있으며, 가격은 0.025 \$/hour 입니다.

8.3. Transformer 를 사용하여 question-answering

이 코드는 context (context) 와 question (question) 을 사용하여

context: Albert Einstein (14 March 1879 - 18 April 1955) was a German-born theoretical physicist who developed the theory of relativity, one of the two pillars of modern physics (alongside quantum mechanics). His work is also known for its influence on the philosophy of science. He is best known to the general public for his mass-energy equivalence formula $E = mc^2$, which has been dubbed "the world's most famous equation". He received the 1921 Nobel Prize in Physics "for his services to theoretical physics, and especially for his discovery of the law of the photoelectric effect", a pivotal step in the development of quantum theory.

question: In what year did Einstein win the Nobel prize?

이 코드는 context 를 사용하여 question 을 처리합니다.

answer: 1921

이 코드는 Transformer 를 사용하여 question 을 처리합니다.

이 코드는 [huggingface/transformers](#) 라이브러리를 사용하여 Q&A 문제를 Transformer (Figure 45)로 처리합니다. Docker 이미지를 Docker Hub 에서

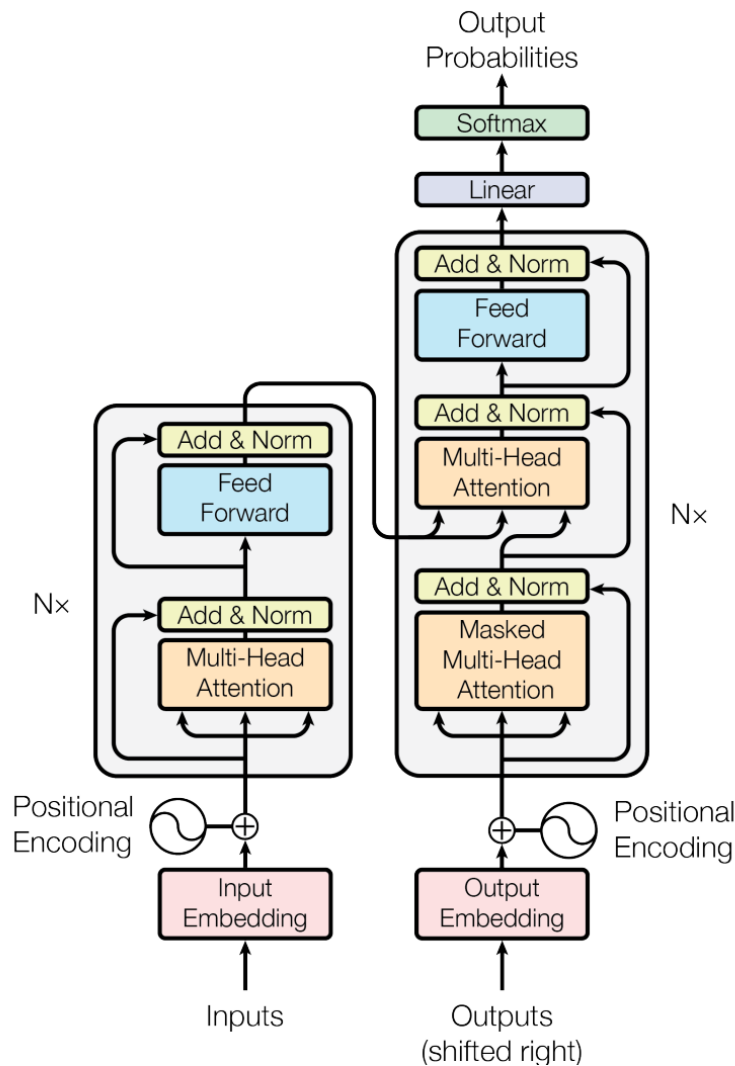


Figure 45. Transformer (Source: Vaswani+ 2017)



Transformer is a deep learning architecture that can be used for both sequence-to-sequence (Seq2Seq) and sequence-to-text (Seq2Text) tasks. It is a type of neural network that uses self-attention mechanisms to process input sequences. The architecture is designed to be highly parallelizable, allowing it to be trained on large datasets using multiple GPUs. The Transformer model is implemented in the Hugging Face library, which provides a simple and easy-to-use interface for working with the model. The model can be trained on a CPU or GPU, and it can be used to generate text or to classify sequences.

Transformer Docker image (pull) command

```
$ docker pull tomomano/qabot:latest
```

pull Transformer Docker image context question

```
$ context="Albert Einstein (14 March 1879 - 18 April 1955) was a German-born theoretical physicist who developed the theory of relativity, one of the two pillars of modern physics (alongside quantum mechanics). His work is also known for its influence on the philosophy of science. He is best known to the general public for his mass-energy equivalence formula  $E = mc^2$ , which has been dubbed the world's most famous equation. He received the 1921 Nobel Prize in Physics for his services to theoretical physics, and especially for his discovery of the law of the photoelectric effect, a pivotal step in the development of quantum theory."
$ question="In what year did Einstein win the Nobel prize ?"
```

□ □

```
$ docker run tomomano/qabot "${context}" "${question}" foo --no_save
```

docker image context question

□ □

```
{'score': 0.9881729286683587, 'start': 437, 'end': 441, 'answer': '1921'}
```

"score" 0.000000000000 [0,1] "start", "end" context "answer"
 1921

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
$ question="Why did Einstein win the Nobel prize ?"
$ docker run tomomano/qabot "${context}" "${question}" foo --no_save
```

111

```
{'score': 0.5235594527494207, 'start': 470, 'end': 506, 'answer': 'his services to  
theoretical physics,'}
```

score = 0.52

Q&A



Question & Answering DistilBERT Transformer
 DistilBert transformers huggingface/distilbert



Q-A Docker 参考 <https://github.com/tomomano/learn-aws-by-coding/blob/main/handson/ga-bot/docker/Dockerfile>

8.4. □□□□□□□□□□

Figure 46


```

11         name="item_id", type=dynamodb.AttributeType.STRING
12     ),
13     billing_mode=dynamodb.BillingMode.PAY_PER_REQUEST,
14     removal_policy=cdk.RemovalPolicy.DESTROY
15 )
16
17 ②
18 vpc = ec2.Vpc(
19     self, "EcsClusterQaBot-Vpc",
20     max_azs=1,
21 )
22
23 ③
24 cluster = ecs.Cluster(
25     self, "EcsClusterQaBot-Cluster",
26     vpc=vpc,
27 )
28
29 ④
30 taskdef = ecs.FargateTaskDefinition(
31     self, "EcsClusterQaBot-TaskDef",
32     cpu=1024, # 1 CPU
33     memory_limit_mib=4096, # 4GB RAM
34 )
35
36 # grant permissions
37 table.grant_read_write_data(taskdef.task_role)
38 taskdef.add_to_task_role_policy(
39     iam.PolicyStatement(
40         effect=iam.Effect.ALLOW,
41         resources=["*"],
42         actions=["ssm:GetParameter"]
43     )
44 )
45
46 ⑤
47 container = taskdef.add_container(
48     "EcsClusterQaBot-Container",
49     image=ecs.ContainerImage.from_registry(
50         "tomomano/qabot:latest"
51     ),
52 )

```

① DynamoDB の作成

② #1, #2 VPC の作成

③ ECS クラスター (cluster) の作成

④ タスク定義 (task definition)

⑤ コンテナイメージ Docker の作成

8.4.1. ECS の Fargate

ECS の Fargate を使ってみよう

```
1 cluster = ecs.Cluster(  
2     self, "EcsClusterQaBot-Cluster",  
3     vpc=vpc,  
4 )  
5  
6 taskdef = ecs.FargateTaskDefinition(  
7     self, "EcsClusterQaBot-TaskDef",  
8     cpu=1024, # 1 CPU  
9     memory_limit_mib=4096, # 4GB RAM  
10 )  
11  
12 container = taskdef.add_container(  
13     "EcsClusterQaBot-Container",  
14     image=ecs.ContainerImage.from_registry(  
15         "tomomano/qabot:latest"  
16     ),  
17 )
```

`cluster` = ECS クラスターオブジェクト

`taskdef=ecs.FargateTaskDefinition` は Fargate を使ったタスク定義で、1 CPU、4GB RAM のコンテナを動かすことができる。1024 CPU、4096 MB RAM の制限がある。

`container` = Docker image を指定して、Docker Hub から image を取得する。

このコードを実行すると、ECS クラスターが作成され、タスク定義が作成される。



`cpu=1024` は CPU の制限値で、1 CPU は 1024 vCPU に相当する。vCPU (virtual CPU) は 0.25 から 0.5 vCPU の単位で指定できる。1/4, 1/2 の CPU は指定できない。1024 CPU は 2 から 8 GB のメモリ制限がある。詳しくは ["Amazon ECS on AWS Fargate"](#) を参照。

Table 4. CPU の制限値とメモリ制限値

CPU の制限値	メモリ制限値
256 (.25 vCPU)	0.5 GB, 1 GB, 2 GB
512 (.5 vCPU)	1 GB, 2 GB, 3 GB, 4 GB
1024 (1 vCPU)	2 GB, 3 GB, 4 GB, 5 GB, 6 GB, 7 GB, 8 GB
2048 (2 vCPU)	Between 4 GB and 16 GB in 1-GB increments

4096 (4 vCPU)

Between 8 GB and 30 GB in 1-GB increments

8.5. 環境構築

環境構築の手順を以下に示す。

環境構築の手順は、SSH で環境に接続し、以下のコマンドを実行する。 (# はプロンプト、\$ はコマンドライン) 1, 2 は、必要に応じて、(Section 15.3) の手順に従って行う。

```
# 環境構築
$ cd handson/qa-bot

# venv の作成
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# デプロイ
$ cdk deploy
```

環境構築の手順は、Figure 47 の通りである。

```
✓ EcsClusterQaBot
Outputs:
EcsClusterQaBot.ClusterName = EcsClusterQaBot-EcsClusterQaBotCluster6488E31F-
EcsClusterQaBot.TaskDefinitionArn = arn:aws:ecs:ap-northeast-1:606887060834:task-definition/EcsClusterQaBotEcsClusterQaBotTaskDef:
Stack ARN:
arn:aws:cloudformation:ap-northeast-1:606887060834:stack/EcsClusterQaBot/f89b5980-b42d-
ac70
(.env) tomoyuki@balthasar:03-qa-bot$
```

Figure 47. CDK 環境構築

AWS 環境に ECS クラスターを作成する。 ECS クラスターは、Figure 48 の通り、EcsClusterQaBot-XXXX と命名する。

クラスターのタスク定義は、Figure 48 の FARGATE タスク定義で、0 Running tasks, 0 Pending tasks と表示される。

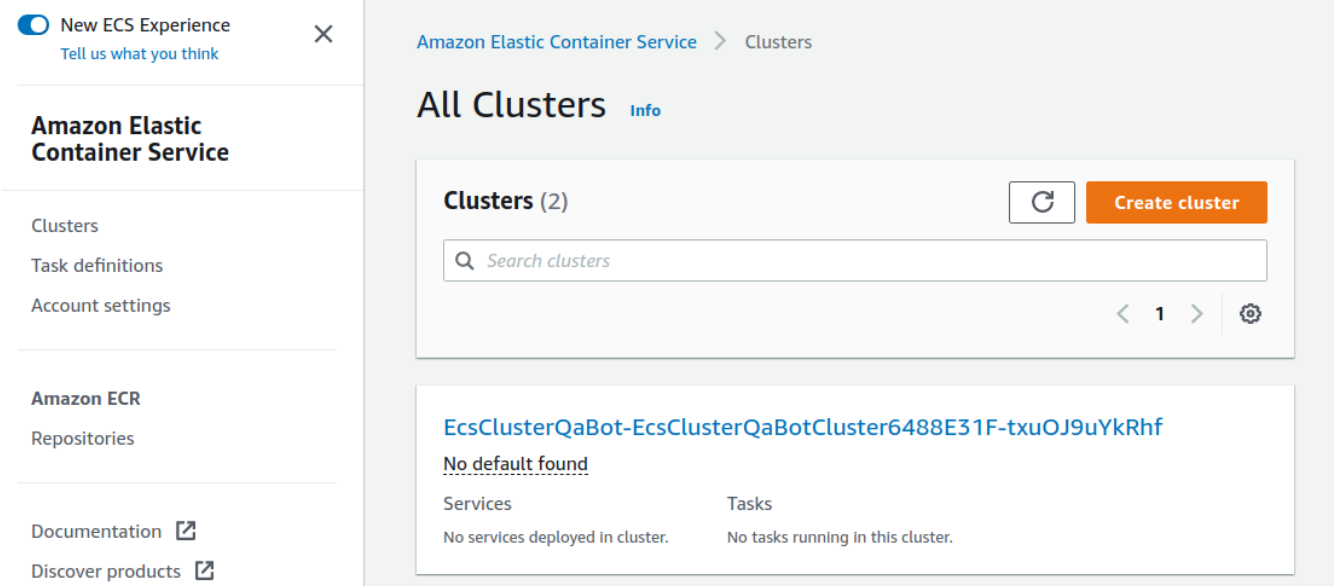


Figure 48. ECS console

Task Definitions

EcsClusterQaBotEcsClusterQaBotTaskDefXXXX

CPU Docker container Task Definition

Task size ?

The task size allows you to specify a fixed size for your task. Task size is required for tasks using the Fargate launch type and is optional for the EC2 or External launch type. Container level memory settings are optional when task size is set. Task size is not supported for Windows containers.

Task memory (MiB) 4096

Task CPU (unit) 1024

Task memory maximum allocation for container memory reservation

0 4096 shared of 4096 MiB

Task CPU maximum allocation for containers

0 1024 shared of 1024 CPU units

Task Placement

Constraint No constraints

Container Definitions

Container ...	Image	CP...	GP...	Infe...	Hard/Soft memory limits (MiB)	Ess...
▼ EcsClu...	tomomano/qabot:latest	0			--/--	true

Figure 49. Task definition

8.6. 実装

実装の概要を説明します。

ECS のタスクを実行するためのスクリプト (`run_task.py`) を作成します ([handson/qa-bot/run_task.py](#))。

このスクリプトは ECS のタスクを実行するためのものです。

```
$ python run_task.py ask "A giant peach was flowing in the river. She picked it up and brought it home. Later, a healthy baby was born from the peach. She named the baby Momotaro." "What is the name of the baby?"
```



`run_task.py` は ECS のタスクを実行するためのスクリプトです。

"ask" は context (context) と question (question) を実行します。

このスクリプトは "Waiting for the task to finish..." というメッセージを出力し、AWS の ECS サービスを使用して Fargate を実行し、Docker イメージを AWS の ECS サービスに実行させます。

この ECS サービスは "Tasks" という名前で実行されます (Figure 50)。

Task	Last status	Description	Task definition	Revision	Started at	Started at
aad78e...	Pending	-	EcsClusterQaBo	8	-	-
b1a869...	Stopped	CannotPullCon...	EcsClusterQaBo	6	-	-

Figure 50. ECS のタスクの実行状況

Figure 50 の "Last status = Pending" は、Fargate が Docker イメージをプルしている状態を示しています。

この Status は "RUNNING" であり、Status は "STOPPED" であり、ECS サービスは Fargate を実行しています。

Figure 50 の "Task" は、Task ID (Figure 51) の "Last status", "Platform version" を示しています。また、"Logs" は、Task のログを示しています。


```
$ python run_task.py ask_many
```

Amazon ECS を使ったタスクの実行 (Figure 53) は、Amazon Fargate を使ったタスクの実行と似ています。

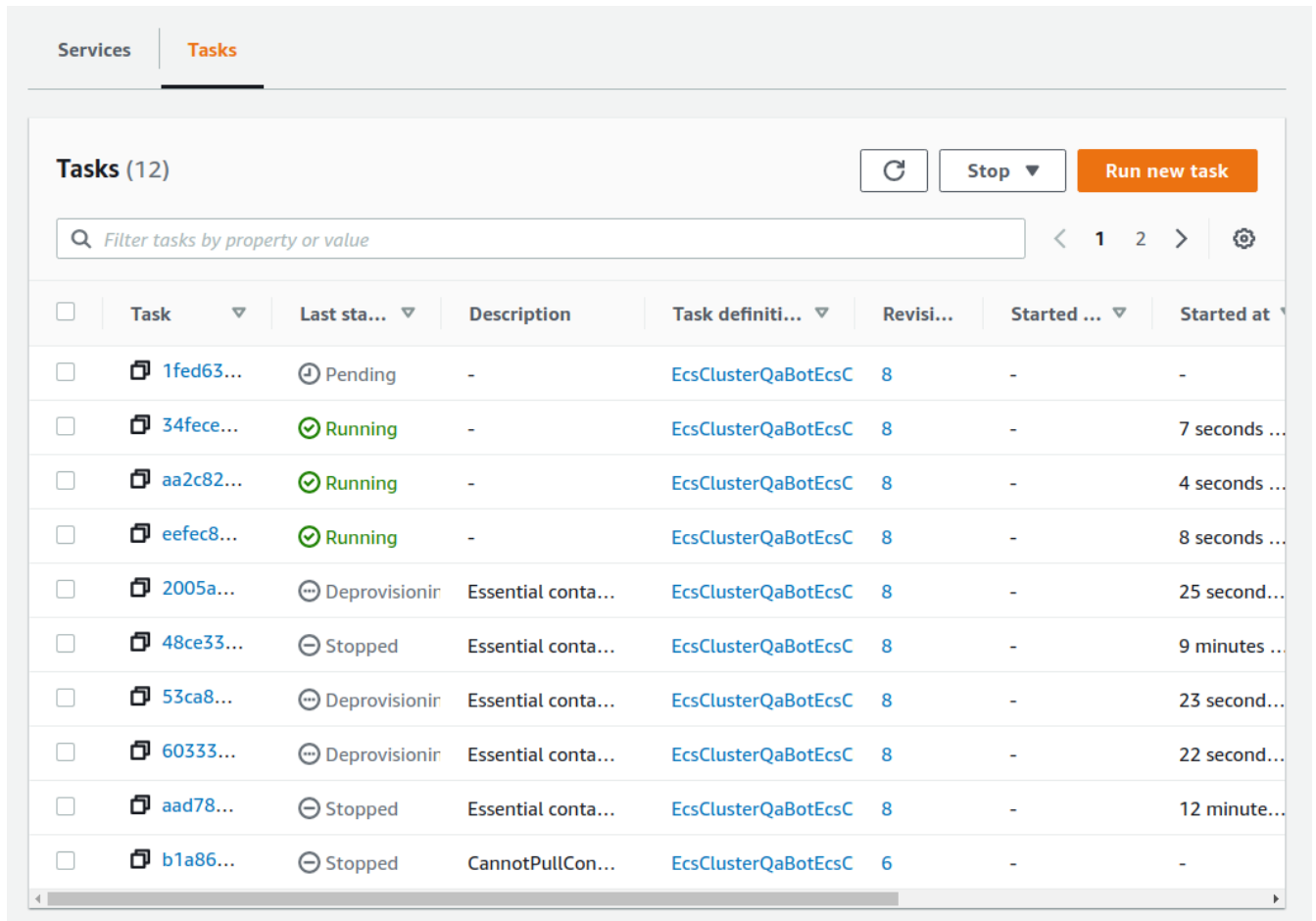


Figure 53. タスクの実行状況

タスクのステータスが "STOPPED" の場合は、タスクの再実行が可能です。

```
$ python run_task.py list_answers
```

Figure 54 は、タスクの実行結果を示しています。

Figure 54. `$ python run_task.py list_answers`

[illegible]

```
$ python run_task.py clear
```

[illegible]

```
$ cdk destroy
```

[illegible][illegible]

Auto scaling groups (ASG) EC2

EC2 インスタンス **ASG** のインスタンスを削除すると ASG が EC2 インスタンスを自動的に置き換えます。ASG のインスタンスを置き換えるときに、**desired capacity**、**minimum capacity**、**maximum capacity** の値が、minimum capacity から maximum capacity の範囲内にない場合は、ASG はエラーを返します。

[illegible]

77



Figure 55. AWS Batch

EC2 ASG ECS ASG
ECS ASG
AWS Batch

AWS Batch (Batch) ()
AWS Batch
ECS/ASG/EC2

AWS Batch (Figure 56) Job AWS Batch
Job definitions Docker
CPU/RAM Job definition Job queues
Job queues priority queue queue
priority () priority queue
(" ") Compute environment
(EC2 Fargate) Compute environment EC2
Job queues Compute environment
Compute environment

AWS Batch

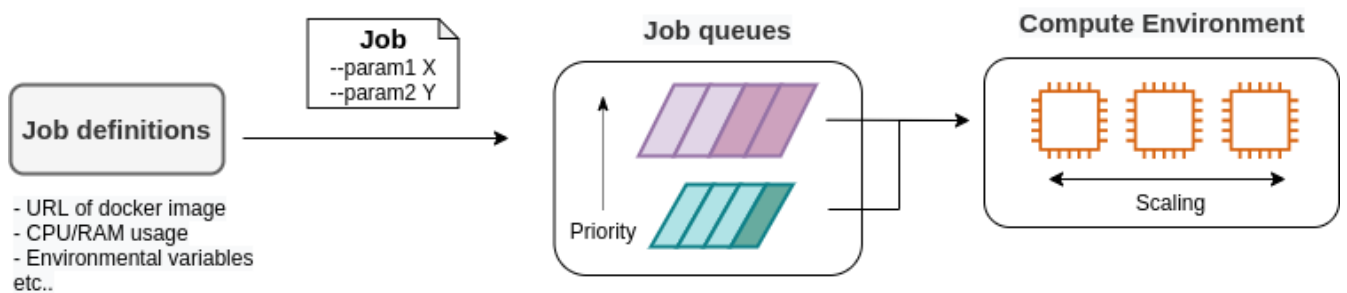


Figure 56. AWS Batch

EC2 or Fargate?



ECS EC2 Fargate
Table 5
Fargate

Table 5. EC2 vs Fargate

	EC2	Fargate
Compute capacity	Medium to large	Small to medium

	EC2	Fargate
GPU	Yes	No
Launch speed	Slow	Fast
Task placement flexibility	Low	High
Programming complexity	High	Low

EC2 CPU GPU
Fargate CPU 4
Fargate
Fargate
2
CPU
Fargate

EC2 Fargate EC2 Fargate

9.3.

GitHub [handson/aws-batch](#)

(Section 4.1) Docker



`g4dn.xlarge` EC2 (`us-east-1`) 0.526 \$/hour
(`ap-northeast-1`) 0.71 \$/hour



Section 6.1 G AWS EC2
Section 9.5

9.4. MNIST ()

Section 6.7 MNIST
Section 6.7
(SGD)

```
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.5)
```

(`lr=0.01`) (`momentum=0.5`)

[illegible][illegible][illegible][illegible]

00
000

[illegible]

Embarassingly parallel (parallel "embarrassingly parallel")

optuna
GPU

○○○○○○○○○○ Docker ○○○○○○○○○○○○○○○○○○○

Docker [handson/aws-batch/docker](#) Section 6.7

00000000 Docker 000000000000000000000000 Dockerfile 00000000000000000000 mymnist 000000 (Tag)
 000000000000000000

```
$ cd hands-on/aws-batch/docker
$ docker build -t mymnist .
```



docker build `mnist` `mnist` **MNIST** `mnist`
<http://yann.lecun.com/exdb/mnist/> `mnist`
`mnist` `mnist` `mnist`
`mnist`



~~~~~ Docker Hub ~~~ pull ~~~~~

```
$ docker pull tomomano/mymnist:latest
```

□□□□□□□□□□□□□□□□□□□□□□□□ MNIST □□□□□□□□□□

```
$ docker run mymnist --lr 0.1 --momentum 0.5 --epochs 10
```

ターミナル画面の出力結果 ( `--lr` の値、 `--momentum` の値、 `--epochs` の値 ) は、 [Chapter 6](#) の `mnist` の `Loss` の値と `Accuracy` の値 ( [Figure 57](#) ) と一致する。

```
Train Epoch: 0 [0/48000 (0.0%)] Loss: 2.297341
Train Epoch: 0 [6400/48000 (13.3%)] Loss: 0.298299
Train Epoch: 0 [12800/48000 (26.7%)] Loss: 0.083849
Train Epoch: 0 [19200/48000 (40.0%)] Loss: 0.252932
Train Epoch: 0 [25600/48000 (53.3%)] Loss: 0.160509
Train Epoch: 0 [32000/48000 (66.7%)] Loss: 0.082315
Train Epoch: 0 [38400/48000 (80.0%)] Loss: 0.153711
Train Epoch: 0 [44800/48000 (93.3%)] Loss: 0.222485

Val set: Average loss: 0.0733, Accuracy: 97.8%

Train Epoch: 1 [0/48000 (0.0%)] Loss: 0.165711
Train Epoch: 1 [6400/48000 (13.3%)] Loss: 0.136394
Train Epoch: 1 [12800/48000 (26.7%)] Loss: 0.089186
Train Epoch: 1 [19200/48000 (40.0%)] Loss: 0.095106
Train Epoch: 1 [25600/48000 (53.3%)] Loss: 0.025505
Train Epoch: 1 [32000/48000 (66.7%)] Loss: 0.061345
Train Epoch: 1 [38400/48000 (80.0%)] Loss: 0.163712
Train Epoch: 1 [44800/48000 (93.3%)] Loss: 0.122928

Val set: Average loss: 0.0552, Accuracy: 98.4%
```

Figure 57. Docker の出力結果

ターミナル画面の出力結果 CPU のみで実行された場合と GPU を利用して実行された場合 ( `nvidia-docker` ) の出力結果は、ほぼ一致する。

```
$ docker run --gpus all mymnist --lr 0.1 --momentum 0.5 --epochs 10
```

ターミナル画面の出力結果 `--gpus all` の値は、 `all` である。

CPU/GPU の出力結果は、ほぼ一致する ( `Train` の `Loss` の値と `Validation` の `Loss` の値と `Accuracy` の値 )。 [Figure 58](#) の出力結果と一致する。

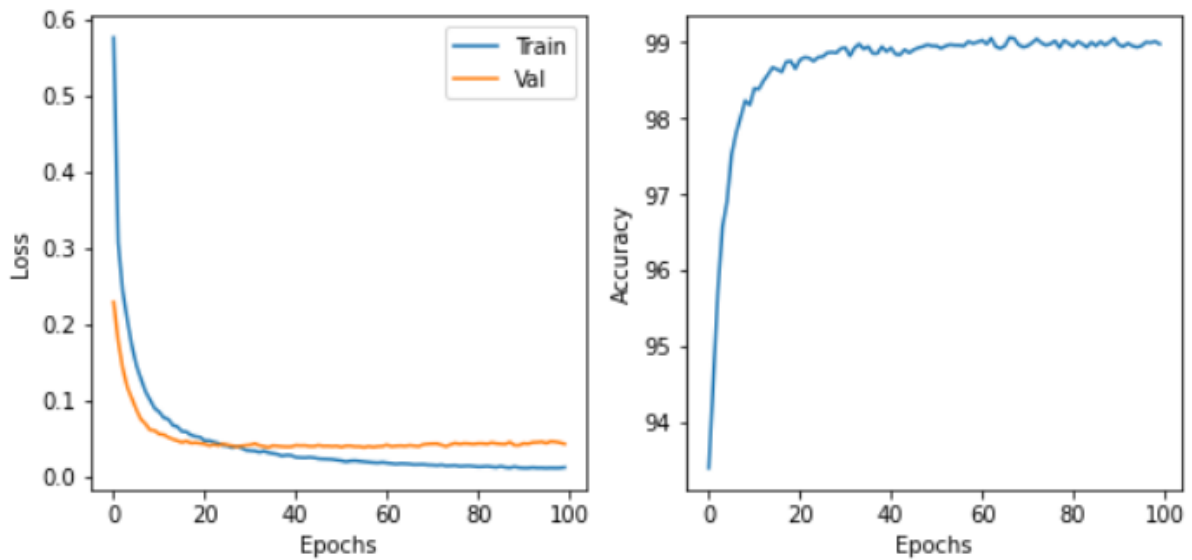


Figure 58. (a) Train/Validation Loss (b) Validation Accuracy

Early stopping is a technique used to prevent overfitting. It involves monitoring the validation loss and stopping the training process when the loss stops improving for a certain number of epochs. This helps to ensure that the model is not overfitting to the training data and is instead learning to generalize to new, unseen data.



MNIST dataset is often used for digit recognition tasks. It contains 60,000 training images and 10,000 test images. The training set is typically split into 80% (48,000 images) and 20% (12,000 images) for validation and testing purposes.

## 9.5. Early stopping

Figure 59 shows the training and validation loss curves for a model trained on the MNIST dataset. The training loss decreases rapidly, while the validation loss starts to increase after approximately 40 epochs, indicating overfitting.

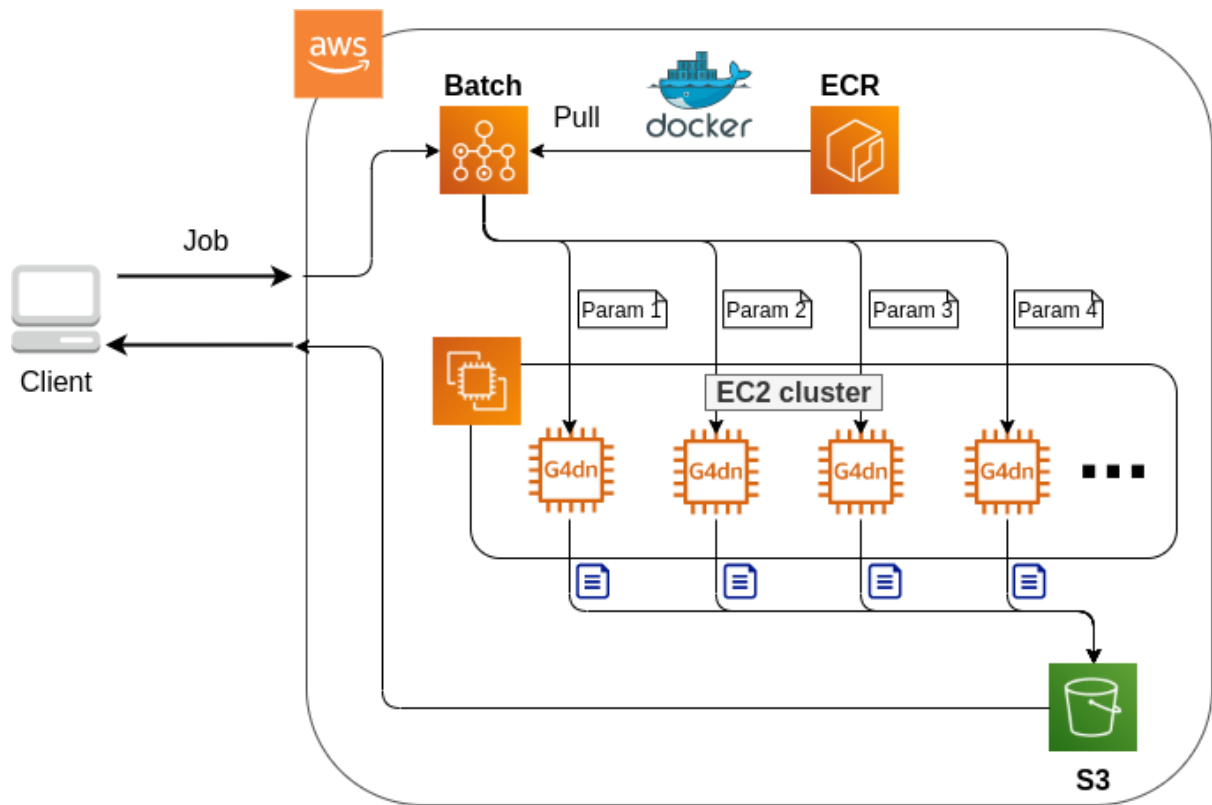


Figure 59.

- 
- Batch
- Batch
- 
- Docker
- 
- 
- 

(handson/aws-batch/app.py)

```

1 class SimpleBatch(Stack):
2
3     def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
4         super().__init__(scope, construct_id, **kwargs)
5
6         ①
7         bucket = s3.Bucket(
8             self, "bucket",
9             removal_policy=cdk.RemovalPolicy.DESTROY,
10            auto_delete_objects=True,
11        )
12
13        vpc = ec2.Vpc(

```

```

14         self, "vpc",
15         # other parameters...
16     )
17
18     ②
19     managed_env = batch_alpha.ComputeEnvironment(
20         self, "managed-env",
21         compute_resources=batch_alpha.ComputeResources(
22             vpc=vpc,
23             allocation_strategy=batch_alpha.AllocationStrategy.BEST_FIT,
24             desiredv_cpus=0,
25             maxv_cpus=64,
26             minv_cpus=0,
27             instance_types=[
28                 ec2.InstanceType("g4dn.xlarge")
29             ],
30         ),
31         managed=True,
32         compute_environment_name=self.stack_name + "compute-env"
33     )
34
35     ③
36     job_queue = batch_alpha.JobQueue(
37         self, "job-queue",
38         compute_environments=[
39             batch_alpha.JobQueueComputeEnvironment(
40                 compute_environment=managed_env,
41                 order=100
42             )
43         ],
44         job_queue_name=self.stack_name + "job-queue"
45     )
46
47     ④
48     job_role = iam.Role(
49         self, "job-role",
50         assumed_by=iam.CompositePrincipal(
51             iam.ServicePrincipal("ecs-tasks.amazonaws.com")
52         )
53     )
54     # allow read and write access to S3 bucket
55     bucket.grant_read_write(job_role)
56
57     ⑤
58     repo = ecr.Repository(
59         self, "repository",
60         removal_policy=cdk.RemovalPolicy.DESTROY,
61     )
62
63     ⑥
64     job_def = batch_alpha.JobDefinition(

```

```

65         self, "job-definition",
66         container=batch_alpha.JobDefinitionContainer(
67             image=ecs.ContainerImage.from_ecr_repository(repo),
68             command=["python3", "main.py"],
69             vcpus=4,
70             gpu_count=1,
71             memory_limit_mib=12000,
72             job_role=job_role,
73             environment={
74                 "BUCKET_NAME": bucket.bucket_name
75             }
76         ),
77         job_definition_name=self.stack_name + "job-definition",
78         timeout=cdk.Duration.hours(2),
79     )

```

- ① 環境変数BUCKET\_NAMEはS3バケット名を指定
- ② Compute environmentの最大vCPU数はg4dn.xlargeインスタンスの最大vCPU数(64)と環境変数MAXIMUM\_VCPUS(0)のいずれか小さい値で指定
- ③ <2>のCompute environmentはJob queueに紐付け
- ④ S3バケットにアクセスするためのIAMロールを指定 (IAMロールの作成方法は [Section 13.2.5](#) 参照)
- ⑤ Docker imageはECRに格納されたイメージを指定
- ⑥ Job definitionのvCPUは4、メモリは12000 MB (=12GB) と RAM 制限を指定。環境変数BUCKET\_NAMEはS3バケット名を指定。<4>のIAMロールを指定



g4dn.xlarge は最大 4 vCPU、最大 16 GB のメモリを持つ Compute environment の maximum vCPUs は 64、maximum memory in GiB は 16384 です。AWS の制限は [AWS Limits](#) ページで確認できます。

AWS の EC2 インスタンスの制限は [AWS Limits](#) ページで確認できます。g4dn.xlarge (EC2 インスタンス G 型) は Running On-Demand All G instances の制限に適用されます。AWS の制限は [AWS Limits](#) ページで確認できます。また、[Amazon EC2 service quotas](#) ページで確認できます。



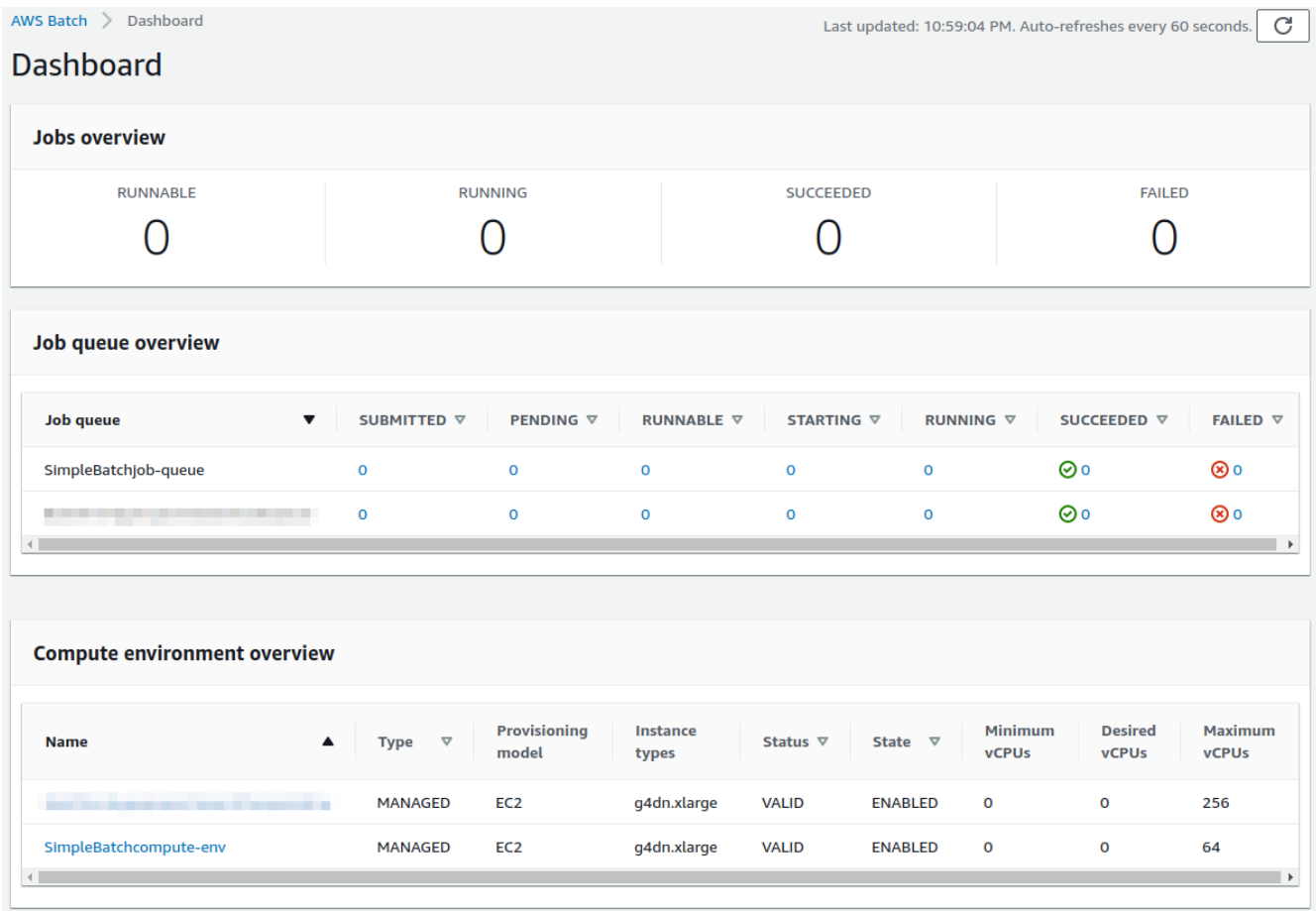


Figure 61. AWS Batch 画面 (ダッシュボード)

画面の「Compute environment overview」タブで「SimpleBatchcompute-env」を選択すると、Compute environment の詳細が表示されます (Figure 62)。この画面で、g4dn.xlarge の Instance types、Minimum vCPUs、Maximum vCPUs (64)、Desired vCPUs (0) が確認できます。Compute environment の詳細については、AWS Batch のドキュメントを参照してください。

画面の「Job queue overview」タブで「SimpleBatch-queue」を選択すると、Job queue の詳細が表示されます。この画面で、PENDING, RUNNING, SUCCEEDED, FAILED の状態が確認できます。Job queue の詳細については、AWS Batch のドキュメントを参照してください。

画面の「Job definition」タブで「SimpleBatchjob-definition」を選択すると、Job definition の詳細が表示されます (Figure 62)。この画面で、vCPUs, Memory, GPU のリソース設定、Image (Docker イメージ)、ECR (Elastic Container Registry) の設定が確認できます。Job definition の詳細については、AWS Batch のドキュメントを参照してください。

| Container properties   |                |
|------------------------|----------------|
| Command                | Image          |
| ["python3", "main.py"] |                |
| User                   | vCpus          |
| --                     | 4              |
| Read only filesystem   | Memory         |
| --                     | 12000          |
| Instance type          | Number of GPUs |
|                        | 1              |

Figure 62. AWS Batch の Job definition の設定

## 9.7. Docker image を ECR にアップロード

この Batch のデフォルトコンテナイメージは Docker Hub から pull される (Chapter 8) のイメージである。Docker Hub から pull するのではなく、AWS のコンテナイメージレジストリ **ECR (Elastic Container Registry)** から image を取得する。ECR は、コンテナイメージを保存、配布するためのサービスである。Batch が ECR から pull するイメージ (Figure 59)。

このセクションでは ECR の使用方法を説明する。

```

①
repo = ecr.Repository(
    self, "repository",
    removal_policy=cdk RemovalPolicy.DESTROY,
)

job_def = batch_alpha.JobDefinition(
    self, "job-definition",
    container=batch_alpha.JobDefinitionContainer(
        image=ecs.ContainerImage.from_ecr_repository(repo), ②
        ...
    ),
    ...
)

```

① ECR のリポジトリを作成

② Job definition のコンテナイメージとして <1> の ECR のリポジトリから取得したイメージを Job definition のコンテナイメージとして IAM ロールに付与する

このセクションでは ECR のイメージを Docker から push する方法を説明する。

このセクションでは AWS の ECR サービス (Elastic Container Registry) の Private レジストリ `simplebatch-repositoryXXXXXX` を作成する方法を説明する (Figure 63)。

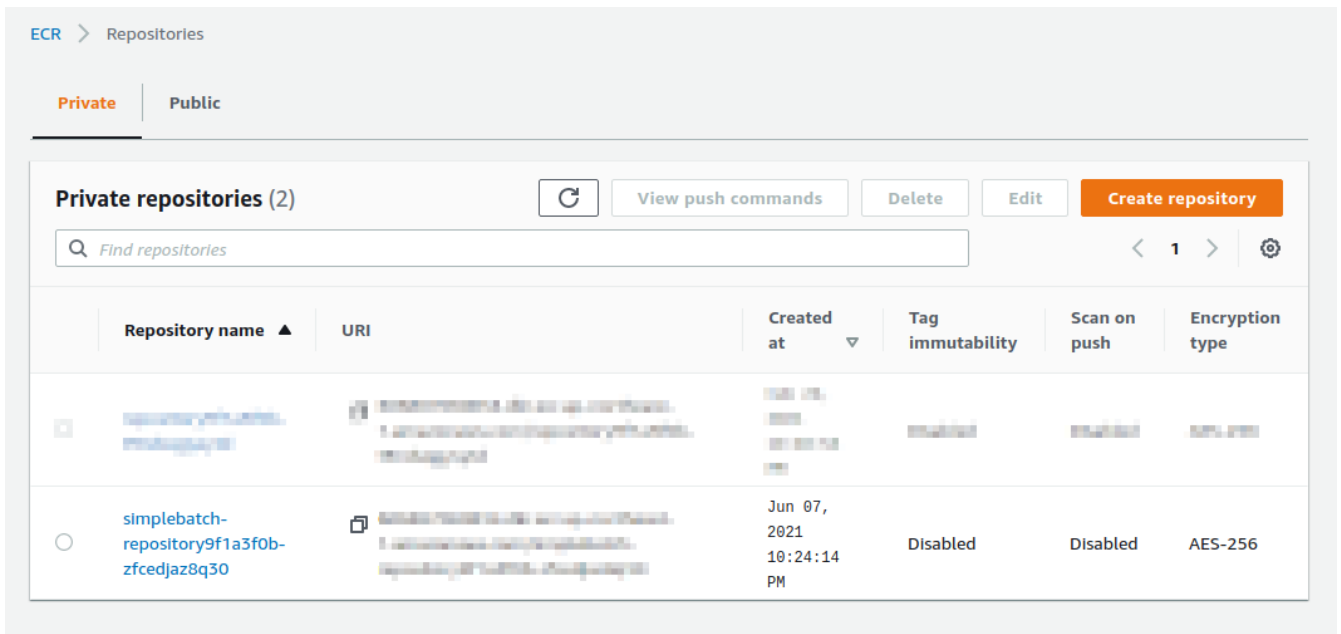


Figure 63. ECR Private repositories

The screenshot shows the ECR Private repositories page. The first repository is 'simplebatch-repository9f1a3f0b-zfcedjaz8q30', created on Jun 07, 2021, with tag immutability disabled and scan on push disabled. The second repository is 'simplebatch-repository9f1a3f0b-zfcedjaz8q30', created on Jun 07, 2021, with tag immutability disabled and scan on push disabled. The page includes a search bar, a 'Find repositories' button, and a 'Create repository' button.

Push commands for simplebatch-repository9f1a3f0b-zfcedjaz8q30

macOS / Linux

Windows

Make sure that you have the latest version of the AWS CLI and Docker installed. For more information, see [Getting Started with Amazon ECR](#).

Use the following steps to authenticate and push an image to your repository. For additional registry authentication methods, including the Amazon ECR credential helper, see [Registry Authentication](#).

- Retrieve an authentication token and authenticate your Docker client to your registry.  
Use the AWS CLI:  

```
aws ecr get-login-password --region ap-northeast-1 | docker login --username AWS --password-
```

Note: If you receive an error using the AWS CLI, make sure that you have the latest version of the AWS CLI and Docker installed.
- Build your Docker image using the following command. For information on building a Docker file from scratch see the instructions [here](#). You can skip this step if your image is already built:  

```
docker build -t simplebatch-repository9f1a3f0b-zfcedjaz8q30 .
```
- After the build completes, tag your image so you can push the image to this repository:  

```
docker tag simplebatch-repository9f1a3f0b-zfcedjaz8q30:latest
```
- Run the following command to push this image to your newly created AWS repository:  

```
docker push
```

Close

Figure 64. ECR push

Docker を使って ECR に push するには **push** コマンドで **AWS** の **docker/** を使います。



2. `docker build -t XXXX .` コマンドで `Dockerfile` を使って Dockerfile をビルドします。

GB の ECR を使って ECR を使うには ECR を使うには ECR を使うには (Figure 65)

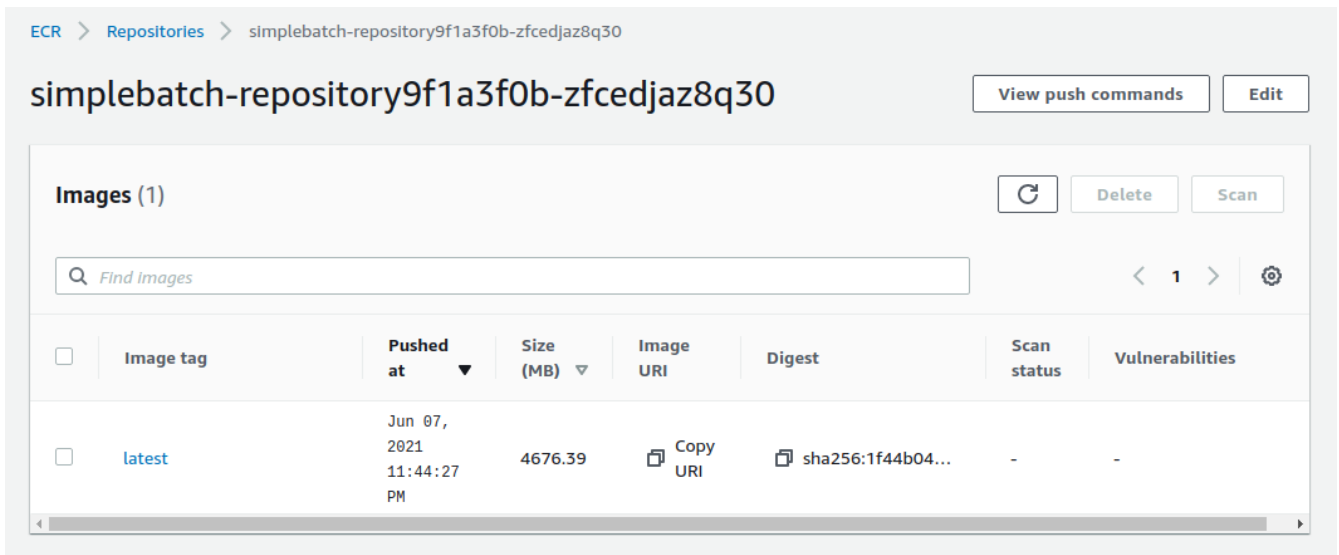


Figure 65. ECR の image の一覧

## 9.8. バッチジョブの実行

バッチジョブを実行する AWS Batch の設定を行います。

まず、`notebook/` ディレクトリに移動し、`run_single.ipynb` ファイルを実行します (`.ipynb` は Jupyter notebook ファイル形式)。

このファイルは `venv` 環境で Jupyter notebook を実行するための設定が記述されています。

```
# .env の設定
(.env) $ cd notebook
(.env) $ jupyter notebook
```

Jupyter notebook を実行すると、`run_single.ipynb` が実行されます。

このコードは、`[1]`, `[2]`, `[3]` のコメント行で、`submit_job()` メソッドを実行しています。

```
1 # [1]
2 import boto3
3 import argparse
4
5 # [2]
6 # AWS のプロファイル名を指定します
7
8 # [3]
9 def submit_job(lr:float, momentum:float, epochs:int, profile_name="default"):
10     if profile_name is None:
11         session = boto3.Session()
12     else:
13         session = boto3.Session(profile_name=profile_name)
14     client = session.client("batch")
```

```

15
16 title = "lr" + str(lr).replace(".", "") + "_m" + str(momentum).replace(".", "")
17 resp = client.submit_job(
18     jobName=title,
19     jobQueue="SimpleBatchjob-queue",
20     jobDefinition="SimpleBatchjob-definition",
21     containerOverrides={
22         "command": ["--lr", str(lr),
23                     "--momentum", str(momentum),
24                     "--epochs", str(epochs),
25                     "--uploadS3", "true"]
26     }
27 )
28 print("Job submitted!")
29 print("job name", resp["jobName"], "job ID", resp["jobId"])

```

`submit_job()` の詳細は [Section 9.4](#) の MNIST の Docker の部分を見てください。

```
$ docker run -it mymnist --lr 0.1 --momentum 0.5 --epochs 10
```

この `--lr 0.1 --momentum 0.5 --epochs 10` の部分は、

AWS Batch の `ContainerOverrides` の `command` の部分です。

```

1 containerOverrides={
2     "command": ["--lr", str(lr),
3                 "--momentum", str(momentum),
4                 "--epochs", str(epochs),
5                 "--uploadS3", "true"]
6 }

```

この [4] の部分は、`submit_job()` の引数 `lr = 0.01`, `momentum=0.1`, `epochs=100` の部分です。

```

# [4]
submit_job(0.01, 0.1, 100)

```



AWS の Jupyter Notebook の部分で、Notebook の [2] の部分 (AWS の部分) の `aws secret key` の部分 (Jupyter の部分) は、AWS の部分です。

この `submit_job()` の `profile_name` は、`~/.aws/credentials` の部分 (Section 15.3) の `profile_name` の部分です。

[4] 確認作業が完了したことを確認し、AWS Batch コンソールに移動し、AWS Batch ジョブのステータスを Figure 66 に示すように確認します。

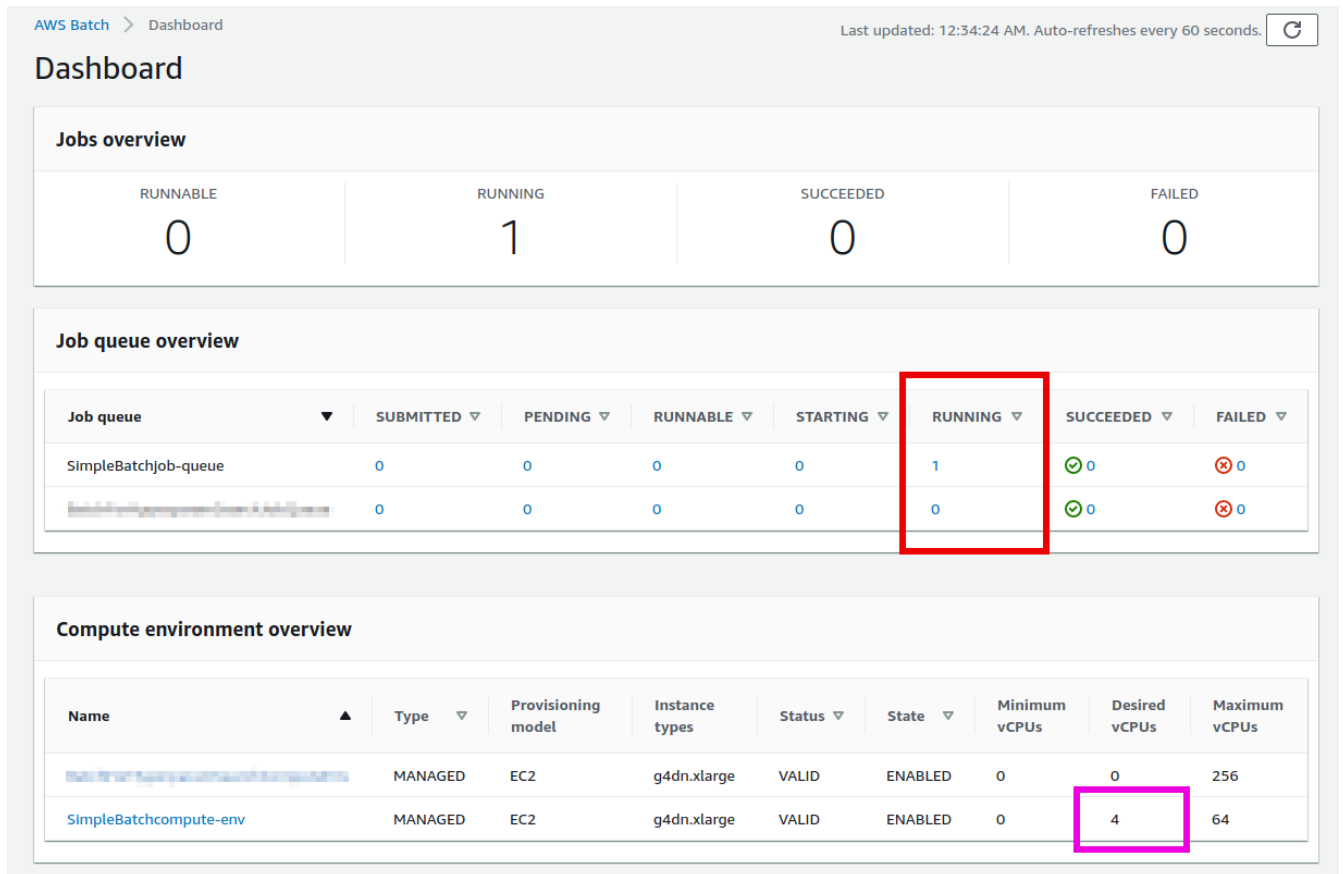


Figure 66. AWS Batch のジョブとコンピュート環境のステータス

Figure 66 のように、ジョブのステータスが SUBMITTED から RUNNING に移行し、コンピュート環境のステータスが MANAGED から ENABLED に移行していることを確認します。

ジョブのステータスが RUNNING になっていることを確認し、コンピュート環境の Desired vCPU が 4 になっていることを確認します (Figure 66 のように)。g4dn.xlarge インスタンスは 16 vCPU を提供し、4 インスタンスが稼働しているため、合計 64 vCPU が提供されます。

ジョブのステータスが RUNNING に移行し、コンピュート環境のステータスが MANAGED から ENABLED に移行していることを確認します。ジョブのステータスが SUCCEEDED (または FAILED) になっていることを確認します。MNIST データセットの 10 万枚の画像が SUCCEEDED になっていることを確認します。

ジョブのステータスが SUCCEEDED (または FAILED) になっていることを確認し、Loss と Accuracy の CSV ファイルが S3 にアップロードされていることを確認します。

S3 バケットに simplebatch-bucketXXXX (XXXX は任意の文字列) を作成し、metrics\_lr0.0100\_m0.1000.csv という CSV ファイルをアップロードします (Figure 67)。この CSV ファイルは、Loss = 0.01, Accuracy = 0.1 を示しています。

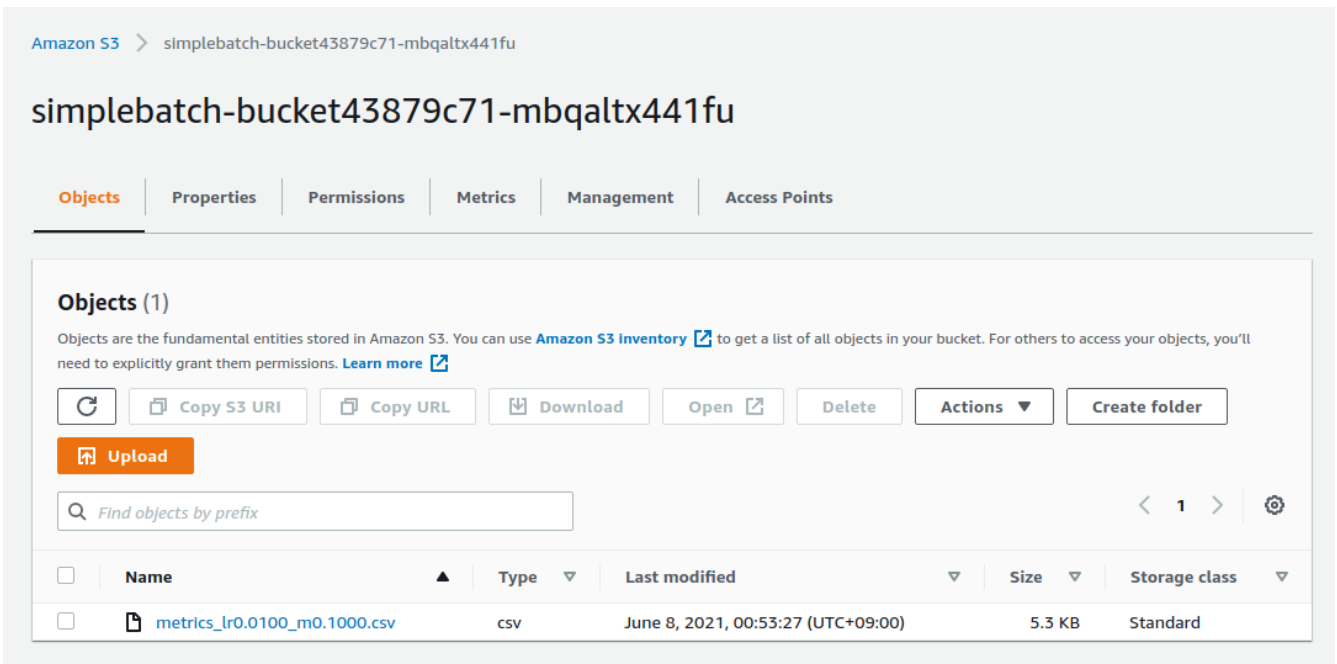


Figure 67. Amazon S3 console

The `run_single.ipynb` notebook [5] in [7] demonstrates how to read a CSV file from Amazon S3.

```

1 # [5]
2 import pandas as pd
3 import io
4 from matplotlib import pyplot as plt
5
6 # [6]
7 def read_table_from_s3(bucket_name, key, profile_name=None):
8     if profile_name is None:
9         session = boto3.Session()
10    else:
11        session = boto3.Session(profile_name=profile_name)
12    s3 = session.resource("s3")
13    bucket = s3.Bucket(bucket_name)
14
15    obj = bucket.Object(key).get().get("Body")
16    df = pd.read_csv(obj)
17
18    return df
19
20 # [7]
21 bucket_name = "simplebatch-bucket43879c71-mbqaltx441fu"
22 df = read_table_from_s3(
23     bucket_name,
24     "metrics_lr0.0100_m0.1000.csv"
25 )

```

[6] is a function that reads a CSV file from Amazon S3 using `pandas` and returns a `DataFrame`. The function takes three arguments: `bucket_name` (the name of the S3 bucket, e.g., `simplebatch-bucketXXXX`), `key` (the name of the object in the bucket), and `profile_name` (the name of the AWS profile to use).

Figure 68) AWS Batch MNIST

```
1 # [9]
2 fig, (ax1, ax2) = plt.subplots(1,2, figsize=(9,4))
3 x = [i for i in range(df.shape[0])]
4 ax1.plot(x, df["train_loss"], label="Train")
5 ax1.plot(x, df["val_loss"], label="Val")
6 ax2.plot(x, df["val_accuracy"])
7
8 ax1.set_xlabel("Epochs")
9 ax1.set_ylabel("Loss")
10 ax1.legend()
11
12 ax2.set_xlabel("Epochs")
13 ax2.set_ylabel("Accuracy")
14
15 print("Best loss:", df["val_loss"].min())
16 print("Best loss epoch:", df["val_loss"].argmin())
17 print("Best accuracy:", df["val_accuracy"].max())
18 print("Best accuracy epoch:", df["val_accuracy"].argmax())
```

```
Best loss: 0.2320499013662338
Best loss epoch: 0
Best accuracy: 99.15833333333332
Best accuracy epoch: 67
```

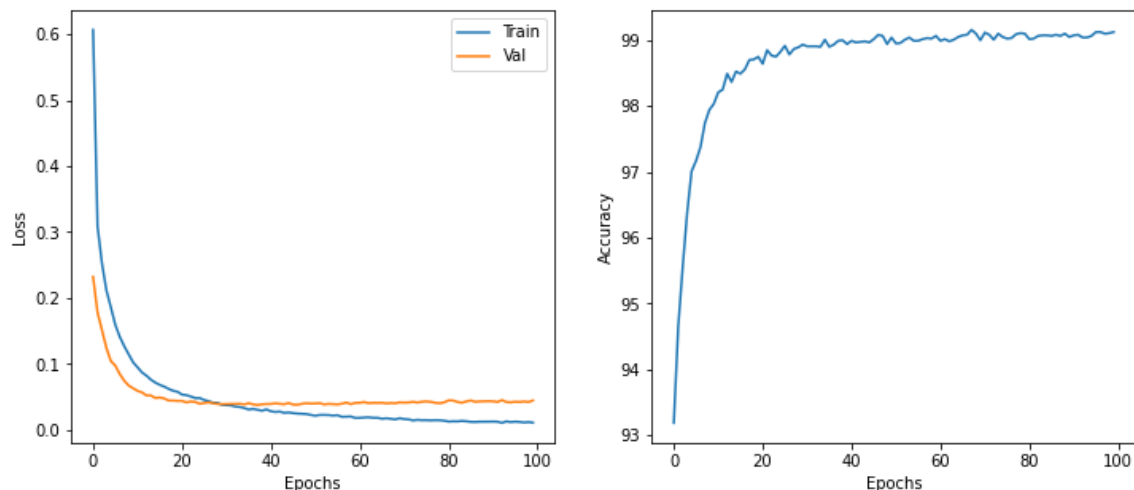


Figure 68. AWS Batch MNIST

## 9.9. Job

AWS Batch

run\_single.ipynb run\_sweep.ipynb

[1], [2], [3] run\_single.ipynb

```
1 # [1]
```

```
2 import boto3
3 import argparse
4
5 # [2]
6 # AWS ...
7
8 # [3]
9 def submit_job(lr:float, momentum:float, epochs:int, profile_name=None):
10     # ...
```

[4] for batch size 3x3=9

```
1 # [4]
2 for lr in [0.1, 0.01, 0.001]:
3     for m in [0.5, 0.1, 0.05]:
4         submit_job(lr, m, 100)
```

□ [4] □ Batch □ SUBMITTED > RUNNABLE > STARTING > RUNNING  
□ 9 □ RUNNING □ (Figure 69) □ Compute  
environment □ Desired vCPUs □ 4x9=36 □ (Figure 69)

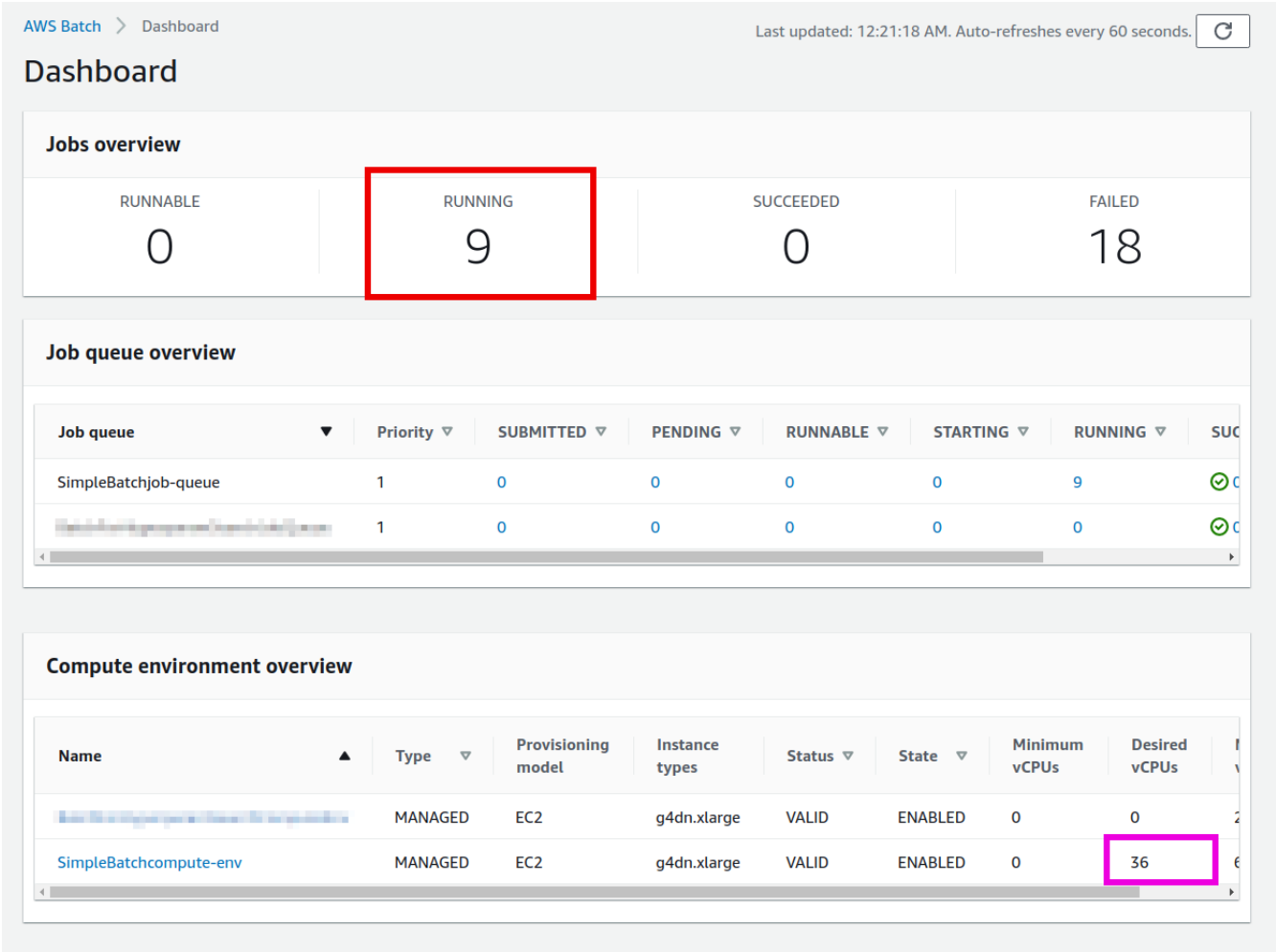
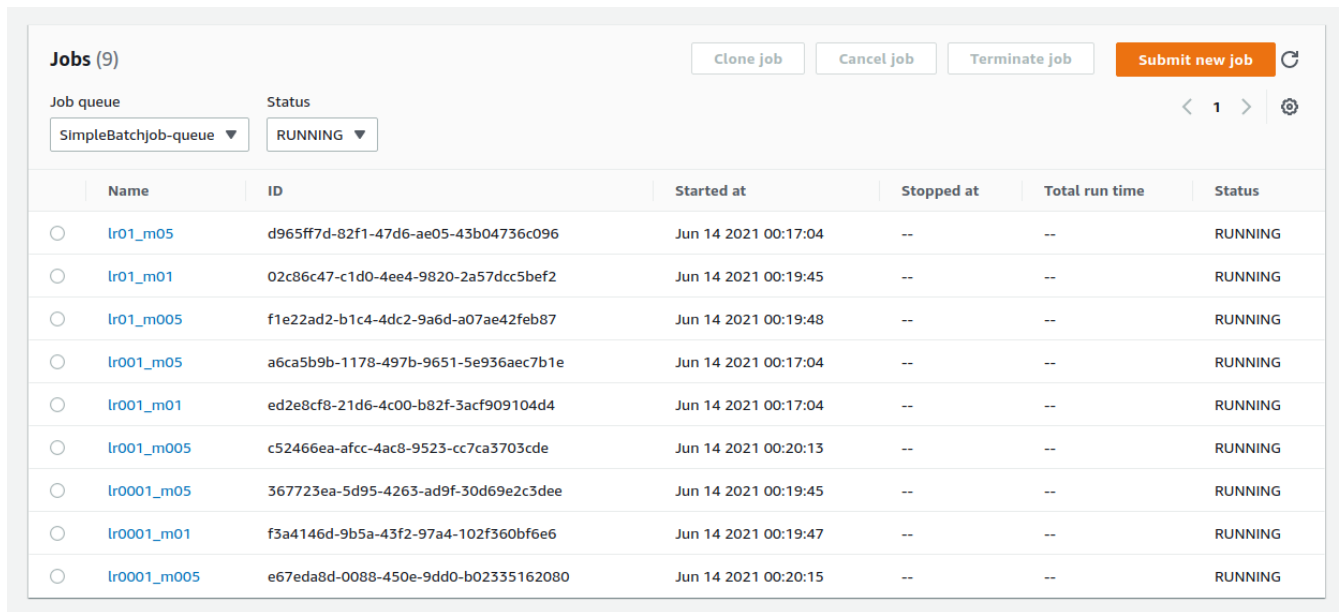


Figure 69. Batch

Batch Jobs (Figure 70)

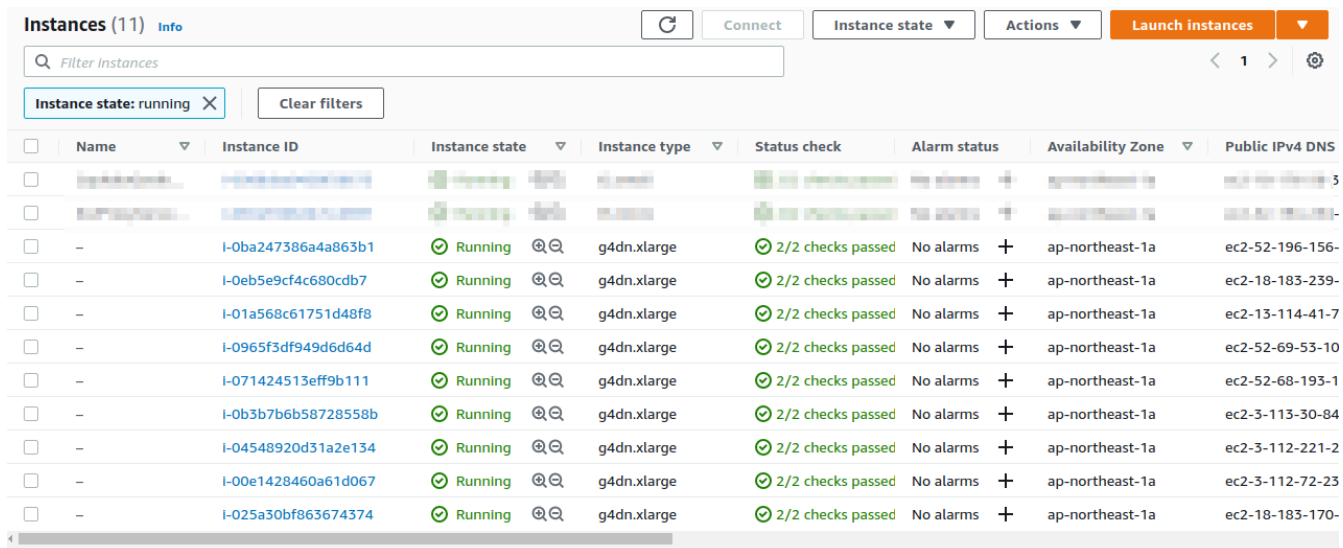
ジョブが9個 RUNNING 状態になっている



| Name        | ID                                   | Started at           | Stopped at | Total run time | Status  |
|-------------|--------------------------------------|----------------------|------------|----------------|---------|
| lr01_m05    | d965ff7d-82f1-47d6-ae05-43b04736c096 | Jun 14 2021 00:17:04 | --         | --             | RUNNING |
| lr01_m01    | 02c86c47-c1d0-4ee4-9820-2a57dcc5bef2 | Jun 14 2021 00:19:45 | --         | --             | RUNNING |
| lr01_m005   | f1e22ad2-b1c4-4dc2-9a6d-a07ae42feb87 | Jun 14 2021 00:19:48 | --         | --             | RUNNING |
| lr001_m05   | a6ca5b9b-1178-497b-9651-5e936aec7b1e | Jun 14 2021 00:17:04 | --         | --             | RUNNING |
| lr001_m01   | ed2e8cf8-21d6-4c00-b82f-3acf909104d4 | Jun 14 2021 00:17:04 | --         | --             | RUNNING |
| lr001_m005  | c52466ea-afcc-4ac8-9523-cc7ca3703cde | Jun 14 2021 00:20:13 | --         | --             | RUNNING |
| lr0001_m05  | 367723ea-5d95-4263-ad9f-30d69e2c3dee | Jun 14 2021 00:19:45 | --         | --             | RUNNING |
| lr0001_m01  | f3a4146d-9b5a-43f2-97a4-102f360bf6e6 | Jun 14 2021 00:19:47 | --         | --             | RUNNING |
| lr0001_m005 | e67eda8d-0088-450e-9dd0-b02335162080 | Jun 14 2021 00:20:15 | --         | --             | RUNNING |

Figure 70. AWS Batch のジョブ一覧

EC2 インスタンスのリストは **Instances** ページで表示されます。Figure 71 は g4dn.xlarge の 9 インスタンスが Batch ジョブのために起動されている様子です。



| Name | Instance ID         | Instance state | Instance type | Status check      | Alarm status | Availability Zone | Public IPv4 DNS |
|------|---------------------|----------------|---------------|-------------------|--------------|-------------------|-----------------|
| -    | I-0ba247386a4a863b1 | Running        | g4dn.xlarge   | 2/2 checks passed | No alarms    | ap-northeast-1a   | ec2-52-196-156- |
| -    | I-0eb5e9cf4c680cdb7 | Running        | g4dn.xlarge   | 2/2 checks passed | No alarms    | ap-northeast-1a   | ec2-18-183-239- |
| -    | I-01a568c61751d48f8 | Running        | g4dn.xlarge   | 2/2 checks passed | No alarms    | ap-northeast-1a   | ec2-13-114-41-7 |
| -    | I-0965f3df949d6d64d | Running        | g4dn.xlarge   | 2/2 checks passed | No alarms    | ap-northeast-1a   | ec2-52-69-53-10 |
| -    | I-071424513eff9b111 | Running        | g4dn.xlarge   | 2/2 checks passed | No alarms    | ap-northeast-1a   | ec2-52-68-193-1 |
| -    | I-0b3b7b6b58728558b | Running        | g4dn.xlarge   | 2/2 checks passed | No alarms    | ap-northeast-1a   | ec2-3-113-30-84 |
| -    | I-04548920d31a2e134 | Running        | g4dn.xlarge   | 2/2 checks passed | No alarms    | ap-northeast-1a   | ec2-3-112-221-2 |
| -    | I-00e1428460a61d067 | Running        | g4dn.xlarge   | 2/2 checks passed | No alarms    | ap-northeast-1a   | ec2-3-112-72-23 |
| -    | I-025a30bf863674374 | Running        | g4dn.xlarge   | 2/2 checks passed | No alarms    | ap-northeast-1a   | ec2-18-183-170- |

Figure 71. AWS Batch の EC2 インスタンス

ジョブのステータスは (約 10-15 分程度) 完了後 **SUCCEEDED** になります。9 インスタンスが **Desired vCPUs** 0 の状態で EC2 インスタンスとして起動されている g4dn のインスタンスが 9 個あります。

AWS Batch のジョブは EC2 インスタンス上で実行されます。AWS Batch のジョブは 10 インスタンス (9 インスタンス + 1 インスタンス) で実行され、90 インスタンス (10 インスタンス \* 9) が起動されます。

run\_sweep.ipynb を実行すると [5] が表示されます。

1 # [5]

```

2 import pandas as pd
3 import numpy as np
4 import io
5 from matplotlib import pyplot as plt
6
7 # [6]
8 def read_table_from_s3(bucket_name, key, profile_name=None):
9     if profile_name is None:
10         session = boto3.Session()
11     else:
12         session = boto3.Session(profile_name=profile_name)
13     s3 = session.resource("s3")
14     bucket = s3.Bucket(bucket_name)
15
16     obj = bucket.Object(key).get().get("Body")
17     df = pd.read_csv(obj)
18
19     return df
20
21 # [7]
22 grid = np.zeros((3,3))
23 for (i, lr) in enumerate([0.1, 0.01, 0.001]):
24     for (j, m) in enumerate([0.5, 0.1, 0.05]):
25         key = f"metrics_lr{lr:0.4f}_m{m:0.4f}.csv"
26         df = read_table_from_s3("simplebatch-bucket43879c71-mbqaltx441fu", key)
27         grid[i,j] = df["val_accuracy"].max()
28
29 # [8]
30 fig, ax = plt.subplots(figsize=(6,6))
31 ax.set_aspect('equal')
32
33 c = ax.pcolor(grid, edgecolors='w', linewidths=2)
34
35 for i in range(3):
36     for j in range(3):
37         text = ax.text(j+0.5, i+0.5, f"{grid[i, j]:0.1f}",
38                        ha="center", va="center", color="w")

```

Figure 72

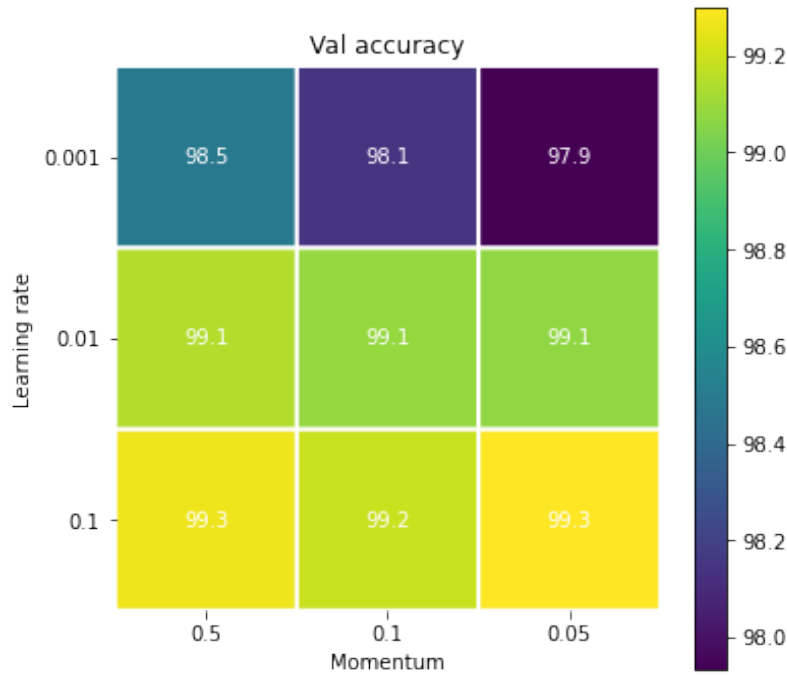


Figure 72. Validation accuracy

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.



Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

## 9.10. Validation

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.

Validation accuracy is 0.1. Validation accuracy is 0.1. Validation accuracy is 0.1.



Figure 73. ECR Docker image

다음은 AWS CLI를 사용하여 ECR 리포지토리의 이미지를 삭제하는 명령어입니다 (XXXX는 ECR 리포지토리 이름입니다).

```
$ aws ecr batch-delete-image --repository-name XXXX --image-ids imageTag=latest
```

이제 이미지를 삭제하고 다음 명령어를 실행합니다.

```
$ cdk destroy
```

[sec:batch\_development\_and\_debug] === 배치 개발 및 디버그

이 섹션에서는 AWS Batch를 사용하여 GPU를 활용한 병렬 컴퓨팅을 설정하고 실행하는 방법을 설명합니다.

이 섹션에서는 GPU를 지원하는 EC2 인스턴스를 사용하여 Jupyter Notebook을 실행하고, 이를 Docker로 패키징하여 AWS Batch를 사용하여 병렬 컴퓨팅을 수행하는 방법을 설명합니다. Figure 74는 이 프로세스의 개요를 보여줍니다. Chapter 6에서는 GPU를 지원하는 EC2 인스턴스를 설정하는 방법을 설명합니다. Jupyter Notebook을 실행하는 방법은 Chapter 9에서 설명합니다. AWS Batch를 사용하여 병렬 컴퓨팅을 수행하는 방법은 Chapter 8에서 설명합니다.

이 섹션에서는 MNIST 데이터셋을 사용하여 Jupyter Notebook을 실행하고, 이를 Docker로 패키징하여 AWS Batch를 사용하여 병렬 컴퓨팅을 수행하는 방법을 설명합니다.

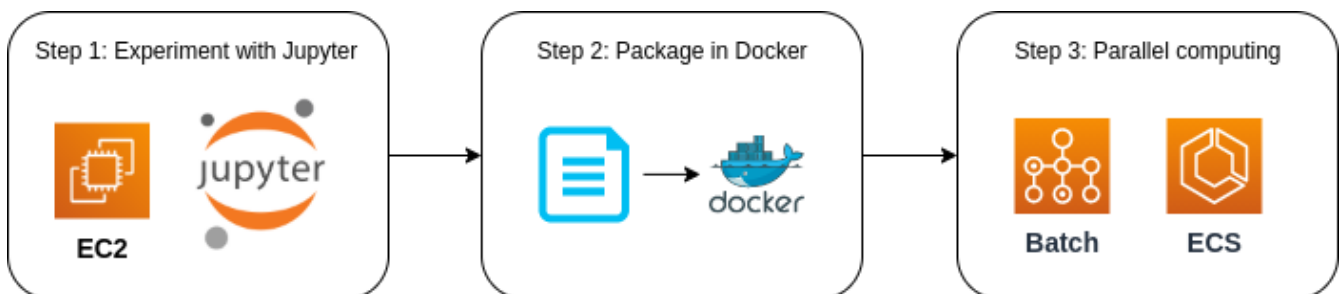


Figure 74. 배치 개발 및 디버그

## 9.11. 배치

이 섹션에서는 배치 개발 및 디버그를 위한 설정을 설명합니다.

この章では、GPU を利用した EC2 インスタンスで MNIST のディープ学習を実行する方法を説明します (Chapter 6)。

この章では、Docker を利用して ECS クラスターでディープ学習を実行する方法を説明します (Chapter 7)。  
また、Docker を利用して ECS クラスターでディープ学習を実行する方法を説明します (Chapter 8)。  
この章では、Docker を利用して ECS クラスターでディープ学習を実行する方法を説明します。

この章では、Chapter 9 で説明した AWS Batch を利用してディープ学習を実行する方法を説明します。  
この章では、AWS Batch を利用してディープ学習を実行する方法を説明します。  
この章では、AWS Batch を利用してディープ学習を実行する方法を説明します。

この章では、SNS を利用してディープ学習を実行する方法を説明します。  
この章では、SNS を利用してディープ学習を実行する方法を説明します。

# Chapter 10. Web

EC2, ECS, Fargate, Batch

## 10.1. — Twitter

Twitter, Facebook, YouTube

HTTP Twitter Figure 75

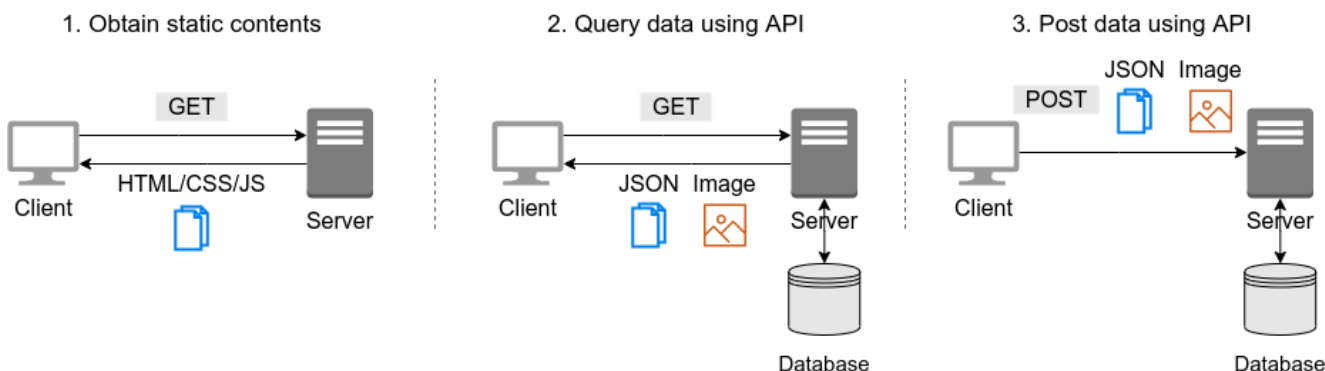


Figure 75. Web

HTTP (Hypertext Transfer Protocol) HTTPS (HTTPS (Hypertext Transfer Protocol Secure)) HTML (Hypertext Markup Language) CSS (Cascading Style Sheets) JavaScript (JS) Twitter API (Application Programming Interface) JSON (JavaScript Object Notation)

## 10.2. REST API

API (Application Programming Interface) は、アプリケーションとアプリケーションの間でデータをやり取りするための標準的な方法です。API は、アプリケーションが他のアプリケーションと通信するための方法を提供します。API は、アプリケーションが他のアプリケーションと通信するための方法を提供します。API は、アプリケーションが他のアプリケーションと通信するための方法を提供します。

REST (Representational State Transfer) は、API の一種です。REST API は、RESTful API と呼ばれます。

REST API は、Figure 76 のように Method と URI (Universal Resource Identifier) を使用します。



Figure 76. REST API

Method (動詞) は、API の操作を表します。HTTP の動詞 (verb) は、GET, POST, PUT, PATCH, DELETE などです (Table 6)。API の操作は、CRUD (create, read, update, and delete) があります。

Table 6. REST API Methods

| Method | Description             |
|--------|-------------------------|
| GET    | Read data               |
| POST   | Create new resource     |
| PUT    | Update resource         |
| PATCH  | Partial update resource |
| DELETE | Delete resource         |

URI は、API の操作の対象となるリソースを指定します。URI は、http または https のプロトコルで始まり、/ (スラッシュ) で区切られます。Figure 76 のように、https://api.twitter.com/1.1/status/home\_timeline (GET) は、Twitter API の 1.1 バージョンで、API の操作対象となるリソースを指定しています。



REST API は、Table 6 のように HTTP の動詞 (OPTIONS, TRACE など) を使用します。

URI は、API の操作の対象となるリソースを指定します。URI は、https://api.twitter.com/1.1/statuses/destroy/:id などの形式で指定されます。Twitter API の DELETE 操作は、API の操作対象となるリソースを指定します。



REST は、2000 年代前半に登場した API の設計方法です。REST API は、GraphQL と比較して、API の操作対象となるリソースを指定します。

## 10.3. Twitter API

[Twitter API](#)
[Twitter API](#)
[Twitter Developer Documentation](#)
[Table 7](#)

Table 7. Twitter API

| メソッド                        | 説明                |
|-----------------------------|-------------------|
| GET statuses/home_timeline  | 自分のタイムライン         |
| GET statuses/show/:id       | :id のツイートの詳細      |
| GET search                  | 検索結果              |
| POST statuses/update        | ツイートを更新           |
| POST media/upload           | メディアをアップロード       |
| POST statuses/destroy/:id   | :id のツイートを削除      |
| POST statuses/retweet/:id   | :id のツイートをリツイート   |
| POST statuses/unretweet/:id | :id のツイートをリツイート解除 |
| POST favorites/create       | ツイートをブックマーク       |
| POST favorites/destroy      | ツイートをブックマーク解除     |

API Twitter

```

GET statuses/home_timeline
id, text, user, coordinates,
entities
ID text user URL
coordinates entities
GET statuses/home_timeline
GET statuses/show/:id

```

GET search API  
 API GET statuses/home\_timeline  
 JSON

```

POST statuses/update
POST statuses/update
ID
POST media/upload
POST statuses/destroy/:id

```

```

POST statuses/retweet/:id
POST statuses/unretweet/:id
API
POST favorites/create
POST favorites/destroy

```

□□□□□□□□□□      Twitter      □□□□□□□□□□□□□□      □□□□□□□□□□□□□□□□□□□□□□      API



## Chapter 11. Serverless architecture

Serverless architecture (Serverless computing)  
AWS 2014 [Lambda](#)  
Google Microsoft

[illegible]

## 11.1. Serverful □□□□ (□□□)

Figure 77 shows the API endpoints for the AWS EC2 instance. The API endpoints are listed in the table below.

Load balancer

out scale-in

[illegible]

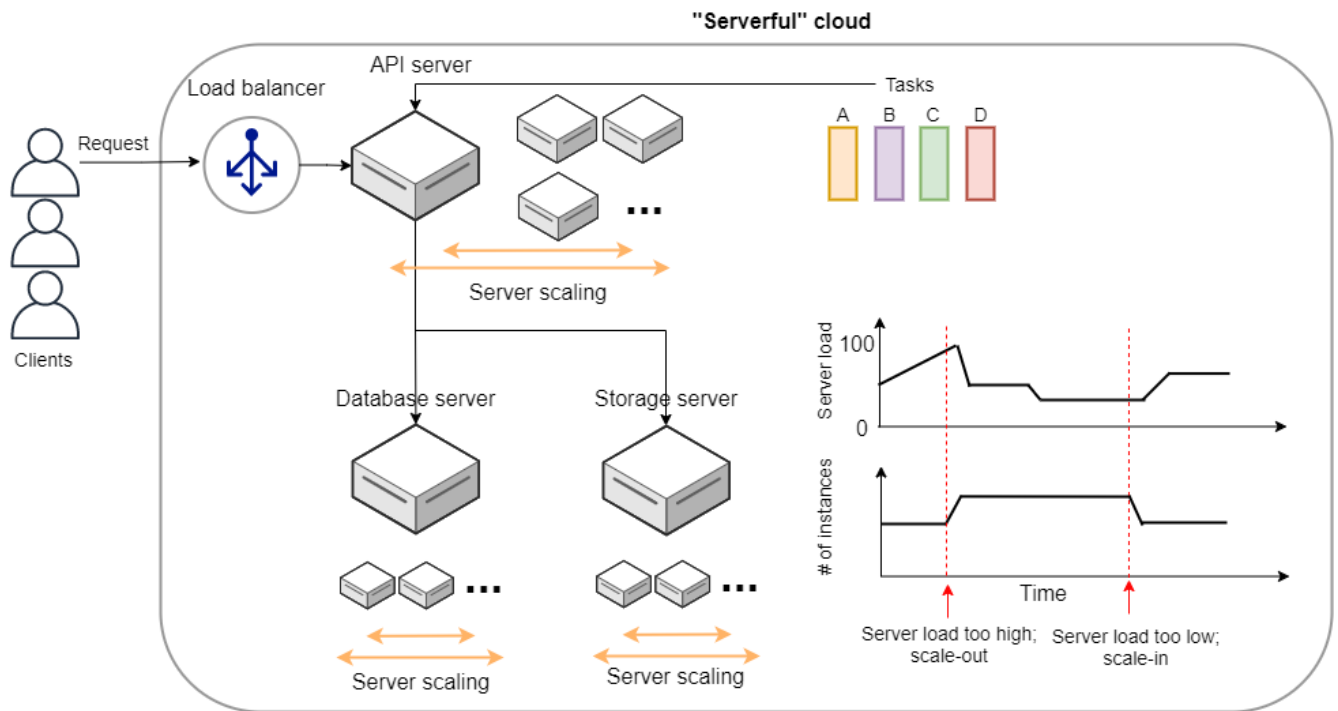


Figure 77. Serverful architecture

## 11.2. Serverless architecture

**Section 11.1** Serverless architecture is a cloud computing model where the provider manages the infrastructure and the scaling of the application. The user only needs to provide the code and the data, and the provider handles the rest.

Serverless architecture is a cloud computing model where the provider manages the infrastructure and the scaling of the application. The user only needs to provide the code and the data, and the provider handles the rest.

Compared to serverful architecture, serverless architecture has several advantages. First, it is more cost-effective because you only pay for the resources you use. Second, it is more scalable because the provider automatically scales the application for you. Third, it is more secure because the provider manages the infrastructure and the scaling of the application.

Serverless architecture is a cloud computing model where the provider manages the infrastructure and the scaling of the application. The user only needs to provide the code and the data, and the provider handles the rest.

Figure 78. Serverless architecture

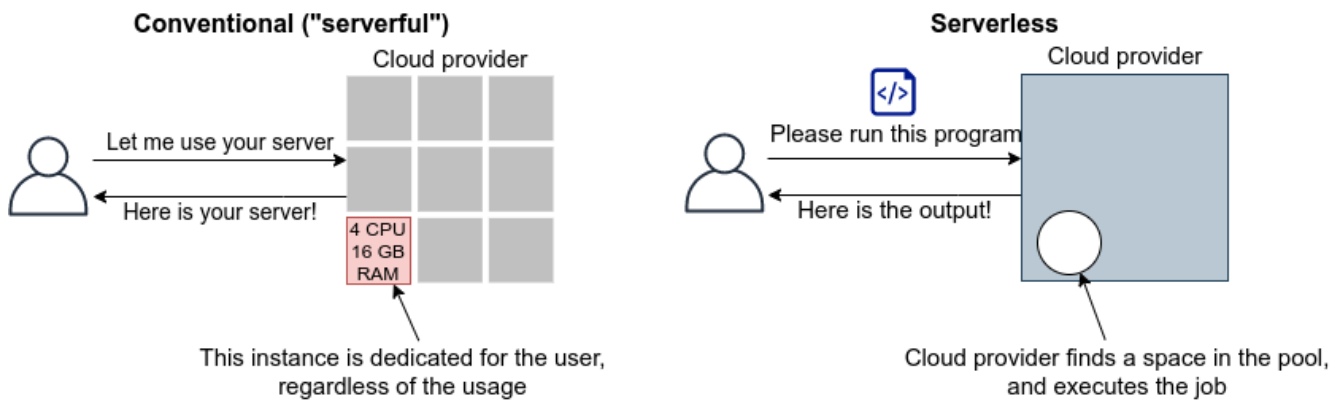


Figure 78. Serverless

Serverless architecture is a cloud service model in which the cloud provider dynamically manages the infrastructure, and the user only needs to provide the code and configuration. This model allows users to run code without provisioning or managing servers. The cloud provider finds a space in the pool and executes the job, returning the output to the user.

Serverless architecture is a cloud service model in which the cloud provider dynamically manages the infrastructure, and the user only needs to provide the code and configuration. This model allows users to run code without provisioning or managing servers.



Serverless architecture is a cloud service model in which the cloud provider dynamically manages the infrastructure, and the user only needs to provide the code and configuration. This model allows users to run code without provisioning or managing servers.

Serverless architecture is a cloud service model in which the cloud provider dynamically manages the infrastructure, and the user only needs to provide the code and configuration. This model allows users to run code without provisioning or managing servers.

### 11.3. Serverless architecture

Serverless architecture is a cloud service model in which the cloud provider dynamically manages the infrastructure, and the user only needs to provide the code and configuration. This model allows users to run code without provisioning or managing servers. AWS provides several serverless services, including **Lambda**, **S3**, **DynamoDB**, and **Step Functions**. These services are designed to be used together to build scalable, resilient applications. For more information, see [Chapter 12](#).



Figure 79. Lambda, S3, DynamoDB

#### 11.3.1. Lambda

AWS provides several serverless services, including **Lambda**. Lambda is a service that lets you run code without provisioning or managing servers. You can run code in response to events and automatically scale. For more information, see [Figure 80](#). Lambda supports several programming languages, including Python, Node.js, Ruby, and Java.



### 11.3.2. 클라우드 스토리지: S3

클라우드 스토리지는 클라우드 컴퓨팅의 핵심 서비스 중 하나입니다.

클라우드 스토리지 (클라우드 스토리지)는 운영 체제(OS)를 사용하여 데이터를 저장하고 CPU를 사용하여 데이터를 처리하는 클라우드 스토리지 서비스입니다. 클라우드 스토리지는 클라우드 스토리지 서비스 (ZFS)를 사용하여 데이터를 저장하고 CPU를 사용하여 데이터를 처리하는 클라우드 스토리지 서비스입니다. 클라우드 스토리지는 클라우드 스토리지 서비스 (Figure 81)를 사용하여 데이터를 저장하고 CPU를 사용하여 데이터를 처리하는 클라우드 스토리지 서비스입니다.

Simple Storage Service (S3)는 클라우드 스토리지 서비스 (Figure 81)를 사용하여 데이터를 저장하고 CPU를 사용하여 데이터를 처리하는 클라우드 스토리지 서비스입니다. S3는 클라우드 스토리지 서비스 (OS)를 사용하여 데이터를 저장하고 CPU를 사용하여 데이터를 처리하는 클라우드 스토리지 서비스입니다. S3는 클라우드 스토리지 서비스 (API)를 사용하여 데이터를 저장하고 CPU를 사용하여 데이터를 처리하는 클라우드 스토리지 서비스입니다. S3는 클라우드 스토리지 서비스 (API)를 사용하여 데이터를 저장하고 CPU를 사용하여 데이터를 처리하는 클라우드 스토리지 서비스입니다. S3는 클라우드 스토리지 서비스 (API)를 사용하여 데이터를 저장하고 CPU를 사용하여 데이터를 처리하는 클라우드 스토리지 서비스입니다.

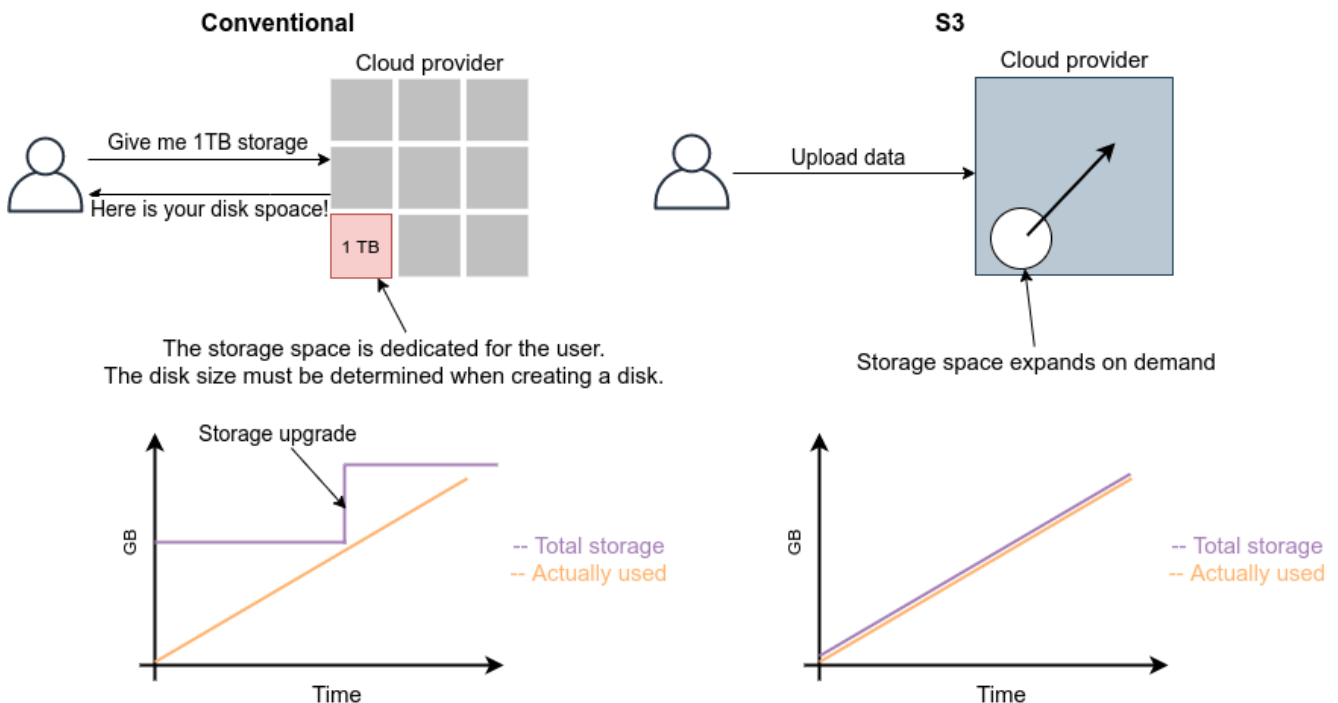


Figure 81. S3 클라우드 스토리지 서비스

S3 클라우드 스토리지 서비스의 가격은 Table 9에 표시되어 있습니다. (us-east-1 리전)의 가격은 "Amazon S3 pricing" 페이지에서 확인할 수 있습니다.

Table 9. S3 가격

| 구분                                                       | 단위                       |
|----------------------------------------------------------|--------------------------|
| Data storage (First 50TB)                                | \$0.023 per GB per month |
| PUT, COPY, POST, LIST requests (per 1,000 requests)      | \$0.005                  |
| GET, SELECT, and all other requests (per 1,000 requests) | \$0.0004                 |
| Data Transfer IN To Amazon S3 From Internet              | \$0                      |

| 項目                                           | 単価            |
|----------------------------------------------|---------------|
| Data Transfer OUT From Amazon S3 To Internet | \$0.09 per GB |

Amazon S3 のデータ転送料金は、\$0.025 per GB (データ入) / \$0.09 per GB (データ出) である。1000GB のデータ転送料金は、\$25 (データ入) / \$90 (データ出) となる。PUT, COPY, POST などの書き込み操作は、\$0.005 per GB (データ入) である。GET, SELECT などの読み込み操作は、\$0.0004 per GB (データ入) である。S3 からインターネットへデータを送信する際 (data-out) は、\$0.09 per GB の料金がかかる。S3 へのデータ転送 (data-in) は、AWS のリージョン (Region) によって異なる (Lambda や EC2 など) である。AWS のリージョンごとの料金表を参照してください。

### 11.3.3. 非関係データベース: DynamoDB

Amazon DynamoDB は、NoSQL データベースである。

Web アプリケーションやモバイルアプリケーションなど、高可用性とスケーラビリティを必要とするアプリケーションに、MySQL, PostgreSQL, MongoDB などの関係データベースよりも、Amazon DynamoDB が適している。DynamoDB は、単一の API で、高速な読み書きが可能で、スケーリングが容易である。

DynamoDB の料金体系は、CPU 使用量、メモリ使用量、ネットワーク使用量、ストレージ使用量などによって決まる。詳しくは、[Section 11.1](#) を参照してください。

DynamoDB は、AWS のマネージドサービスであり、API を介して簡単にアクセスできる。S3 と連携して、データのバックアップやアーカイブを行うこともできる。DynamoDB は、NoSQL データベースである。

DynamoDB の料金体系は、"On-demand Capacity" モードで、[Table 10](#) (us-east-1 リージョン) を参照してください。"Pricing for On-Demand Capacity" を参照してください。

Table 10. DynamoDB の料金

| 項目                  | 単価                                     |
|---------------------|----------------------------------------|
| Write request units | \$1.25 per million write request units |
| Read request units  | \$0.25 per million read request units  |
| Data storage        | \$0.25 per GB-month                    |

DynamoDB の料金体系は、write request unit と read request unit である。1 write request unit は、1KB のデータを 1 秒間で書き込むことである。1 read request unit は、1KB のデータを 1 秒間で読み込むことである。DynamoDB は、"Read/Write Capacity Mode" モードで、write request units が 100 単位で \$1.25、read request units が 100 単位で \$0.25 である。DynamoDB のストレージ料金は、\$0.25 per GB である。DynamoDB は、S3 と連携して、データのバックアップやアーカイブを行うこともできる。DynamoDB は、NoSQL データベースである。

### 11.3.4. 非関係データベース: Amazon ElastiCache

Amazon ElastiCache は、Redis や Memcached のマネージドサービスである。Lambda, S3, DynamoDB と連携して、アプリケーションの性能を向上させることができる。



# Chapter 12. Hands-on #5: 関数型アーキテクチャ

関数型アーキテクチャの構築に必要となる AWS サービスとして、AWS Lambda、Amazon S3、Amazon DynamoDB を利用する。また、AWS Lambda のデプロイとテストを行う。

## 12.1. Lambda の構築

この章で Lambda を構築するためのコードを GitHub の [handson/serverless/lambda](https://github.com/hands-on/serverless/lambda) に公開する。

Figure 82 は STEP 1 の AWS CDK を使って Python で Lambda を構築する様子と、STEP 2 の Invoke API を使って Lambda を呼び出す様子、そして Lambda の並行実行の様子を示している。

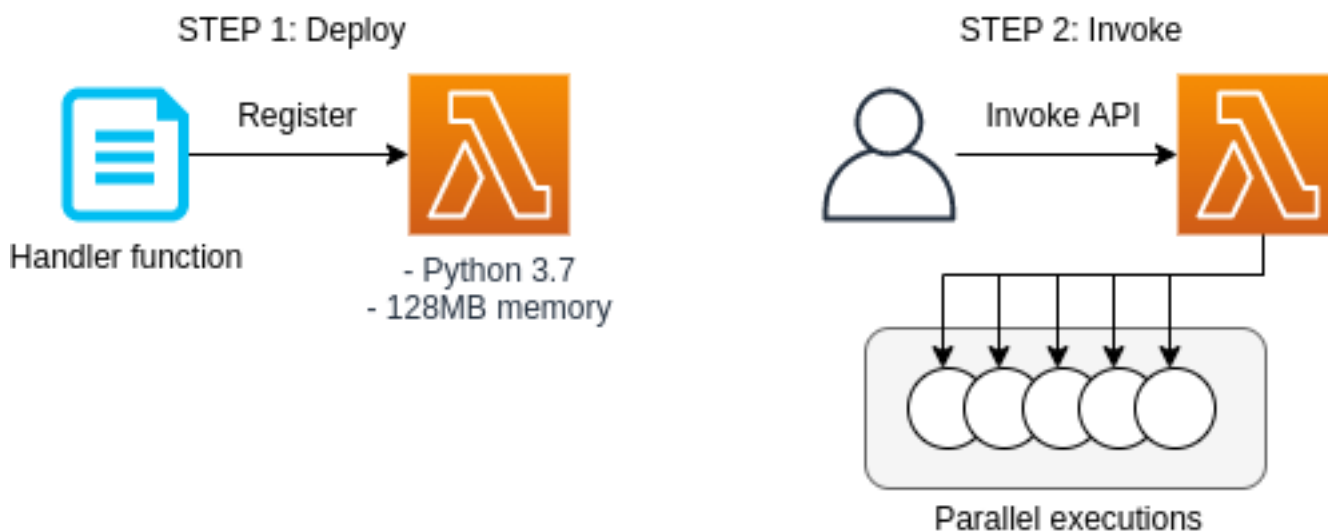


Figure 82. Lambda の構築と実行



この章では AWS Lambda を使って関数型アーキテクチャを構築する。

`app.py` は Lambda の handler 関数を実装するコードである。

```
1 ①
2 FUNC = """
3 import time
4 from random import choice, randint
5 def handler(event, context):
6     time.sleep(randint(2,5))
7     sushi = ["salmon", "tuna", "squid"]
8     message = "Welcome to Cloud Sushi. Your order is " + choice(sushi)
9     print(message)
10    return message
11 """
12
13 class SimpleLambda(Stack):
14
15     def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
16         super().__init__(scope, construct_id, **kwargs)
```

```

17
18     ②
19     handler = _lambda.Function(
20         self, 'LambdaHandler',
21         runtime=_lambda.Runtime.PYTHON_3_7,
22         code=_lambda.Code.from_inline(FUNC),
23         handler="index.handler",
24         memory_size=128,
25         timeout=cdk.Duration.seconds(10),
26         dead_letter_queue_enabled=True,
27     )

```

① 以下の Lambda 関数を実行すると、ランタイム環境に2-5秒間待機した後に["salmon", "tuna", "squid"]のリストを返す。このリストは、"Welcome to Cloud Sushi. Your order is XXXX" (XXXX はランタイム環境に待機した時間)に依存する。

② 以下の Lambda 関数 <1> は、ランタイム環境に待機した後に、ランタイム環境に待機した時間に応じて返す。

- `runtime=_lambda.Runtime.PYTHON_3_7`: Python3.7 ランタイム環境に待機した後に Python3.7 ランタイム環境に待機した時間に応じて返す。
- `code=_lambda.Code.from_inline(FUNC)`: 関数 `FUNC` のコードを inline に埋め込む。
- `handler="index.handler"`: 関数 `handler` の名前を指定する。
- `memory_size=128`: メモリサイズを 128MB に設定する。
- `timeout=cdk.Duration.seconds(10)`: タイムアウトを 10 秒に設定する。
- `dead_letter_queue_enabled=True`: デッドレターキューを有効にする。

以下のコードは、Lambda 関数をデプロイするためのコードです。

### 12.1.1. 実行

以下のコードを実行すると、Lambda 関数がデプロイされます。 (# はコメント行) デプロイは、1, 2 行のコードを実行することで完了します (Section 15.3)。

```

# 環境を設定
$ cd handson/serverless/lambda

# venv を作成
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# デプロイ
$ cdk deploy

```

以下のコードを実行すると、Figure 83 のように、Lambda 関数がデプロイされます。 `SimpleLambda.FunctionName = XXXX` のように、XXXX はランタイム環境に待機した時間です。

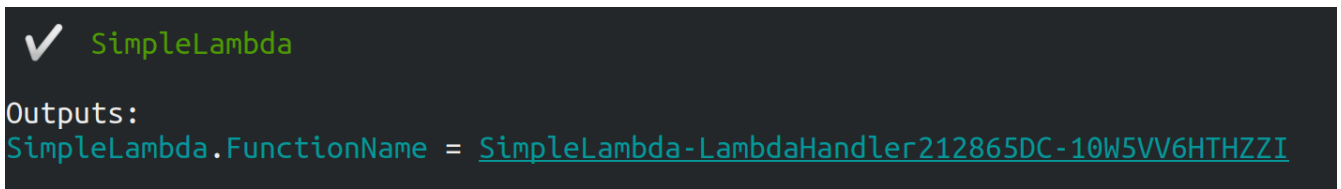


Figure 83. CDK の実行結果

AWS のコンソールで Lambda 関数を確認する。Figure 84 のように Lambda 関数が作成されている。

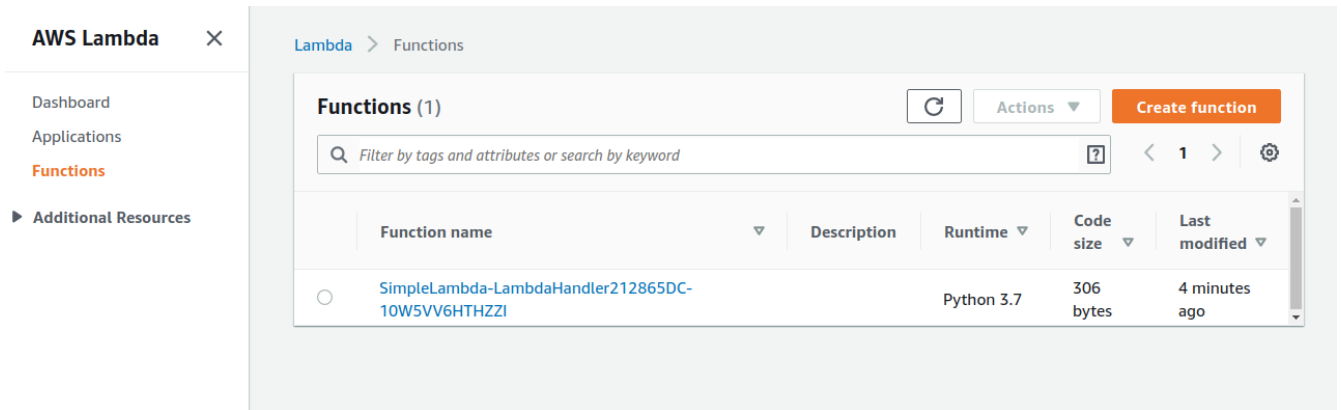


Figure 84. Lambda 関数の一覧

Figure 85 のように SimpleLambda 関数のコードを確認する。Python のコードが表示されている。

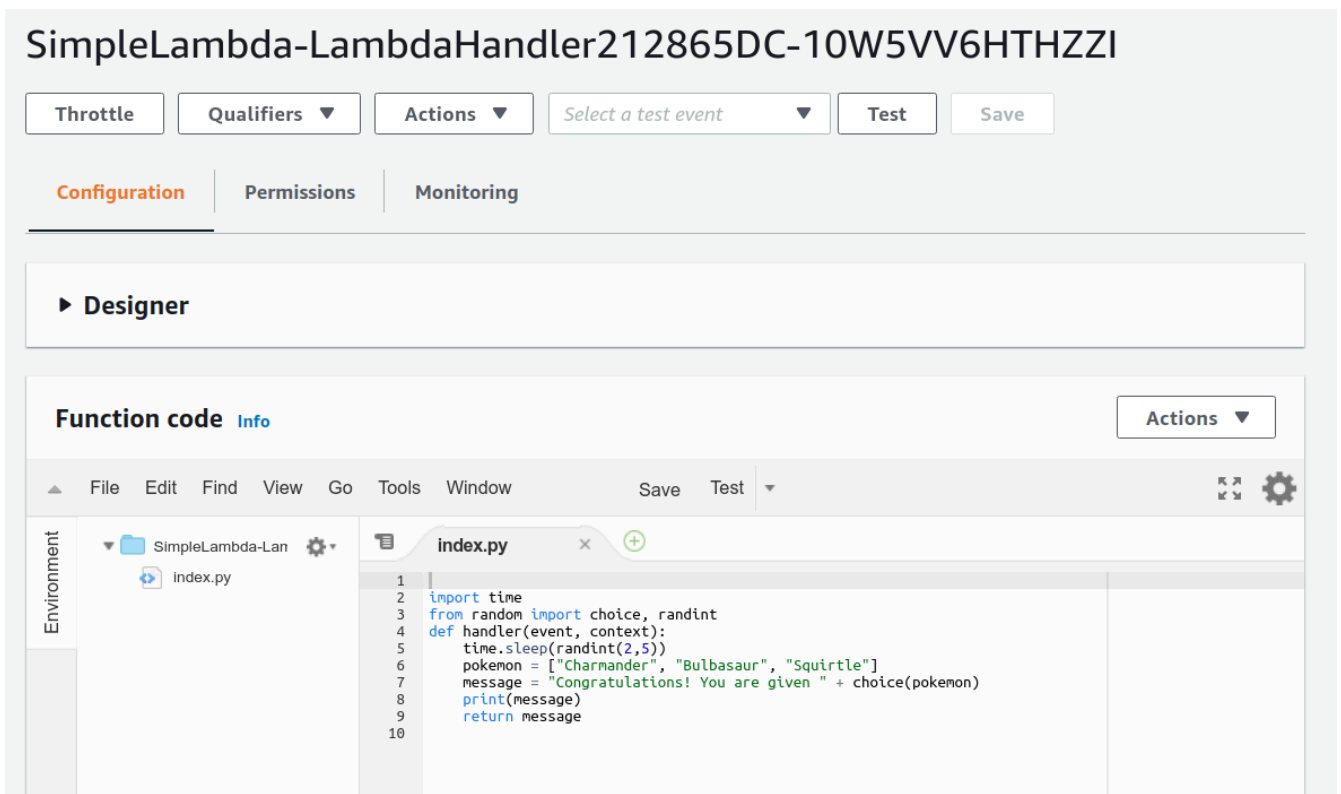


Figure 85. Lambda 関数のコード



Lambda 関数のコードを確認する。Figure 85 のように Python のコードが表示されている。CDK の実行結果を確認する。

## 12.1.2. Lambda 実行

ここでは Lambda を `invoke` して AWS の API を呼び出すコードを実行する。 [handson/serverless/lambda/invoke\\_one.py](#) を実行する。

Lambda を実行するコードは `SimpleLambda.FunctionName = XXXX` として `XXXX` を指定する。

```
$ python invoke_one.py XXXX
```

実行すると `"Welcome to Cloud Sushi. Your order is salmon"` と表示される。

ここでは Lambda を 100 回実行するコード [handson/serverless/lambda/invoke\\_many.py](#) を実行する。

実行するコードは `XXXX` として `100` を指定する。

```
$ python invoke_many.py XXXX 100
```

実行すると

```
.....  
.....  
Submitted 100 tasks to Lambda!
```

100 回実行した結果は [Figure 85](#) のように表示される。また [Figure 86](#) のように表示される。



### 12.1.3. 破壊デプロイ

デプロイしたアプリケーションを破壊するには、次のコマンドを実行します。

```
$ cdk destroy
```

## 12.2. DynamoDB アプリケーション

この章では、DynamoDB を使用して簡単なアプリケーションを構築します。このアプリケーションのソースコードは [GitHub](https://github.com/stackbuilders/handson/serverless/dynamodb) の [/handson/serverless/dynamodb](https://github.com/stackbuilders/handson/serverless/dynamodb) リポジトリにあります。

このアプリケーションは、Figure 87 の STEP 1 で AWS CDK を使用して DynamoDB テーブルをデプロイし、STEP 2 で API を使用してデータを操作します。



Figure 87. DynamoDB アプリケーション



このアプリケーションは [AWS DynamoDB](https://aws.amazon.com/dynamodb/) を使用して構築されています。

[handson/serverless/dynamodb/app.py](https://github.com/stackbuilders/handson/serverless/dynamodb/app.py) は、このアプリケーションのソースコードを示しています。

```
1 class SimpleDynamoDb(Stack):
2     def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
3         super().__init__(scope, construct_id, **kwargs)
4
5         table = ddb.Table(
6             self, "SimpleTable",
7             ①
8             partition_key=ddb.Attribute(
9                 name="item_id",
10                type=ddb.AttributeType.STRING
11            ),
12            ②
13            billing_mode=ddb.BillingMode.PAY_PER_REQUEST,
14            ③
15            removal_policy=cdk.RemovalPolicy.DESTROY
16        )
```

DynamoDB Partition key Partition key

- ① **partition\_key**: DynamoDB Partition key Partition key ( ) ID Partition key ( : Sort Key "Core Components of Amazon DynamoDB" ) Partition key **item\_id**
- ② **billing\_mode**: `ddb.BillingMode.PAY_PER_REQUEST` **On-demand Capacity Mode** DynamoDB **PROVISIONED**
- ③ **removal\_policy**: CloudFormation DynamoDB **DESTROY**

### 12.2.1.

(# ) (Section 15.3)

```
#  
$ cd handson/serverless/dynamodb  
  
# venv  
$ python3 -m venv .env  
$ source .env/bin/activate  
$ pip install -r requirements.txt  
  
#  
$ cdk deploy
```

Figure 88 `SimpleDynamoDb.TableName = XXXX` XXXX

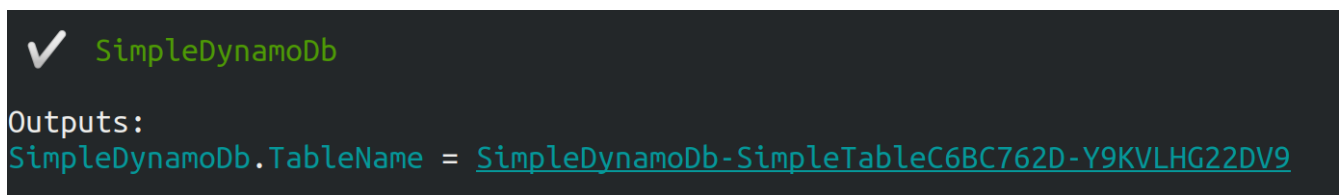


Figure 88. CDK

AWS DynamoDB "Tables" Figure 89

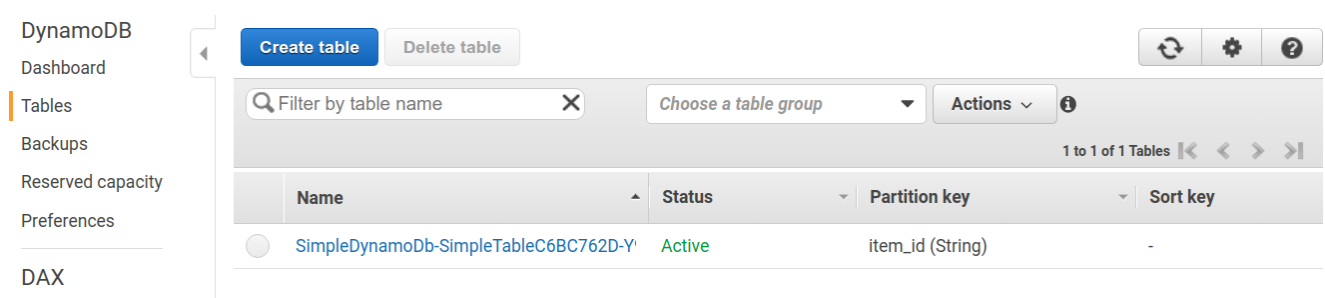


Figure 89. DynamoDB ( )

Figure 90 shows the AWS Management Console interface for a DynamoDB table named SimpleDynamoDb-SimpleTableC6BC762D-Y9KVLHG22DV9. The 'Items' tab is selected, showing a search bar and a 'Create item' button. The table's primary key is 'item\_id'.

Figure 90. DynamoDB console (Screenshot)

### 12.2.2. boto3

Section 12.2.1 describes how to use the boto3 library to interact with DynamoDB. Python's boto3 library is used to interact with DynamoDB.

The following code snippet, simple\_write.py, demonstrates how to write an item to a DynamoDB table using boto3.

```
1 import boto3
2 from uuid import uuid4
3 ddb = boto3.resource('dynamodb')
4
5 def write_item(table_name):
6     table = ddb.Table(table_name)
7     table.put_item(
8         Item={
9             'item_id': str(uuid4()),
10            'first_name': 'John',
11            'last_name': 'Doe',
12            'age': 25,
13        }
14    )
```

The boto3 library is used to interact with DynamoDB. The write\_item() function is used to write an item to a DynamoDB table. The table name is SimpleDynamoDb-XXXX. The put\_item() method is used to write an item to the DB. The item is a dictionary with the following keys: item\_id, first\_name, last\_name, age. The item\_id is a UUID4 string. The item\_id is the Partition key.

The simple\_write.py file is used to write an item to a DynamoDB table. The table name is SimpleDynamoDb-XXXX. The item is a dictionary with the following keys: item\_id, first\_name, last\_name, age.

```
$ python simple_write.py XXXX
```

Figure 90 AWS DynamoDB console screenshot showing the 'Simple DynamoDb-SimpleTableC6BC762D-1K9Z83Q7426FD' table. The 'Items' tab is selected, and a scan operation is shown. The scan results table displays columns: item\_id, age, first\_name, and last\_name. One item is visible with item\_id '2b4874e5-aa5e-4e18-b303-72b61839da6f', age 25, first\_name John, and last\_name Doe.

Figure 91. DynamoDB console screenshot showing the 'Simple DynamoDb-SimpleTableC6BC762D-1K9Z83Q7426FD' table. The 'Items' tab is selected, and a scan operation is shown. The scan results table displays columns: item\_id, age, first\_name, and last\_name. One item is visible with item\_id '2b4874e5-aa5e-4e18-b303-72b61839da6f', age 25, first\_name John, and last\_name Doe.

boto3 Python library is used to interact with AWS services. The `simple_read.py` script is used to read data from the table.

```
1 import boto3
2 ddb = boto3.resource('dynamodb')
3
4 def scan_table(table_name):
5     table = ddb.Table(table_name)
6     items = table.scan().get("Items")
7     print(items)
```

`table.scan().get("Items")` returns a list of items from the table.

The script `simple_read.py` is run with the command `python simple_read.py XXXX`, where `XXXX` is the table name.

```
$ python simple_read.py XXXX
```

The output of the script is a list of items from the table.

### 12.2.3. Batch Write

DynamoDB provides a batch write API to write multiple items in a single call.

The `batch_rw.py` script is used to write data to the table.

The script `batch_rw.py` is run with the command `python batch_rw.py XXXX write 1000`, where `XXXX` is the table name.

```
$ python batch_rw.py XXXX write 1000
```

The script writes 1000 items to the table.

python batch\_rw.py XXXX search\_under\_age 2

age

python batch\_rw.py XXXX search\_under\_age 2

age

```
$ python batch_rw.py XXXX search_under_age 2
```

python batch\_rw.py XXXX search\_under\_age 2

## 12.2.4. Python

DynamoDB Python

python batch\_rw.py XXXX search\_under\_age 2

```
$ cdk destroy
```

## 12.3. S3 Python

python S3 Python Python GitHub [hands-on/serverless/s3](https://github.com/hands-on/serverless/s3) Python

Figure 92 Python S3 Python STEP 1 Python AWS CDK Python S3 Python (Bucket) Python STEP 2 Python

### STEP 1: Deploy



S3

### STEP 2: Manipulate data



Read / Write



Figure 92. S3 Python



python S3 Python Python

app.py Python Python

```
1 class SimpleS3(Stack):
2     def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
3         super().__init__(scope, construct_id, **kwargs)
4
5         # S3 bucket to store data
6         bucket = s3.Bucket(
7             self, "bucket",
8             removal_policy=cdk.RemovalPolicy.DESTROY,
9             auto_delete_objects=True,
```

`s3.Bucket()` クラスを使用して、AWS S3 バケットを作成します。バケットの名前は `bucket_name` で指定します (i.e. AWS バケットの名前)。バケットの作成は、CloudFormation テンプレートを使用して行われます。



CloudFormation テンプレートを使用して、S3 バケットを作成します。バケットの作成は、`cdk` コマンドを使用して行われます。バケットの作成は、`destroy` コマンドを使用して行われます。バケットの作成は、`removal_policy=cdk.RemovalPolicy.DESTROY`、`auto_delete_objects=True` を指定します。

### 12.3.1. テンプレート

テンプレートは、AWS CloudFormation テンプレートを使用して作成されます (# テンプレート) (Section 15.3)

```
# テンプレートを作成
$ cd handson/serverless/s3

# venv を作成
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# テンプレート
$ cdk deploy
```

Figure 93 テンプレートを作成する。テンプレートは `SimpleS3.BucketName = XXXX` を指定します (テンプレートは、AWS CloudFormation テンプレートを使用して作成されます)



SimpleS3

Outputs:

SimpleS3.BucketName = [simples3-bucket43879c71-j96v4b58l3jd](#)

Figure 93. テンプレート

### 12.3.2. テンプレート

テンプレートは、AWS CloudFormation テンプレートを使用して作成されます

テンプレートは、`tmp.txt` に保存されます

```
$ echo "Hello world!" >> tmp.txt
```

テンプレートは、`simple_s3.py` に `boto3` を使用して S3 バケットを作成します。テンプレートは、`simple_s3.py` に `tmp.txt` を指定します。テンプレートは、`XXXX` を指定します。

```
$ python simple_s3.py XXXX upload tmp.txt
```

simple\_s3.py

```
1 def upload_file(bucket_name, filename, key=None):
2     bucket = s3.Bucket(bucket_name)
3
4     if key is None:
5         key = os.path.basename(filename)
6
7     bucket.upload_file(filename, key)
```

bucket = s3.Bucket(bucket\_name) Bucket() upload\_file()

S3 Key (Path) Key Object storage --key simple\_s3.py Key

```
$ python simple_s3.py XXXX upload tmp.txt --key a/b/tmp.txt
```

a/b/tmp.txt Key

AWS S3 S3 simples3-bucket (Figure 94)

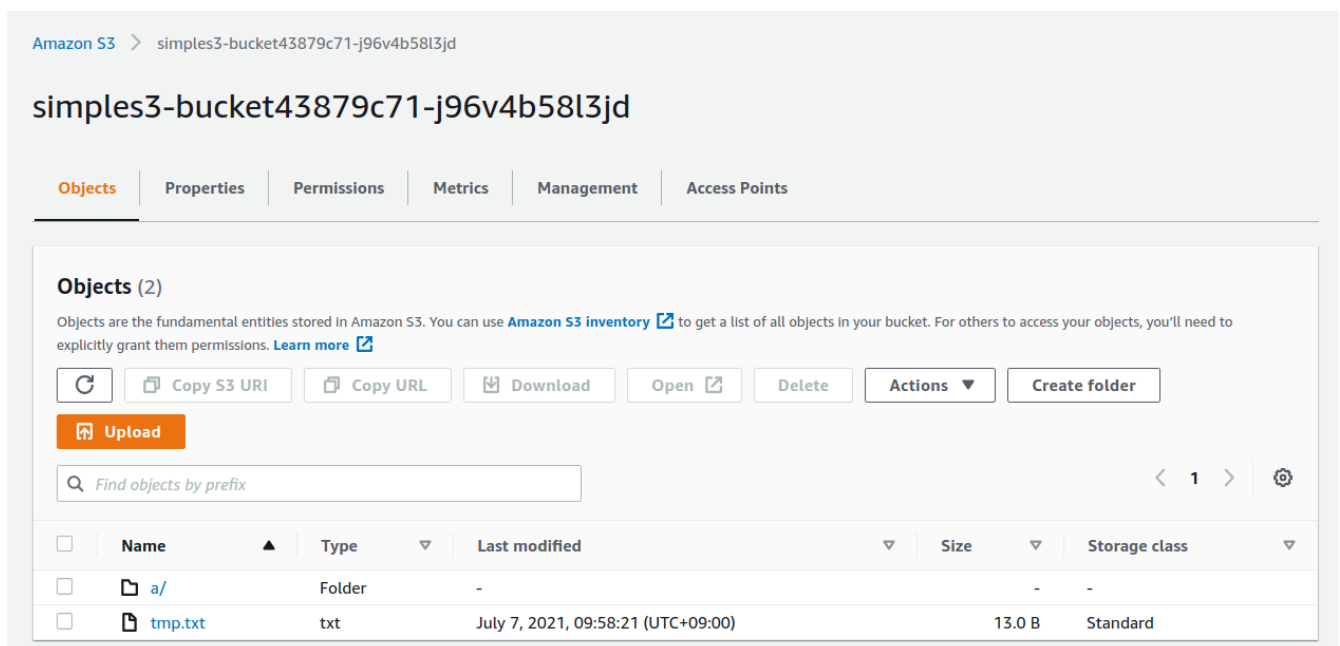


Figure 94. S3

tmp.txt a/b/tmp.txt S3 Key "/"



このコードを実行すると、S3 バケットからファイルがダウンロードされます。

このコードを実行すると、S3 バケットからファイルがダウンロードされます。

`simple_s3.py`

このコードを実行すると、S3 バケットからファイルがダウンロードされます。

XXXX

```
$ python simple_s3.py XXXX download tmp.txt
```

`simple_s3.py` のコードは以下の通りです。

```
1 def download_file(bucket_name, key, filename=None):
2     bucket = s3.Bucket(bucket_name)
3
4     if filename is None:
5         filename = os.path.basename(key)
6
7     bucket.download_file(key, filename)
```

S3 バケットからファイルがダウンロードされる。このコードを実行すると、S3 バケットからファイルがダウンロードされます。Key はファイルの名前です。

### 12.3.3. テスト

このコードを実行すると、S3 バケットからファイルがダウンロードされます。このコードを実行すると、S3 バケットからファイルがダウンロードされます。

```
$ cdk destroy
```

# Chapter 13. Hands-on #6: Bashoutter

この章では、SNSアプリケーション「Bashoutter」を開発します。

このアプリケーションは、SNSサービス（**Bashoutter**）として、AWS Lambda、DynamoDB、S3、Amazon SNS を利用して構築されます。Figure 95 は、このアプリケーションのスクリーンショットです。

Bashoutter

API Endpoint URL  
<https://ig5181x0bd.execute-api.ap-northeast-1.amazonaws.com/prod/>

Post your Haiku!

五  
クラウドで

七  
SNSを

五  
つくったよ

Your name  
詠み人知らず

POST

People's Haikus

REFRESH

dec2bd2bfcdd144208201d652c703d9922020/07/0701:25

柿食えば  
鐘が鳴るなり  
法隆寺

♡ 3 正岡子規

521b6a8ed4c34e6b98f4fc59cf5564962020/07/0701:26

閑けさや  
岩にしみ入る  
蟬の声

♡ 6 松尾芭蕉

9cc9f01987bd4575a1741f0cb24a94ab2020/07/0701:26

古池や  
蛙飛び込む  
水の音

♡ 1 松尾芭蕉

fe181b3ac913439aa1471c6a35cc33822020/07/0701:28

菜の花や  
月は東に  
日は西に

♡ 1 与謝蕪村

6cdad2516ccb4a77b297c88690d9f7c12020/07/0701:29

クラウドで  
SNSを  
つくったよ

♡ 20 詠み人知らず

Figure 95. Hands-on #6 SNS アプリケーション "Bashoutter"

## 13.1. 準備

この章のコードは、GitHub の [hands-on/bashoutter](https://github.com/hands-on/bashoutter) リポジトリにあります。

この章のコードは、AWS CLI を使用してインストールされます (Section 4.1)。



この章のコードは、AWS CLI を使用してインストールされます。



- 簡単な API を構築する **API Gateway** (API) を構築する API の URI を構築する **Lambda** を構築する
- 簡単な API を構築する (API) を構築する **Lambda** を構築する
- データベース (データベース) を構築する **DynamoDB** を構築する
- **Lambda** を構築する **DynamoDB** を構築する
- 静的コンテンツ (静的コンテンツ) を構築する **S3** を構築する **S3** を構築する **HTML/CSS/JS** を構築する

簡単な API を構築する (handson/bashoutter/app.py)

```

1 class Bashoutter(Stack):
2
3     def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
4         super().__init__(scope, construct_id, **kwargs)
5
6         ①
7         # dynamoDB table to store haiku
8         table = ddb.Table(
9             self, "Bashoutter-Table",
10             partition_key=ddb.Attribute(
11                 name="item_id",
12                 type=ddb.AttributeType.STRING
13             ),
14             billing_mode=ddb.BillingMode.PAY_PER_REQUEST,
15             removal_policy=cdk RemovalPolicy.DESTROY
16         )
17
18         ②
19         bucket = s3.Bucket(
20             self, "Bashoutter-Bucket",
21             website_index_document="index.html",
22             public_read_access=True,
23             removal_policy=cdk RemovalPolicy.DESTROY
24         )
25
26         common_params = {
27             "runtime": _lambda.Runtime.PYTHON_3_7,
28             "environment": {
29                 "TABLE_NAME": table.table_name
30             }
31         }
32
33         ③
34         # define Lambda functions
35         get_haiku_lambda = _lambda.Function(
36             self, "GetHaiku",
37             code=_lambda.Code.from_asset("api"),
38             handler="api.get_haiku",
39             memory_size=512,

```

```

40         **common_params,
41     )
42     post_haiku_lambda = _lambda.Function(
43         self, "PostHaiku",
44         code=_lambda.Code.from_asset("api"),
45         handler="api.post_haiku",
46         **common_params,
47     )
48     patch_haiku_lambda = _lambda.Function(
49         self, "PatchHaiku",
50         code=_lambda.Code.from_asset("api"),
51         handler="api.patch_haiku",
52         **common_params,
53     )
54     delete_haiku_lambda = _lambda.Function(
55         self, "DeleteHaiku",
56         code=_lambda.Code.from_asset("api"),
57         handler="api.delete_haiku",
58         **common_params,
59     )
60
61     ④
62     # grant permissions
63     table.grant_read_data(get_haiku_lambda)
64     table.grant_read_write_data(post_haiku_lambda)
65     table.grant_read_write_data(patch_haiku_lambda)
66     table.grant_read_write_data(delete_haiku_lambda)
67
68     ⑤
69     # define API Gateway
70     api = apigw.RestApi(
71         self, "BashoutterApi",
72         default_cors_preflight_options=apigw.CorsOptions(
73             allow_origins=apigw.Cors.ALL_ORIGINS,
74             allow_methods=apigw.Cors.ALL_METHODS,
75         )
76     )
77
78     haiku = api.root.add_resource("haiku")
79     haiku.add_method(
80         "GET",
81         apigw.LambdaIntegration(get_haiku_lambda)
82     )
83     haiku.add_method(
84         "POST",
85         apigw.LambdaIntegration(post_haiku_lambda)
86     )
87
88     haiku_item_id = haiku.add_resource("{item_id}")
89     haiku_item_id.add_method(
90         "PATCH",

```

```

91         apigw.LambdaIntegration(patch_haiku_lambda)
92     )
93     haiku_item_id.add_method(
94         "DELETE",
95         apigw.LambdaIntegration(delete_haiku_lambda)
96     )

```

- ① 簡単に DynamoDB を構築する
- ② 簡単に S3 を構築する
- ③ 簡単に API Gateway を構築する Python3.7 を構築する [handson/bashoutter/api/api.py](https://handson.bashoutter.com/api/api.py)
- ④ <3> 簡単に Lambda を構築する
- ⑤ 簡単に API Gateway を構築する API Gateway を構築する Lambda を構築する

簡単に構築する

### 13.2.3. Public access mode の S3 の構築

S3 を構築する

```

1 bucket = s3.Bucket(
2     self, "Bashoutter-Bucket",
3     website_index_document="index.html",
4     public_read_access=True,
5     removal_policy=cdk.RemovalPolicy.DESTROY
6 )

```

簡単に `public_read_access=True` を構築する S3 を構築する S3 の **Public access mode** を構築する Public access mode を構築する (i.e. 簡単に) を構築する `public access mode` を構築する <http://XXXX.s3-website-ap-northeast-1.amazonaws.com/> URL を構築する URL を構築する `index.html` を構築する (`website_index_document="index.html"` を構築する)

簡単に HTML/CSS/JS を構築する



簡単に `public access mode` の S3 を構築する [CloudFront](https://aws.amazon.com/cloudfront/) を構築する CloudFront を構築する **Content Delivery Network (CDN)** を構築する HTTPS を構築する CloudFront を構築する [What is Amazon CloudFront?](https://aws.amazon.com/cloudfront/) を構築する

簡単に CloudFront を構築する

- <https://github.com/aws-samples/aws-cdk-examples/tree/master/typescript/static-site>



簡単に S3 を構築する AWS を構築する URL を構築する `example.com`

### 13.2.4. API □□□□□□

API 関数 (handler) GET /haiku API Lambda

```
1 get_haiku_lambda = _lambda.Function(  
2     self, "GetHaiku",  
3     code=_lambda.Code.from_asset("api"),  
4     handler="api.get_haiku",  
5     memory_size=512,  
6     **common_params  
7 )
```

```
memory_size=512 512MB
code=_lambda.Code.from_asset("api") (api/) handler="api.get_haiku"
api.py get_haiku()
```

get\_haiku() (handson/bashoutter/api/api.py)

```

1 ddb = boto3.resource("dynamodb")
2 table = ddb.Table(os.environ["TABLE_NAME"])
3
4 def get_haiku(event, context):
5     """
6     handler for GET /haiku
7     """
8     try:
9         response = table.scan()
10
11         status_code = 200
12         resp = response.get("Items")
13     except Exception as e:
14         status_code = 500
15         resp = {"description": f"Internal server error. {str(e)}"}
16     return {
17         "statusCode": status_code,
18         "headers": HEADERS,
19         "body": json.dumps(resp, cls=DecimalEncoder)
20     }

```

```
response = table.scan() # DynamoDB returns up to 1MB of data (1000 items)
# If you want more, you need to make multiple calls
```

API API



```
GET /haiku response = table.scan()
```

DynamoDB □ scan() □□□□□□□□ 1MB □□□□□□□□□□□□ □□□□□□□□□□□□ 1MB  
□□□□□□□□□□□□□□□□ scan() □□□□□□□□□□□□ □□□ boto3 □□□□□□□□□□ □□□

### 13.2.5. AWS 認証 (IAM)

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

```
1 table.grant_read_data(get_haiku_lambda)
2 table.grant_read_write_data(post_haiku_lambda)
3 table.grant_read_write_data(patch_haiku_lambda)
4 table.grant_read_write_data(delete_haiku_lambda)
```

○○○○○○○○○○○○○○○○○○○○○○○○○○○○ AWS ○○ IAM (Identity and Access Management) ○○○○○○○○○○○ IAM  
○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○○

LambdaIAM Lambda DynamoDB

CDK 的 `dynamodb.Table` 的 `grant_read_write_data()` 方法可以授予 Lambda 函数对 DynamoDB 表的读写权限。IAM 角色是 CDK 的 `s3.Bucket` 的 `grant_read_write()` 方法授予的。Chapter 9 的 AWS Batch 部分也有类似的操作。



このIAM ポリシーは、Amazon S3 のバケットに格納されているオブジェクトの読み取り権限を付与しています。

このポリシーは、`grant_read_data()` オペレーションを使用して、`read` アクションを実行できるようにしています。

### 13.2.6. API Gateway

[illegible]



```

20 )
21
22 ④
23 haiku_item_id = haiku.add_resource("{item_id}")
24 ⑤
25 haiku_item_id.add_method(
26     "PATCH",
27     apigw.LambdaIntegration(patch_haiku_lambda)
28 )
29 haiku_item_id.add_method(
30     "DELETE",
31     apigw.LambdaIntegration(delete_haiku_lambda)
32 )

```

- ① `api = apigw.RestApi()` API Gateway API Gateway
- ② `api.root.add_resource()` API Gateway `/haiku` API Gateway
- ③ `add_method()` API Gateway `GET, POST` API Gateway `/haiku` API Gateway
- ④ `haiku.add_resource("{item_id}")` API Gateway `/haiku/{item_id}` API Gateway
- ⑤ `add_method()` API Gateway `PATCH, DELETE` API Gateway `/haiku/{item_id}` API Gateway

API Gateway API Gateway API Gateway Lambda API Gateway



API Gateway API Gateway URL API Gateway `api.example.com` API Gateway AWS API Gateway URL API Gateway DNS API Gateway



API Gateway API Gateway `default_cors_preflight_options=` API Gateway Cross Origin Resource Sharing (CORS) API Gateway Web API Gateway

### 13.3. API Gateway

API Gateway API Gateway API Gateway (# API Gateway) API Gateway (Section 15.3)

```

# API Gateway
$ cd intro-aws/handson/bashoutter

# venv API Gateway
$ python3 -m venv .env
$ source .env/bin/activate
$ pip install -r requirements.txt

# API Gateway
$ cdk deploy

```

API Gateway API Gateway Figure 98 API Gateway Bashoutter.BashoutterApiEndpoint =

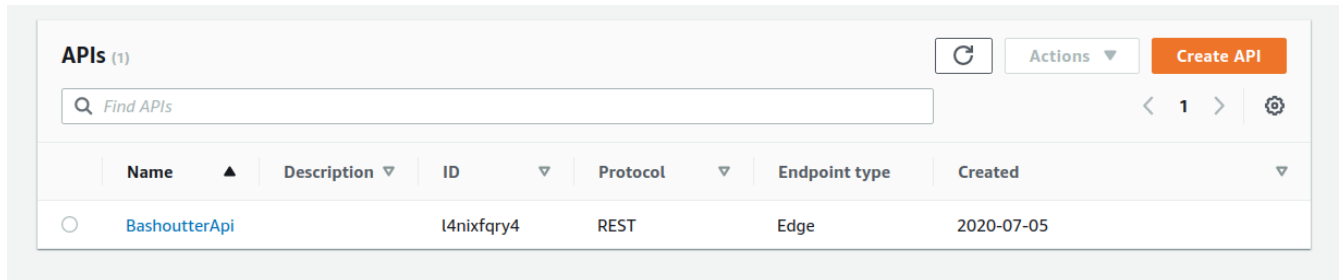
XXXX, Bashoutter.BucketUrl = YYYY

```
✓ Bashoutter

Outputs:
Bashoutter.BashoutterApiEndpointE9A7B94F = https://ig5181x0bd.execute-api.ap-northeast-1.amazonaws.com/pr
od/
Bashoutter.BucketUrl = bashoutter-bashoutterbucket6f28d9f3-zmi05o477p6r.s3-website-ap-northeast-1.amazona
ws.com
```

Figure 98. CDK

AWS API Gateway



| Name          | Description | ID         | Protocol | Endpoint type | Created    |
|---------------|-------------|------------|----------|---------------|------------|
| BashoutterApi |             | l4nixfqry4 | REST     | Edge          | 2020-07-05 |

Figure 99. API Gateway (1)

"BashoutterApi" API GET /haiku, POST /haiku

API Gateway "Method Request" Figure 100 Lambda

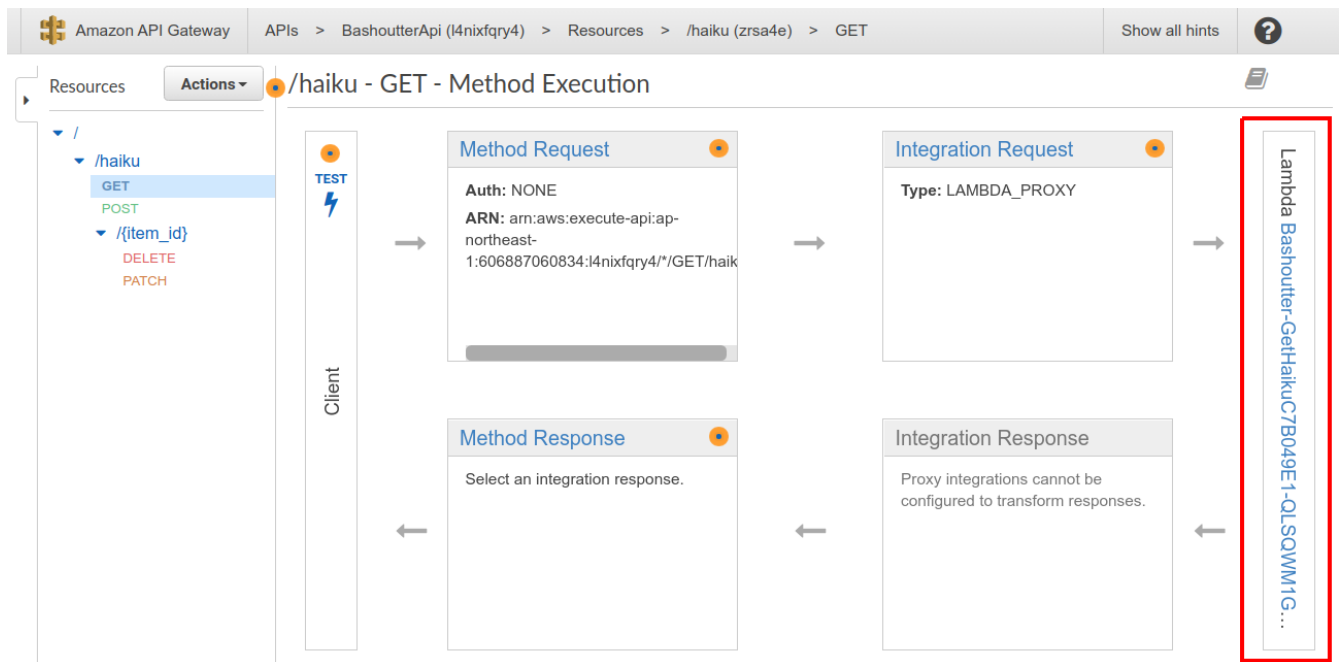


Figure 100. API Gateway (2)

S3 bashouter- (Figure 101)

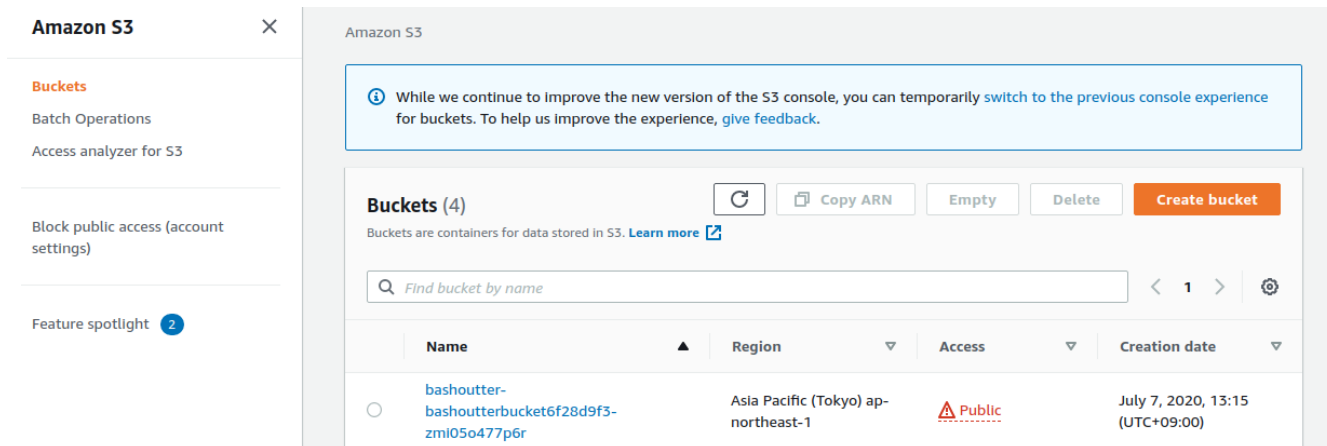


Figure 101. S3 コンソール

index.html ファイルを css/, js/ フォルダにアップロードする (Figure 102)。

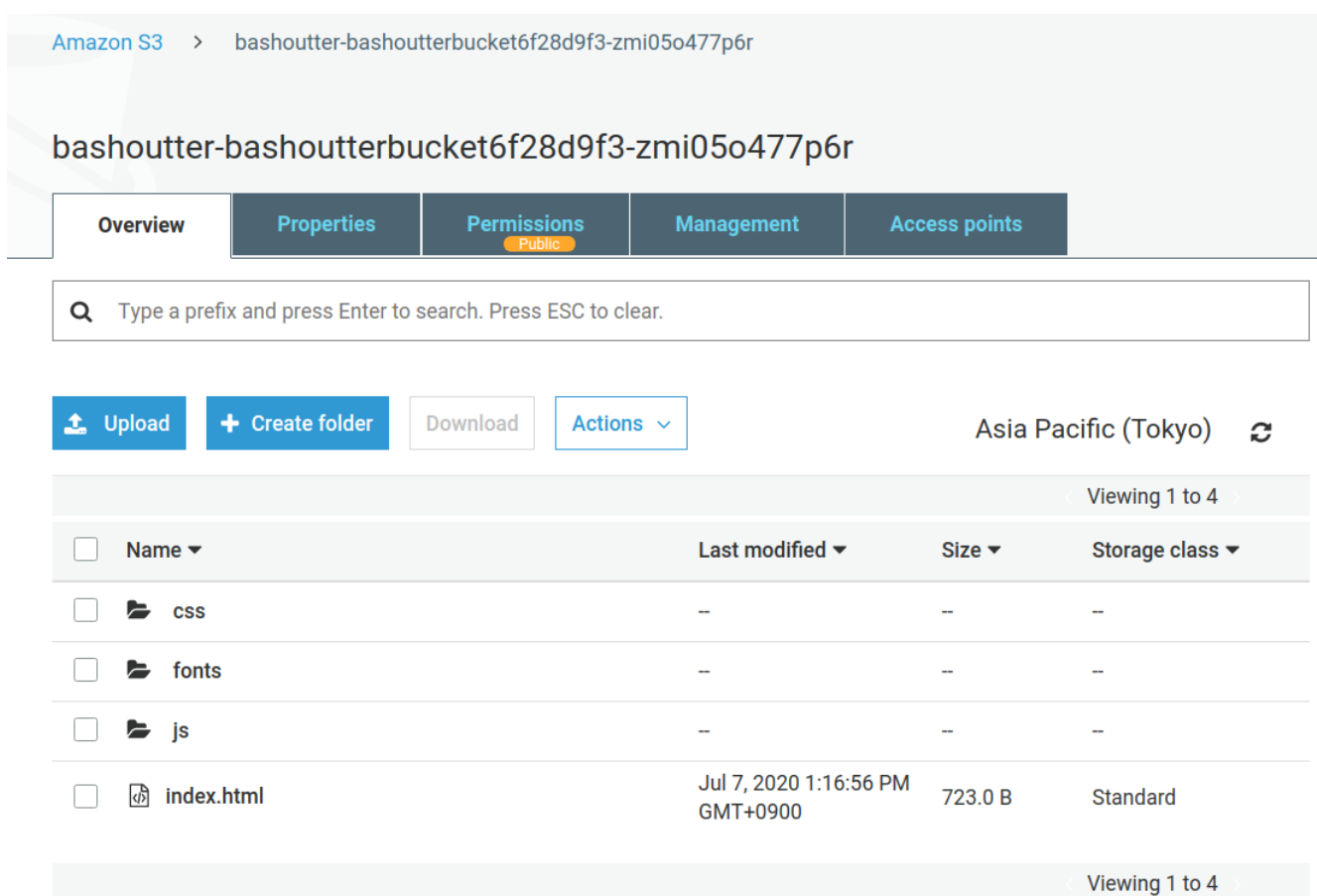


Figure 102. S3 コンソール

## 13.4. API を利用する方法

この章では、API を利用して、Amazon S3 にファイルをアップロードする方法について説明します。

この章では、HTTP API を利用して、Amazon S3 にファイルをアップロードする方法について説明します。HTTPie を利用して、Python を利用して、http を利用して、Amazon S3 にファイルをアップロードする方法について説明します。

API URL (Bashoutter.BashoutterApiEndpoint = XXXX XXXX  
)

```
$ export ENDPOINT_URL=XXXX
```

GET /haiku API

```
$ http GET "${ENDPOINT_URL}/haiku"
```

([])

POST /haiku

```
$ http POST "${ENDPOINT_URL}/haiku" \  
username=" " \  
first=" " \  
second=" " \  
third=" "
```

```
HTTP/1.1 201 Created  
Connection: keep-alive  
Content-Length: 49  
Content-Type: application/json  
....  
{  
  "description": "Successfully added a new haiku"  
}
```

GET

```
$ http GET "${ENDPOINT_URL}/haiku"
```

```
HTTP/1.1 200 OK  
Connection: keep-alive  
Content-Length: 258  
Content-Type: application/json  
...  
[  
  {  
    "created_at": "2020-07-06T02:46:04+00:00",  
    "first": "  
    "item_id": "7e91c5e4d7ad47909e0ac14c8bbab05b",  
    "likes": 0.0,  
    "second": "  
  },  
]
```

```
[{"first": "John",  
  "last": "Doe",  
  "third": "000",  
  "username": "0000"  
}]
```

☐ ☐ ☐ ☐ ☐

```
0000 PATCH /haiku/{item_id} 000000000000000000000000 item_id 00000000 XXXX
000000000000
```

```
$ http PATCH "${ENDPOINT_URL}/haiku/XXXX"
```

```
{"description": "OK"} GET /likes 1
```

```
$ http GET "${ENDPOINT_URL}/haiku"
...
[
  {
    ...
    "likes": 1.0,
    ...
  }
]
```

```
DELETE FROM XXXX WHERE item_id = 1
```

```
$ http DELETE "${ENDPOINT_URL}/haiku/XXXX"
```

GET 方法 ([]) 使用

API

### 13.5. API

API  
SNS

[handson/bashoutter/client.py](#) のAPIのテストコードを動かすには、POST /haiku のAPIのテストコードを実行する。

□□□□□□ API □300□□□□□□□ □□□□□□□□□□

```
$ python client.py $ENDPOINT_URL post_many 300
```

API  
1



```
$ python client.py $ENDPOINT_URL clear_database
```

## 13.6. Bashoutter GUI

API GUI (Figure 75)

CDK Public access mode S3 (HTML/CSS/JavaScript) AWS CLI S3

```
$ aws s3 cp --recursive ./gui/dist s3://<BUCKET_NAME>
```

Bashoutter (./gui/dist <BUCKET\_NAME> AWS



GUI Bashoutter Vue.js Vuetify UI  
Vue Single page application (SPA)  
handson/bashoutter/gui

Bashoutter.BucketUrl= URL (Figure 98) Public access mode S3 URL

S3 URL Figure 103

## Post your Haiku!

五

七

五

Your name

POST

# People's Haikus

REFRESH

Figure 103. "Bashoutter" GUI

`"API Endpoint URL"` の値を API Gateway に URL で渡す  
 (この例では API Gateway に URL を渡して GUI を更新する)に 対応する `"REFRESH"`  
 イベントは、`like` というイベント名で呼ばれる。

|             |        |             |        |             |           |
|-------------|--------|-------------|--------|-------------|-----------|
| <div></div> | "POST" | <div></div> | "POST" | <div></div> | "REFRESH" |
| <div></div> |        |             |        |             |           |

### 13.7. □□□□□□□□□□

**Bashoutter** の SNS に関するお問い合わせは [Section 13.5](#)

[illegible][illegible]

```
$ cdk destroy
```



CDK `destroy` 명령을 실행하면 S3 버킷도 삭제됩니다.

```
S3   "Actions" → "Delete"
```

```

0000000000000000 0000000000 <BUCKET NAME> 00000000 00000000
("BashoutterBucketXXXX" 00000000000000000000) 00000000000000

```

```
$ aws s3 rm <BUCKET NAME> --recursive
```

## 13.8.

Chapter 12 AWS

Lambda, S3, DynamoDB Chapter 13

"Bashoutter"

## Chapter 14. □□□

# Chapter 15. Appendix: 環境構築

この章では、本書で使用する環境の構築方法について説明します。AWS CLI、AWS CDK、Python、Docker、WSL、Linux、Mac、Windows、Windows Subsystem for Linux (WSL) のインストール方法について説明します。

環境構築

AWS

インストール方法

- AWS インストール (Section 15.1)
- AWS CLI インストール (Section 15.2)
- AWS CLI の使い方 (Section 15.3)
- AWS CDK のインストール (Section 15.4)
- WSL のインストール (Section 15.5)
- Docker のインストール (Section 15.6)
- Python venv のインストール (Section 15.7)
- Docker image のインストール (Section 15.8)

本書で使用する OS は Linux/Mac/Windows です。Windows の場合は Windows Subsystem for Linux (WSL) をインストールする必要があります (Section 15.5)。

本書で使用する Docker、AWS CLI/CDK、Python のインストール方法について説明します。Docker のインストール方法については Docker の公式ドキュメントを参照してください。

## 15.1. AWS インストール

本書で使用する AWS のインストール方法について説明します。AWS のインストール方法は、AWS の公式ドキュメントを参照してください。

本書で使用する AWS のインストール方法について説明します。AWS のインストール方法は、AWS の公式ドキュメントを参照してください。Create an AWS Account (Figure 104 参照)。

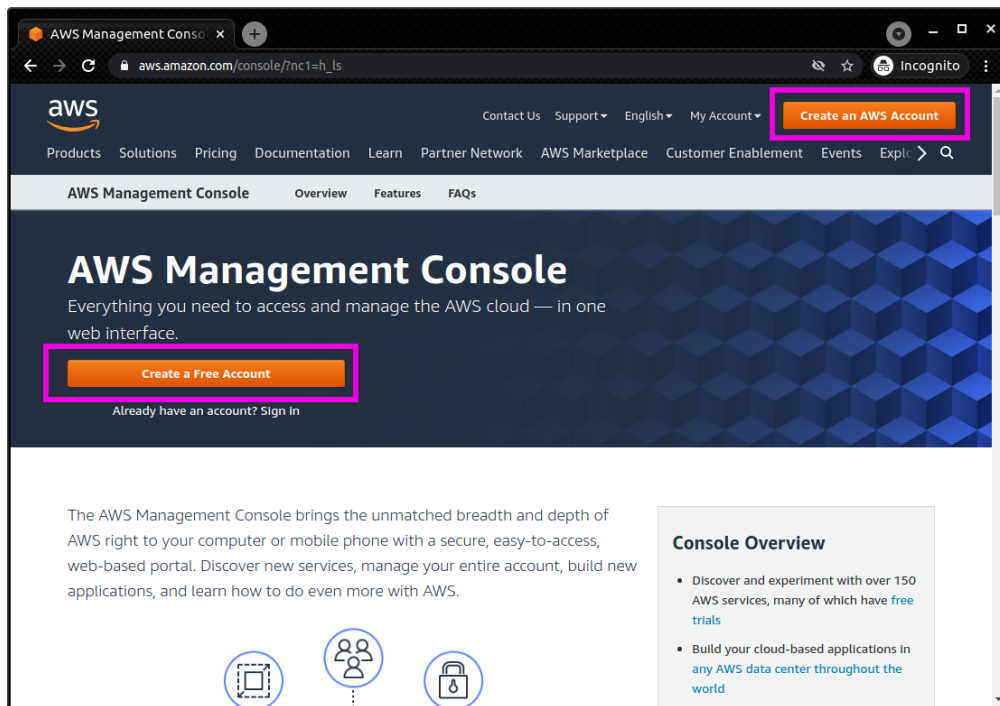


Figure 104. 스크린샷 (1): AWS 콘솔 화면

다음 단계를 진행합니다 (Figure 105)

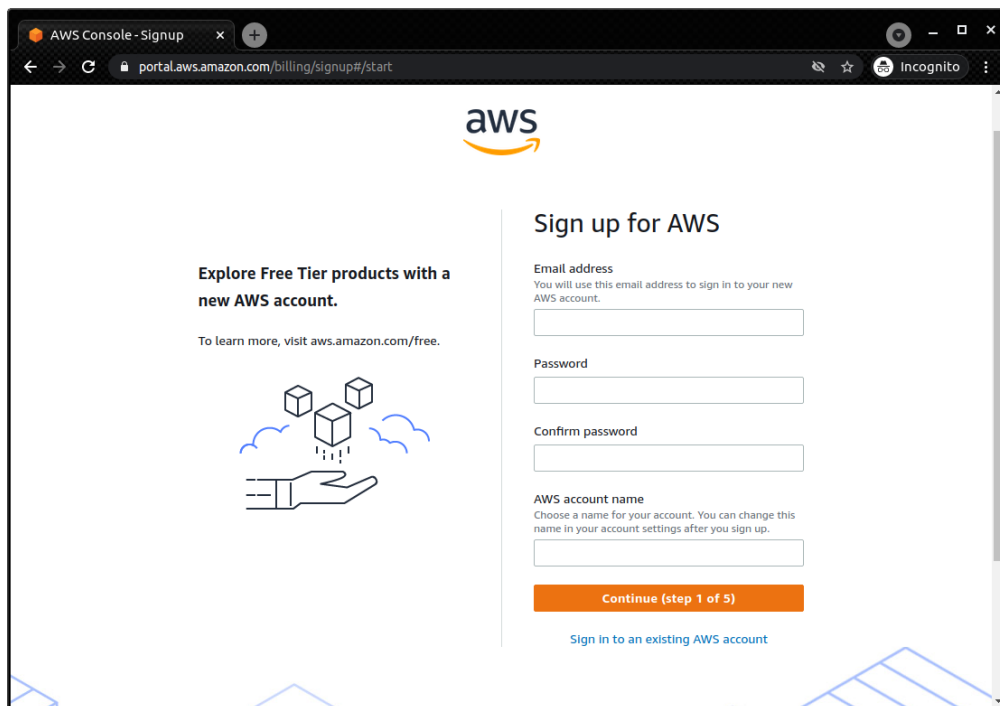


Figure 105. 스크린샷 (2): 회원가입 화면

다음 단계를 진행합니다 (Figure 106)

**Free Tier offers**

All AWS accounts can explore 3 different types of free offers, depending on the product used.

- Always free**  
Never expires
- 12 months free**  
Start from initial sign-up date
- Trials**  
Start from service activation date

**Sign up for AWS**

**Contact Information**

How do you plan to use AWS?

☐ Business - for your work, school, or organization

☐ Personal - for your own projects

Who should we contact about this account?

Full Name

Phone Number  
Enter your country code and your phone number.

Country or Region

Address

Figure 106. ステップ (3): 連絡先情報

ステップ (3) の「連絡先情報」画面で、AWS アカウントの作成を進めます。この画面では、AWS アカウントの作成を進めるための必要事項を入力します。

**Sign up for AWS**

**Billing Information**

Credit or Debit card number



AWS accepts all major credit and debit cards. To learn more about payment options, review our [FAQ](#)

Expiration date  
Month  Year

Cardholder's name

Billing address  
☒ Use my contact address  
1919-1 Tancha, Onna-son  
Kunigami-gun Okinawa 904-0412  
JP  
☐ Use a new address

**Secure verification**



 We will not charge for usage below AWS Free Tier limits. We temporarily hold \$1 USD/EUR as a pending transaction for 3-5 days to verify your identity.

Figure 107. ステップ (4): 支払い情報

ステップ (4) の「支払い情報」画面で、SMS 認証コードを入力します (Figure 108)。

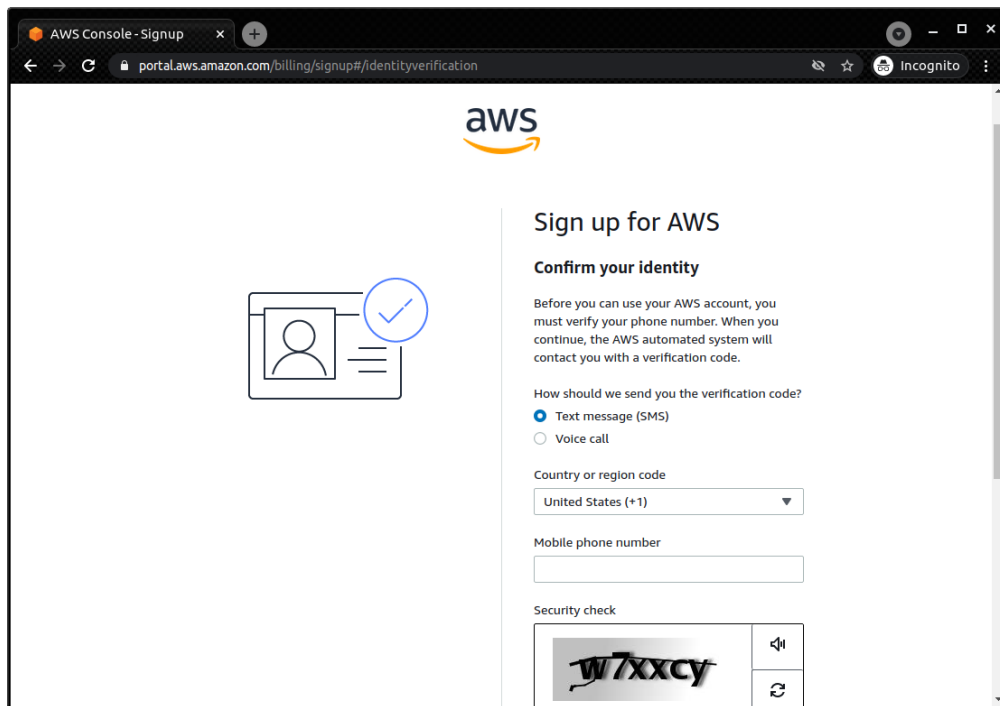


Figure 108. Step 5: Confirm your identity

After completing the identity verification step (Figure 109), you can choose the Basic support plan.

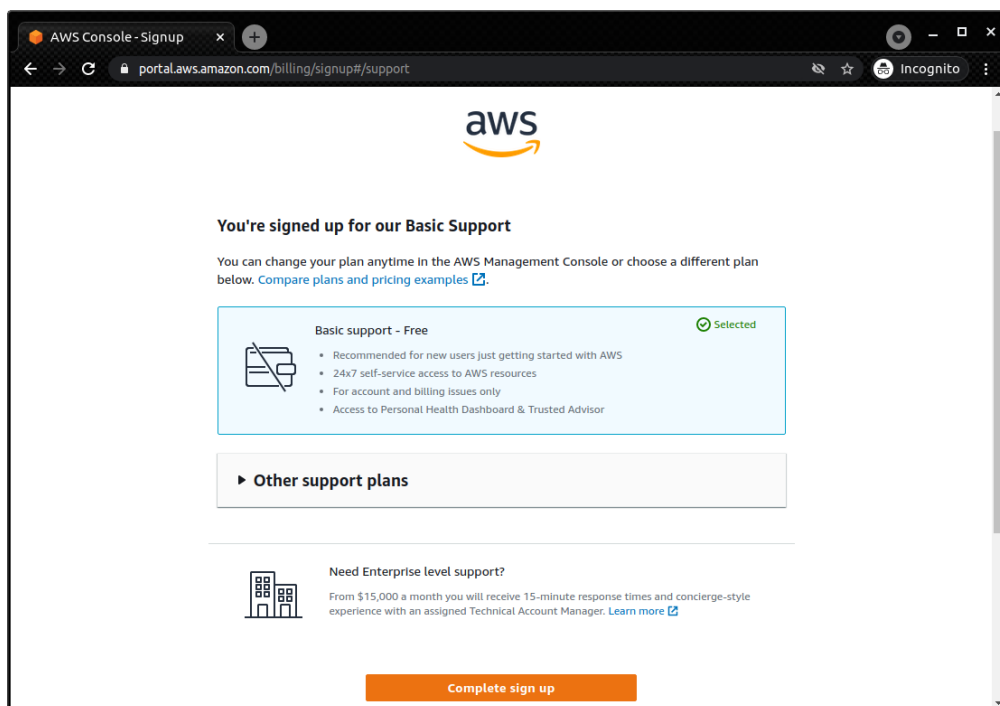


Figure 109. Step 6: Confirm your support plan

After completing the support plan selection step (Figure 110), you can complete the AWS account setup.

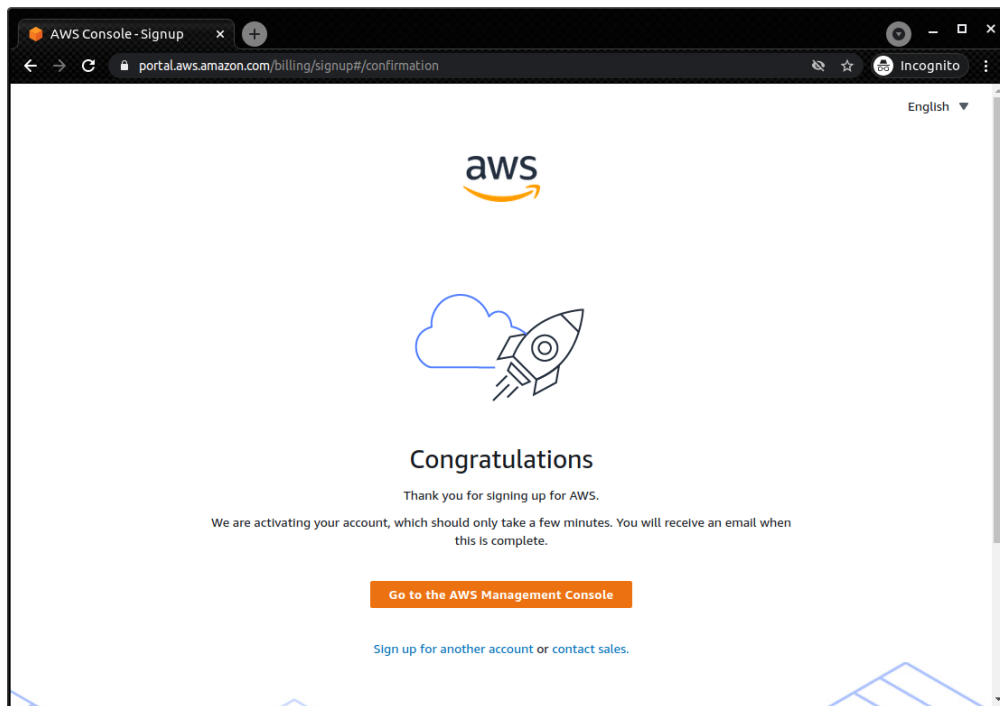


Figure 110. □□□□□ (7): □□□□□□□□□□□□

## 15.2. AWS □□□□□□□□□□

AWS のインストール AWS CLI のインストール AWS の API のインストール AWS CLI/CDK のインストール AWS のインストール ["Understanding and getting your AWS credentials"](#) のインストール

1. AWS 設定を完了する
2. 設定画面で "My Security Credentials" をクリック (Figure 111)
3. "Access keys for CLI, SDK, & API access" をクリックし "Create access key" をクリック (Figure 112)
4. Access key ID, Secret access key をコピー (パスワードとして保存する)
5. 設定を完了する
6. 設定ファイル `~/.aws/credentials` を編集して、コピーした Access key ID と Secret access key を入力する (参照 Section 15.3)

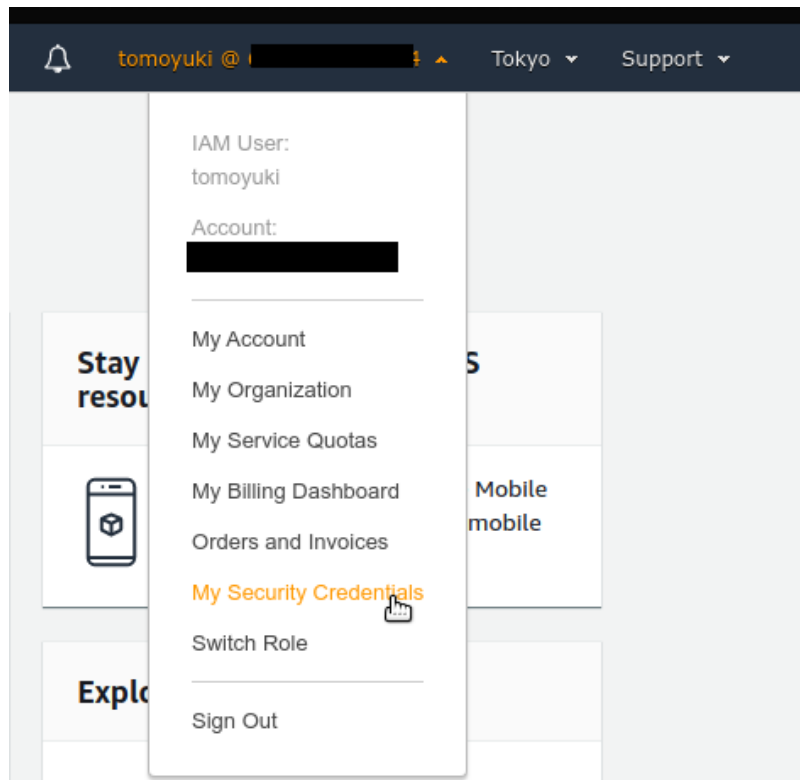


Figure 111. AWS 画面のユーザーメニュー

#### Access keys for CLI, SDK, & API access

Use access keys to make programmatic calls to AWS from the AWS Command Line Interface (AWS CLI), Tools for Windows PowerShell, the AWS SDKs, or direct AWS API calls. **If you lose or forget your secret key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.** [Learn more](#)

[Create access key](#)

Figure 112. AWS 画面のアクセスキー作成ボタン

#### AWS Educate Starter Account 画面の注意事項

- AWS Educate 画面の **vocareum** 画面 (Figure 113)
- **Account Details** 画面の **AWS CLI: Show** ボタン
- **aws\_access\_key\_id**, **aws\_secret\_access\_key**, **aws\_session\_token** の値 (Figure 114) を `~/.aws/credentials` ファイルにコピー (Section 15.3) し、**aws\_session\_token** を削除する
- `~/.aws/config` ファイルに **AWS Starter Account** の **us-east-1** リージョンを指定する



```
[default]
region = us-east-1
output = json
```

- `default` プロファイルを `default` として設定する (Section 15.3)

**Welcome to your AWS Educate Account**

AWS Educate provides you with access to a wide variety of AWS Services for you to get your hands on and build on AWS! To get started, click on the AWS Console button to log in to your AWS console.

Please read the FAQ below to help you get started on your Starter Account.

- What are the list of services supported?
- What regions are supported with Starter Accounts or Classroom Accounts?

**Your AWS Account Status**

- Active**  
full access ( )
- \$100**  
remaining credits (estimated)
- 1:47**  
session time

[Account Details](#) [AWS Console](#)

**AWS Access**  
 Session started at: 2021-06-20T18:29:05-0700  
 Session to end at: 2021-06-20T21:29:05-0700  
 Remaining session time: 2h18m12s

**AWS Starter account**  
 Term: 364 days 23:13:23

**AWS CLI:**  
 Copy and paste the following into ~/.aws/credentials

```
[default]
aws_access_key_id=
aws_secret_access_key=
aws_session_token=
```

Figure 113. vocareum

**AWS Access**  
 Session started at: 2021-06-20T18:29:05-0700  
 Session to end at: 2021-06-20T21:29:05-0700  
 Remaining session time: 2h18m12s

**AWS Starter account**  
 Term: 364 days 23:13:23

**AWS CLI:**  
 Copy and paste the following into ~/.aws/credentials

```
[default]
aws_access_key_id=
aws_secret_access_key=
aws_session_token=
```

Figure 114. vocareum AWS

## 15.3. AWS CLI

(Linux ) ( ) ( )

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
$ unzip awscliv2.zip
$ sudo ./aws/install
```

```
$ aws --version
```

( )

```
$ aws configure
```

AWS Access Key ID, AWS Secret Access Key Section 15.2  
 Default region name ( ap-northeast-1 = )  
 Default output format json

~~~~~ ~/.aws/credentials ~ ~/.aws/config~~~~~ cat  
~~~~~

```
$ cat ~/.aws/credentials
[default]
aws_access_key_id = XXXXXXXXXXXXXXXXXXXX
aws_secret_access_key = YYYYYYYYYYYYYYYYYY

$ cat ~/.aws/config
[profile default]
region = ap-northeast-1
output = json
```

~/.aws/credentials ~/.aws/config AWS CLI ~~~~~

~~~~~ [default] ~~~~~ default ~~~~~ [myprofile]  
~~~~~

AWS CLI ~~~~~

```
$ aws s3 ls --profile myprofile
```

~~~~~ --profile ~~~~~

~~~~~ --profile ~~~~~ AWS\_PROFILE ~~~~~

```
$ export AWS_PROFILE=myprofile
```

~~~~~

```
export AWS_ACCESS_KEY_ID=XXXXXX
export AWS_SECRET_ACCESS_KEY=YYYYYY
export AWS_DEFAULT_REGION=ap-northeast-1
```

~~~~~ ~/.aws/credentials ~~~~~ ( )



**AWS Educate Starter Account** us-east-1 ~~~~~ ( ) AWS  
Educate Starter Account ~~~~~ default region us-east-1 ~~~~~

## 15.4. AWS CDK ~~~~~

~~~~~ (Linux ) ~~~~~ ~~~~~  
~~~~~

Node.js ~~~~~

```
$ sudo npm install -g aws-cdk
```



API version	CDK version
1.100.0	1.100.0

```
$ npm install -g aws-cdk@1.100
```

~~~~~

```
$ cdk --version
```

~~~~~

```
$ cdk bootstrap
```



`cdk bootstrap` ~~~~~AWS~~~~~ ~~~~~ `~/.aws/config` ~~~~~  
~~~~~ [Section 15.3](#) ~~~~~



AWS CDK ~~~~~ AWS CLI ~~~~~ [Section 15.3](#) ~~~~~

15.5. WSL ~~~~~

~~~~~ AWS CLI ~~~~~ Python ~~~~~ UNIX ~~~~~ Linux ~ Mac ~ OS ~~~~~ Windows ~~~~~ [Windows Subsystem for Linux \(WSL\)](#) ~~~~~ Linux ~~~~~ [Cygwin](#) ~ Linux ~~~~~ WSL ~~~~~

WSL ~ Windows ~ OS ~ Linux ~~~~~ Microsoft ~~~~~ Ubuntu ~~~~~ Linux distribution ~~~~~ Linux ~~~~~

~~~~~ [WSL 2](#) ~~~~~ WSL 2 ~~~~~ [~~~~~](#) ~~~~~

~~~~~ OS ~ Windows 10 (Pro ~ Home ~~~~~) ~~~~~ Windows 10~~~~~WSL~~~~~ X64 ~~~~~ Version 1903, Build 18362 ~~~~~ Windows ~~~~~

~~~~~ Administrator ~ PowerShell ~~~~~ (Figure 115)~ Windows ~~~~~ powershell ~~~~~ PowerShell ~~~~~ Run as administrator ~~~~~

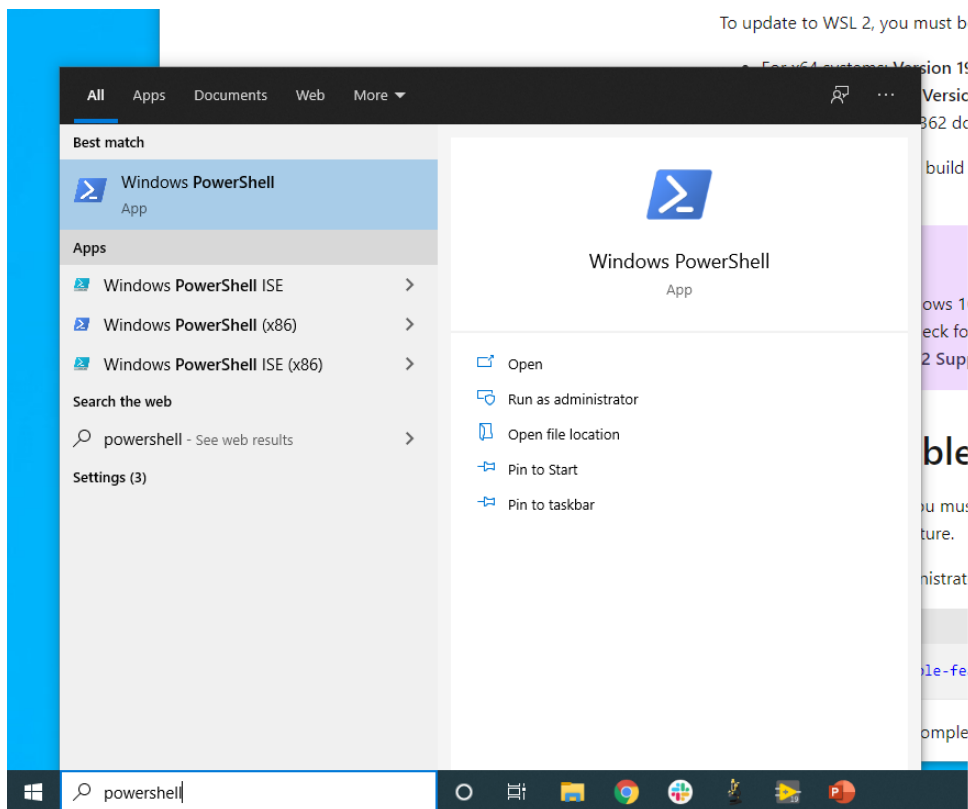


Figure 115. Windows PowerShell search

PowerShell installation steps

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
```

“The operation completed successfully.” message appears, indicating WSL is enabled.

Administrator PowerShell command

```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

“The operation completed successfully.” message appears, indicating Virtual Machine Platform is enabled.

Linux kernel update package can be downloaded from https://wslstorestorage.blob.core.windows.net/wslblob/wsl_update_x64.msi

Download the Linux kernel update package and install it.

PowerShell command

```
wsl --set-default-version 2
```

Linux distribution can be installed using Ubuntu 20.04 LTS

Microsoft store Ubuntu Ubuntu 20.04 LTS “Get”

Figure 116) Ubuntu 20.04

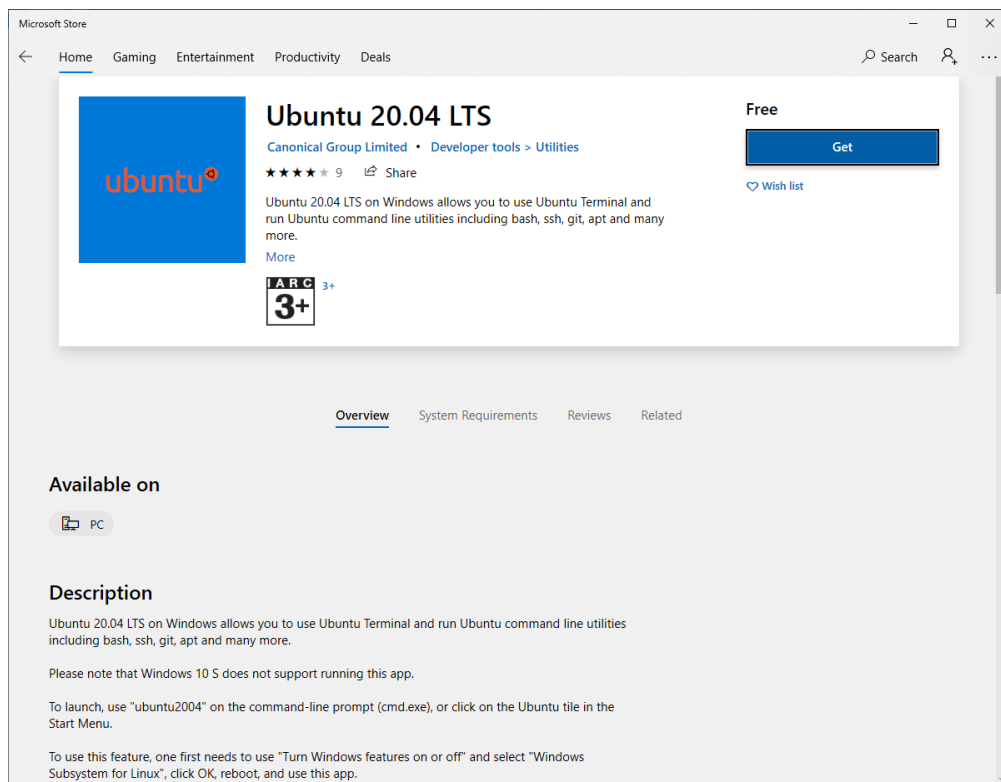


Figure 116. Microsoft store Ubuntu 20.04

Ubuntu 20.04

WSL2 Windows Ubuntu Ubuntu 20.04

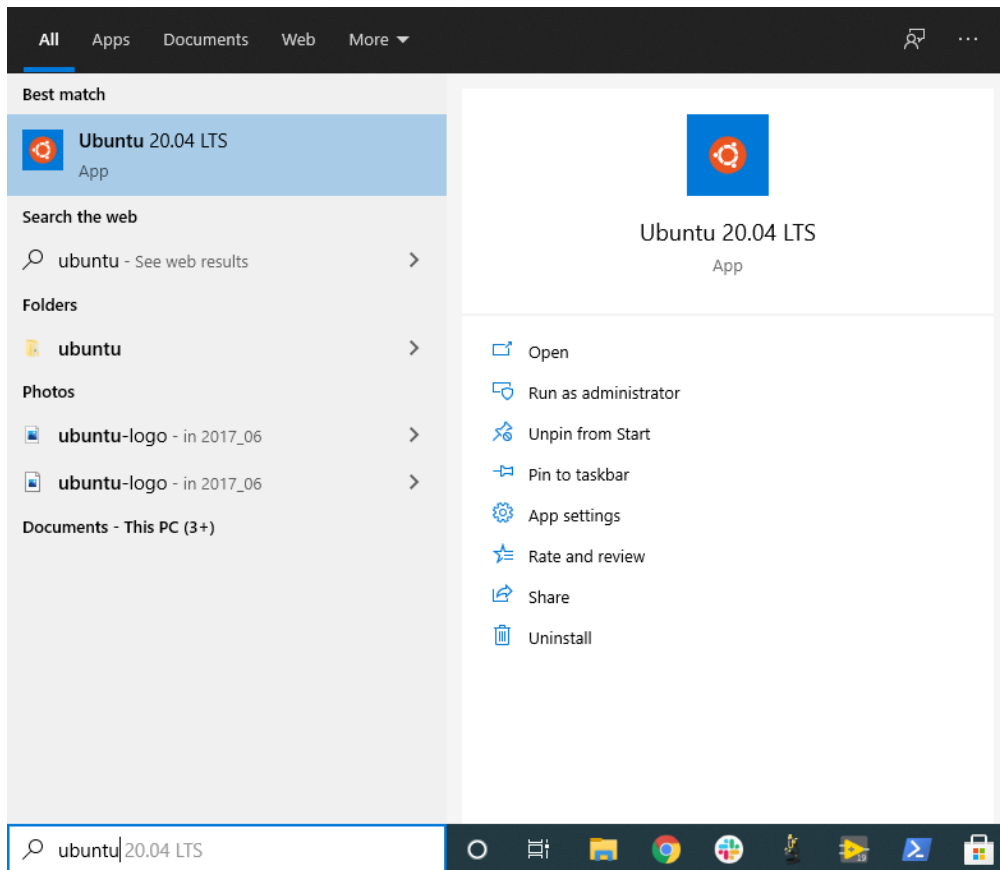


Figure 117. Ubuntu 20.04 アプリ

コマンドプロンプトで `ls, top` を実行すると WSL のコマンドプロンプトが表示されます。

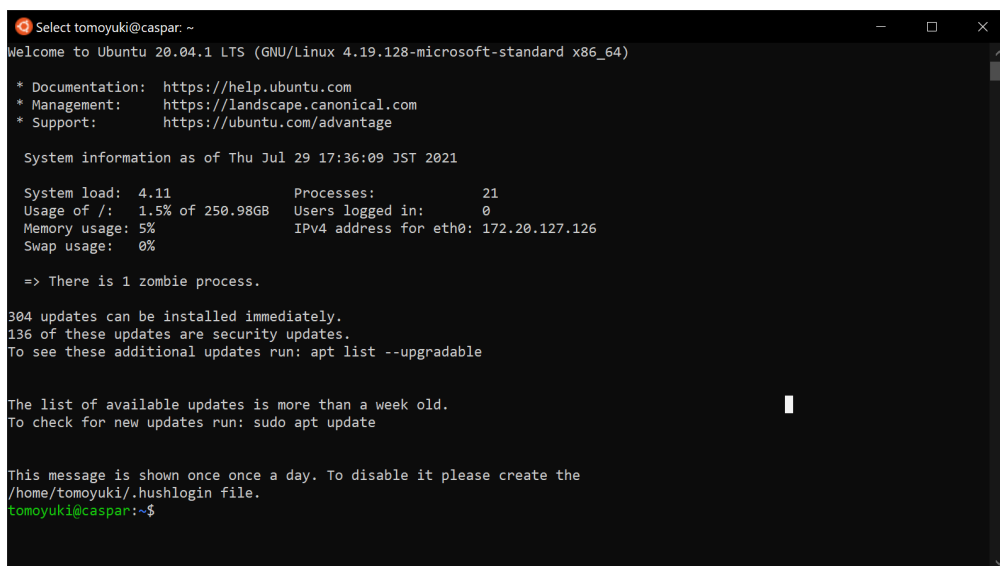


Figure 118. WSL コマンド

Windows Terminal を起動すると WSL のコマンドプロンプトが表示されます。

15.6. Docker のインストール

Docker のインストールは OS によって異なります。

Mac の場合は Docker Desktop をインストールします。Windows の場合は Docker Desktop をインストールします。

Applications をインストールする。 Docker Desktop をインストールする。

Windows Docker Desktop をインストールする。 WSL 2 をインストールする。 Docker Desktop をインストールする。 WSL 2 docker をインストールする。

Linux (Ubuntu) をインストールする。 Docker Desktop をインストールする。

Docker をインストールする。 Docker をインストールする。

```
$ curl -fsSL https://get.docker.com -o get-docker.sh
$ sudo sh get-docker.sh
```

root として docker をインストールする。 sudo を使用する。 docker をインストールする。 (Post-installation steps for Linux)。

docker をインストールする。 docker をインストールする。

```
$ sudo groupadd docker
```

docker をインストールする。

```
$ sudo usermod -aG docker $USER
```

docker をインストールする。

docker をインストールする。

```
$ docker run hello-world
```

sudo を使用する。

15.7. Python venv

numpy, scipy をインストールする。 Python をインストールする。

Python をインストールする。 venv, pyenv, conda をインストールする。

venv Python, pyenv, conda をインストールする。

venv をインストールする。

```
$ python -m venv .env
```

このコマンドを実行すると、`.env/` ディレクトリが作成され、その中に Python の仮想環境が作成されます。

仮想環境を有効化します。

```
$ source .env/bin/activate
```

確認します。

仮想環境の有効化を確認します (`.env`)。仮想環境の有効化が完了すると、`(.env)` が表示されます (Figure 119)。

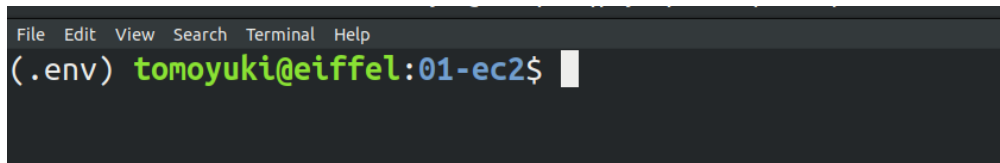


Figure 119. `venv` の有効化

仮想環境が有効化された状態で `pip` を実行すると、`.env/` ディレクトリに `requirements.txt` が作成されます。

Python の `requirements.txt` を作成します。このファイルは、仮想環境にインストールするパッケージを指定するためのファイルです。

```
$ pip install -r requirements.txt
```

仮想環境が有効化された状態で Python を実行すると、仮想環境が有効化されていることが確認できます。



`venv` は、仮想環境を作成するためのコマンドです。このコマンドを実行すると、`.env` ディレクトリが作成されます。

15.8. Docker image の作成

このセクションでは、Node.js、Python、AWS CDK を使用して Docker image を作成する方法について説明します。



このセクションでは、Docker image を作成する方法について説明します。Docker image は、アプリケーションを実行するための環境を提供するためのファイルです。

Docker image を作成するには、[Docker Hub](#) に登録する必要があります。Docker image を作成するためのコマンドは、GitHub の [docker/Dockerfile](#) を参照してください。

このコマンドを実行すると、Docker image が作成されます。

```
$ docker run -it tomomano/labc:latest
```

このコマンドを実行すると、Docker Hub から Docker image をプル (pull) して実行されます。

コンテナ環境で作業するための準備 (コンテナ `-it` を使用して作業する)

```
root@aws-handson:~$
```

コンテナ `ls` を実行すると `handson/` ディレクトリが存在することを確認。 `cd` で移動

```
$ cd handson
```

コンテナ環境で作業するための準備

コンテナ環境で作業するための準備 `virtualenv` をインストール (Section 4.4 を参照)
コンテナ環境で作業するための準備 `virtualenv` をインストール

AWS の設定は Section 15.3 を参照。 `AWS_ACCESS_KEY_ID` と `AWS_SECRET_ACCESS_KEY` を
コンテナ環境で作業するための準備 `~/.aws/credentials` に設定
コンテナ環境で作業するための準備

```
$ docker run -it -v ~/.aws:/root/.aws:ro tomomano/labc:latest
```

コンテナ環境で作業するための準備 `~/.aws` を `/root/.aws` にマウント (`:ro` は `read-only`)
コンテナ環境で作業するための準備 `read-only` を指定



`/root/` ディレクトリはコンテナ環境で作業するための準備。 `SSH` を使用して作業する。

Chapter 16. 謝辞

本書の作成に協力いただいた方々に感謝いたします。

2021年 contributors

- 山田 太郎 - 校正・校閲

2020年 contributors

- 山田 太郎 - Docker 環境構築
- 山田 太郎 - 校正・校閲
- @shuuji3 - MR !15
- @takatama_jp - MR !14

本書は [Asciidoctor](#) で作成されています。

本書のソースコードは [GitHub](#) に公開されています。 [Issues](#) を利用してご報告ください。 [Pull request](#) をお待ちしております。

Chapter 17. 閉鎖型

Tomoyuki Mano (Tomoyuki Mano)

本書は、(閉鎖型) 2021年 (PD) (OIST) によって作成された。本書は、(OIST) によって作成された。

tomoyukimano@gmail.com

GitHub <https://github.com/tomomano>

Chapter 18. 著作権

本書は [CC BY-NC-ND 4.0](#) のライセンスで提供されています

本書の著作権は著者に帰属します。本書の複製、転載、配布は禁止されています。



```
<style>
  .imageblock > .title {
    text-align: inherit;
  }
</style>
```